

Silent Circuit Relinearisation: Sublinear-Size (Boolean and Arithmetic) Garbled Circuits from DCR

Pierre Meyer¹, Claudio Orlandi¹, Lawrence Roy¹, and Peter Scholl¹

Aarhus University, Aarhus, Denmark

{pierre.meyer,orlandi,peter.scholl}@cs.au.dk, ldr709@gmail.com

Abstract. We introduce a general template for building garbled circuits with low communication, assuming decisional composite residuosity (DCR) and a circular security assumption. For the case of layered Boolean circuits, we can garble a circuit of size s with communication proportional to $O(s/\log \log s)$ bits, plus an additive factor that is polynomial in the security parameter. For layered arithmetic circuits with B -bounded integer computation, we obtain a similar result: the garbled arithmetic circuit has size $O(s/\log \log s) \cdot (\lambda + \log B)$ bits, where λ is the security parameter. In both cases, we can remove the circular security assumption by adding a term proportional to the circuit depth. These are the first constructions of general-purpose, garbled circuits with sublinear size, without relying on heavy tools like indistinguishability obfuscation or attribute-based and fully homomorphic encryption.

To achieve these results, our main technical tool is a new construction of a form of homomorphic secret sharing (HSS) where some of the inputs are semi-private, that is, known to one of the evaluating parties. Through a new relinearisation technique that allows performing arbitrary additions and multiplications on semi-private shares, we build such an HSS scheme that supports evaluating any function of the form $C(x) \cdot C'(y)$, where C is any polynomially-sized circuit applied to the semi-private input x , and C' is a restricted-multiplication (or, NC1) circuit applied to the private input y . This significantly broadens the expressiveness of known HSS constructions.

Table of Contents

1	Introduction	3
1.1	Our Contributions	3
1.2	Related Work	4
1.3	Concurrent Work	5
2	Technical Overview	6
3	Preliminaries	9
3.1	Damgård-Jurik Cryptosystem	9
3.2	Damgård-Jurik-ElGamal Cryptosystem	10
3.3	Distributed Discrete Logarithm	11
3.4	Models of computation	12
4	(Semi-Private) Offline-Online HSS	13
4.1	Definition	14
4.2	Semi-private HSS construction from KDM-security of Damgård-Jurik	15
4.3	(Weakly succinct) semi-private HSS construction from DCR	19
4.3.1	Relinearisation, without a circular security assumption.	19
4.3.2	Key-switching.	20
4.3.3	HSS construction from DCR.	21
5	Sublinear-Size Garbled Circuits	25
5.1	Wide-gates and Masking Schemes	26
5.2	General Template for Rate-1 Garbling from Offline-Online HSS	27
5.2.1	From semi-private HSS.	27
5.2.2	From semi-private HSS with linear offline/online shares.	31
5.3	Instantiating the Template and Applications to Sublinear-Size Garbling	34
A	Offline-Online HSS Proofs	38
A.1	From KDM-security of Damgård-Jurik	38
A.1.1	Correctness and authenticated correctness.	38
A.1.2	Security against party $\sigma = 0$	44
A.1.3	Security against party $\sigma = 1$	44
A.2	From DCR	44
A.2.1	Correctness.	45
A.2.2	Key-switching security.	50
A.2.3	Security against party $\sigma = 0$	51
A.2.4	Semi-private security against party $\sigma = 1$	51
B	Deferred Material on Masking	52
B.1	Boolean Truth Tables	52
B.2	Multivariate Polynomials	53

1 Introduction

Garbled circuits, introduced by Yao [Yao82], are a cryptographic tool used to evaluate a circuit on private inputs. Concretely, a garbling scheme consists of an algorithm **Garble**, which takes a circuit C and outputs a garbled circuit $\widehat{C}$ together with some input encoding functions K_i , for the i -th input. There is also an evaluation algorithm, **Eval**, which on input $\widehat{C}$ and the encodings $K_i(x_i)$ for some inputs $x_1, \dots, x_n$, produces a result $y = C(x_1, \dots, x_n)$. For security, it is required that the garbled circuit $\widehat{C}$ and garbled inputs $K_i(x_i)$ reveal nothing about the inputs or circuit besides y , and some structural information about C .

Garbled circuits are a valuable tool in secure two-party and multi-party computation with constant round complexity, and also enjoy usage in a range of other cryptographic applications. One limitation of most garbled circuit constructions is their bandwidth complexity. Typically, for Boolean circuits, the garbled circuit $\widehat{C}$ has size $O(\lambda)$ times larger than the number of gates in C , for security parameter λ , which introduces a large bandwidth overhead in applications where a garbled circuit needs to be sent over a network. There has been much effort to reduce this complexity over the years, with the state-of-the-art construction based on symmetric cryptography being the “three halves” technique [RR21]. This obtains a size of $\approx 1.5\lambda$ bits per AND gate, and zero bits per XOR gate in the circuit.

For more specialized circuits with components such as branches or lookup tables, techniques such as stacked garbling [HK20, HK21b] and one-hot garbling [HK21a, HKN24] can be used to obtain improvements over naively expressing the components in terms of XOR and AND gates. However, these do not lead to noticeable asymptotic improvements for general, Boolean circuits.

If garbling an arithmetic circuit instead of a Boolean circuit, approaches based on symmetric cryptography are typically less efficient, with the state-of-the-art having a size of $O(\lambda\ell)$ bits per garbled multiplication gate of ℓ -bit integers. Other approaches based on linearly homomorphic encryption [AIK11, BLLL23] can obtain a similar or better communication complexity. Recently, [MORS24] presented a new approach to garbling arithmetic circuits, using techniques drawn from homomorphic secret sharing [BGI16, OSY21, RS21]. This led to a rate-1 arithmetic garbling scheme, where each multiplication gate can be garbled with a cost of sending one Damgård-Jurik ciphertext, which has size roughly the same as the maximum size of any wire value when evaluating the arithmetic circuit over the integers. Notably, this was the first rate-1 construction relying solely on a group-theoretic (factoring-related) assumption.

Succinct Garbled Circuits: Overcoming the Circuit Size Barrier. Several works have shown how to construct *succinct* garbled circuits, where if C is publicly known, then the size of the garbling $\widehat{C}$ can be *sublinear* in the size of C . This was shown to be possible under the learning with errors assumption with subexponential modulus-to-noise ratio [GKP⁺13], with later improvements in [BGG⁺14, QWW18]. In the latter construction, the size of the garbled circuit scales polynomially with the depth of the circuit, but not its size. More recently, this dependence on the depth has been removed by relying on LWE with an additional circular security assumption [HLL23]. However, in addition to making strong assumptions, all of these constructions rely on complex primitives such as fully homomorphic encryption and attribute-based encryption. So far, it has not been known how to achieve sublinear-size garbled circuits without such machinery.

1.1 Our Contributions

In this work, we introduce a new approach to garbling Boolean and arithmetic circuits with sublinear size, under the decisional composite residuosity (DCR) assumption. We obtain the

following main results on the communication complexity of garbling layered circuits¹.

Theorem 1. *Assuming DCR, there exists a garbling scheme for layered Boolean circuits, where the size of the garbled circuit and labels is $O(s/\log \log s + m) + (n + D + 1) \cdot \text{poly}(\lambda)$, for a circuit with size s , depth D , n inputs and m outputs.*

For arithmetic circuits, as in several prior works [BLLL23, MORS24], we consider a model of bounded integer computation, where there is a known bound $B \in \mathbb{N}$ such that, for all supported inputs to the circuit, the magnitude of all wire values during evaluation is at most B . Previously, [MORS24] showed how to garble arithmetic circuits with $\approx \log B$ bits per multiplication gate (for large B) under a circular security assumption, or $\approx 2 \log B$ bits under DCR. We instead obtain $O(\log B / \log \log s)$ bits per gate, for layered circuits.

Theorem 2. *Assuming DCR, there exists a garbling scheme for layered arithmetic circuits supporting B -bounded integer computation, where the size of the garbled circuit and labels is $O(s/\log \log s + m)(\lambda + \log B) + (n + D + 1)\text{poly}(\lambda, \log B)$, for a circuit with size s , depth D , n inputs and m outputs.*

We remark that in both theorems, the dependence on the circuit depth D can be removed by making an additional circular security assumption for the Damgård-Jurik encryption scheme.

Main Tool: a Relinearisation Technique for Semi-Private Homomorphic Secret Sharing. Like the recent work of [MORS24], our work builds a garbling scheme using techniques from homomorphic secret sharing, and in fact, we achieve our results by introducing a new *relinearisation* tool that allows to significantly expand the class of functions supported by certain homomorphic secret sharing schemes. Concretely, we use this to build a form of *semi-private* HSS scheme for two servers, where one portion of the inputs to the function being evaluated must be known to one of the servers. Our scheme can support homomorphic evaluation of functions of the form

$$C(\vec{x}) \cdot C_{\text{rm}}(\vec{y})$$

where $\vec{x}$ is the semi-private input known to one server, $\vec{y}$ is the private input, C is any circuit of polynomial size, and C_{rm} is a “restricted multiplication circuit”, roughly equivalent to a log-depth circuit. Additionally, our scheme has an *offline/online* property that we leverage to build garbled circuits, where one server’s share of the semi-private input $\vec{x}$ can be sampled ahead of time, independently of the input $\vec{x}$.

Most classic HSS constructions [BGI16, OSY21, RS21] only support the evaluation of functions of the form $C_{\text{rm}}(\vec{y})$. One exception is the work of [CMPR23], which implicitly builds a semi-private, offline-online HSS for functions of the form $C_{\text{rm}}(\vec{x}) \cdot C_{\text{rm}}(\vec{y})$, as part of their construction of constrained pseudorandom functions. Our construction allows to significantly expand the class of functions that can be evaluated on the semi-private input, and we believe this result and the relinearisation technique will be useful in other applications beyond garbling.

1.2 Related Work

In recent, prior (but independent) work, two other papers have proposed constructions of garbled circuits with small size.

¹ Recall that in a layered circuit, gates can be partitioned into layers L_i , such that any circuit wire goes from a gate in some layer L_i to one in the next layer L_{i+1} .

Garbling with 1 Bit Per Gate [LWYY24]. Liu, Wang, Yang and Yu [LWYY24] show how to garble Boolean circuits with 1 bit per gate, using homomorphic encryption based on either ring-LWE or NTRU, plus a KDM security assumption. Although their assumptions are very different, under the hood, their construction appears similar to our general garbling template when instantiated for boolean circuits, except they use a homomorphic encryption scheme (used to homomorphically evaluate a PRG) instead of homomorphic secret sharing based on DCR, as we do.

Succinct Partial Garbling [ILL24]. Ishai, Li, and Lin [ILL24] also provide novel techniques for succinct garbling schemes. Like us, they take inspiration from the recent work of [MORS24] using HSS to garble arithmetic circuits. They introduce a novel algebraic technique for non-interactive computation of authenticated shares using DDLog, which is quite similar to the relinearisation we do for semi-private HSS. Unlike us, however, they abstract it into an algebraic homomorphic MAC (aHMAC) that can be applied to other problems, while our presentation is specific to semi-private HSS for the tensor product circuit class. One such problem is constrained PRFs – from the aHMAC they construct constrained PRFs for all circuits, while previous work only achieved NC1 [CMPR23]. They also construct a $1/\text{poly}(\lambda)$ correctness error aHMAC using prime-order groups, while we only consider DCR groups where negl correctness error is possible.

Their main application of the aHMAC is succinct partial garbling, which allows the garbling of circuits of the form $C_{\text{priv}}(C_{\text{pub}}(x), y)$ with communication that only depends on the size of the *private* circuit C_{priv} (times a polynomial factor in the security parameter), and crucially is independent of the size of the *public* circuit C_{pub} and public input x . Such a notion of garbling is clearly related to our notion of semi-private HSS: in partial garbling, some of the inputs are known to the evaluator; in semi-private HSS, some of the inputs are known to one of the parties. There are two significant differences, however: (1) our class of circuits is more restricted, being $C_{\text{priv}}(x) \cdot C_{\text{pub}}(y)$ where C_{priv} is an RMS program; and (2) in HSS, the communication cost is independent of *both* the public and private circuits (except perhaps a factor related to the depth) as well as the output size. This allows HSS to have a greater level of succinctness, while partial garbling can have greater generality.

One application of this generality is decrypting homomorphic encryption ciphertexts. [ILL24] runs linearly homomorphic encryption (from DCR or DDH) inside C_{pub} , then decrypts the result C_{priv} , achieving *full-succinctness* for a limited (yet useful) classes of functions, including bounded length branching programs and truth tables. In comparison, our garbling schemes achieve *limited succinctness* for *any* layered circuit.

1.3 Concurrent Work

Succinct Garbling from HSS. In a concurrent work, [ILL25] construct garbling schemes using similar techniques and constructions to our own. Their boolean garbling scheme achieves the same $O(s/\log \log s)$ communication complexity for layered boolean circuits as we do. Their arithmetic garbling scheme is targeted at circuits working over a ring $\mathbb{Z}/B\mathbb{Z}$, using a CRT technique to handle large moduli, while ours is targeted at the weaker class of B -bounded integer arithmetic circuits for $B \gg 2^\lambda$. However, our arithmetic garbling is more compact, using only $O(s \cdot (\lambda + \log(B))/\log \log s)$ bits of communication for a layered circuit of s gates, while theirs uses $O(s \cdot \log(B))$ bits of communication for any circuit.

A major difference from our work is their assumptions. [ILL25] provides constructions from a variety of assumptions: power-ring-LWE and power-DDH (over either prime-order or DCR groups), as well as KDM variants thereof. We focus on just DCR and a KDM variant of DCR; the former is a well-established assumption that is widely used for homomorphic encryption and HSS, and the latter is a variant that appears naturally in HSS constructions

like [MORS24]. In both works, the garbling schemes constructed from non-KDM assumptions require key switching, adding cost proportional to circuit depth. However, while their aHMAC from power-DDH needs to key switch after every multiplication, our semi-private HSS from DCR can evaluate an entire RMS program before needing to change keys, due to its dependence on the *pebbling depth* (definition 13) instead of circuit depth. Thus, our non-KDM version can be more communication efficient, as it switches between fewer keys.

Finally, [ILL25] directs some attention to concrete efficiency, and provides concrete estimates for the setup communication of their schemes. We only provide asymptotic bounds, and leave questions of concrete efficiency to future work.

2 Technical Overview

Secure Computation with DDLogs. For this technical overview, let’s pretend that $\text{Enc}(x) = g^x$ is a secure encryption of x that can be decrypted with key sk (of course, this is not a secure encryption and cannot be decrypted — later, we actually use secure variants of the Damgård-Jurik cryptosystem [DJ01] where the plaintext appears “in the exponent”).

Now, following Roy-Singh [RS21], suppose two parties have subtractive shares $\langle \text{sk} \rangle$ of the secret key, that is, shares $\langle \text{sk} \rangle_0$ and $\langle \text{sk} \rangle_1$ such that $\text{sk} = \langle \text{sk} \rangle_1 - \langle \text{sk} \rangle_0$. There is a *distributed discrete logarithm* procedure DDLog , which allows them to non-interactively compute *authenticated shares* of x as $\langle \text{sk} \cdot x \rangle = \text{DDLog}((g^x)^{\langle \text{sk} \rangle})$ (that is, party σ , for $\sigma \in \{0, 1\}$, computes the share $\langle \text{sk} \cdot x \rangle_\sigma = \text{DDLog}((g^x)^{\langle \text{sk} \rangle_\sigma})$). Here, authenticated refers to the fact that the shares are multiplied by the secret key of the underlying cryptosystem. Moreover, thanks to the linear property of subtractive shares, it is also the case that if the parties have authenticated shares $\langle \text{sk} \cdot y \rangle$ of some value y then they can compute authenticated shares of xy as $\langle \text{sk} \cdot xy \rangle = \text{DDLog}((g^x)^{\langle \text{sk} \cdot y \rangle})$. This ability to non-interactively perform multiplication between encrypted values and secret-shared values has been used in several previous works to build homomorphic secret sharing (HSS) [BGI16, OSY21, RS21], constrained PRFs [CMPR23], and more. One apparent limitation of this technique is that it only allows to perform multiplications between encrypted values and shared values – this leads to the *restricted multiplication straight-line (RMS)* model of computation, where one can perform addition between any values, but multiplications are restricted to only take one *memory value* input (e.g., the output of an internal gate in authenticated secret-shared form), and an *input value* (a value which was input directly by one of the parties in encrypted form). Indeed, despite several years of progress in this area, it is unclear how to move beyond this restricted class of programs.

It is indeed very challenging to non-interactively perform multiplications of secret-shared values. This holds even for restricted cases, for instance where one of the values is known to one party. To be concrete, MPC protocols based on *linearly-homomorphic encryption* (LHE) allow a party that knows a plaintext value y and an encrypted value $\text{Enc}(x)$ to non-interactively compute an encryption of $\text{Enc}(xy)$. Note that there is no known analogue for protocols based on secret-sharing: suppose the parties hold a secret sharing $\langle x \rangle$ and one of the two parties know a plaintext value y . There is currently no known way to non-interactively let the parties derive a sharing $\langle xy \rangle$.

Local Multiplication and Relinearisation of Authenticated Shares. The first and main technical insight of our work is that it is possible to do a form of local multiplication on shared values, followed by a “relinearisation” technique, allowing parties to non-interactively compute authenticated shares of the result of a multiplication xy when one of the parties knows x and y .

Suppose the parties have shares $\langle \text{sk} \cdot x \rangle$ and $\langle \text{sk} \cdot y \rangle$, and party P_1 knows x, y . We can view party P_0 ’s shares as evaluations of a linear function in the secret key, given by:

$$\langle \mathbf{sk} \cdot x \rangle_0 = p_x(\mathbf{sk}) := \langle \mathbf{sk} \cdot x \rangle_1 - \mathbf{sk} \cdot x, \quad \langle \mathbf{sk} \cdot y \rangle_0 = p_y(\mathbf{sk}) := \langle \mathbf{sk} \cdot y \rangle_1 - \mathbf{sk} \cdot y$$

Since P_1 knows x, y as well as its shares, then it knows the coefficients of the above polynomials p_x, p_y .

Then, multiplying the polynomials, P_1 can compute the coefficients of the *degree-two* polynomial $p_x \cdot p_y$, given by $\langle \mathbf{sk} \cdot x \rangle_1 \cdot \langle \mathbf{sk} \cdot y \rangle_1 + C \cdot \mathbf{sk} + xy \cdot \mathbf{sk}^2$, where C depends on P_1 's shares and the values x, y . Meanwhile, P_0 can compute its evaluation of this polynomial at $\mathbf{sk}$, denoted z . Intuitively, this setup can now be seen as a special form of *degree-two* authenticated secret sharing under the key $(\mathbf{sk}, \mathbf{sk}^2)$. Indeed, so far, this observation is exactly the same as one from recent zero-knowledge proof systems based on vector oblivious linear evaluation [DIO21, YSWW21]. However, these ZK proof protocols all require a step of interaction, in order to “relinearise” the shares back to degree one.

We observe that, with the help of DDLog, this relinearisation can be done entirely non-interactively. First, subtracting P_1 's constant coefficient from P_0 's evaluation, we get:

$$\langle \mathbf{sk} \cdot x \rangle_1 \cdot \langle \mathbf{sk} \cdot y \rangle_1 - z = -C \cdot \mathbf{sk} - xy \cdot \mathbf{sk}^2 = (-C - xy \cdot \mathbf{sk}) \cdot \mathbf{sk}$$

Viewing the above as an authenticated sharing of $-C - xy \cdot \mathbf{sk}$, notice that if we provide the parties with an encryption of the inverse of the secret key $\text{Enc}(\mathbf{sk}^{-1}) = g^{\frac{1}{\mathbf{sk}}}$, they can use DDLog to “remove” a layer of authentication, obtaining simply a sharing of $-C - xy \cdot \mathbf{sk}$. Finally, since party P_1 knows C , it can remove this from its share so that both parties end up with an authenticated sharing $\langle xy \cdot \mathbf{sk} \rangle$ as desired.

This trick is extremely powerful, as it allows us for the first time to compute authenticated shares of arbitrary circuits for the parts of the computation where the inputs are known to one of the parties.

We call this kind of homomorphic secret-sharing where one of the parties knows the plaintext values *semi-private HSS*. We note that there are related notions in the literature for garbling schemes (e.g., privacy-free garbling [FNO15], partial garbling [IW14], etc. where parts of or the whole input is known to one of the parties), but this is the first such notion in the context of HSS.

Offline-Online HSS. Another key technical observation of this work is that the above trick allows us to split the inputs to our computation into two kinds. The first is the “classic” kind, which we refer to as the *offline inputs* (since they need to be known by both parties before the computation starts). As usual, offline inputs are to be provided in encrypted form e.g., to input a value x into the computation the party owning x provides $\text{Enc}(x) = g^x$.

On top of the classic offline inputs, our relinearization trick allows us to support a new type of inputs to the computation which we call *semi-private, online inputs* (as they can be defined adaptively after the computation starts). An online input is an input which is provided directly as an authenticated secret-sharing where the plaintext value is known to one of the parties, that is, an input x is shared as subtractive shares $\langle \mathbf{sk} \cdot x \rangle$. Crucially, as online inputs are just linear sharings, they can be sampled “lazily” where party P_0 defines their share just as a random value, and party P_1 's share will be the necessary correction value (together with the plaintext value, which we assume is known to P_1).

Using their semi-private online input shares $\langle \mathbf{sk} \cdot \vec{x} \rangle$ and the relinearisation encryption $\text{Enc}(\mathbf{sk}^{-1})$, the parties can non-interactively compute an authenticated sharing $\langle \mathbf{sk} \cdot C(\vec{x}) \rangle$ any arithmetic circuit C on their online inputs. The result of this computation can then be combined with a standard HSS evaluation method for restricted multiplication circuits: given an encryption $\text{Enc}(y) = g^y$ of an offline input y , the parties can then compute $\langle \mathbf{sk} \cdot C(x) \cdot y \rangle = \text{DDLog}((g^y)^{\langle \mathbf{sk} \cdot C(x) \rangle})$. Repeating this process (as in standard HSS) and removing the multiple of $\mathbf{sk}$ using $\text{Enc}(\mathbf{sk}^{-1})$ allows them to compute shares $\langle C(\vec{x}) \cdot C_{\text{rm}}(\vec{y}) \rangle$, where $\vec{y}$ are the (private)

offline inputs and C_{rm} is any restricted multiplication circuit.

Avoiding the Circular Security Assumption. One limitation of the above relinearisation technique is that it requires the parties to have an encryption of sk^{-1} . When using Damgård-Jurik encryption, we are not currently able to prove security of this, so need to make an additional “circular security” (or, key-dependent message) assumption. We also give a second relinearisation technique that avoids the need for an extra assumption, and is provable only based on DCR. This technique differs slightly, in that it takes as input sharings $\langle sk \cdot x \rangle$ and $\langle sk' \cdot y \rangle$ under two independent secret keys sk, sk' , and outputs a sharing $\langle sk \cdot xy \rangle$ under one of the two keys. This introduces some additional challenges when constructing an offline-online HSS, since now, for instance, when evaluating addition gates, we need to ensure that parties’ shares of the two input values are authenticated under the same key. To make this work, we use a “key-switching” technique where given encryptions of one key under another, the parties can switch the key under which a memory value is authenticated. Taking care to avoid any circularity issues, we define a chain of secret keys, which the parties progressively switch through as the circuit evaluation proceeds.

A similar approach was also used for the arithmetic garbling construction of [MORS24]. However, unlike [MORS24], we can avoid having to switch keys at every layer of the circuit, since as long as the inputs to two multiplication gates are under different keys sk, sk' , we don’t need to switch to a new key. The number of distinct secret keys (and hence, additional ciphertexts) needed in our construction scales with a kind of “pebbling depth” of the circuit, which in the worst case may equal the multiplicative depth, but can be much smaller in general.

Masking and Sublinear-size Garbling. Using our offline-online HSS, we design a general template for garbling circuits with high-fan-in *wide-gates* and constant-rate. We then instantiate this template in different ways, to obtain our results for Boolean and arithmetic garbling. We now give a brief overview of the Boolean instantiation, referring to section 5.2 for further details.

The high-level idea is that for each wire in the circuit with value $w \in \{0, 1\}$, we want the garbler and evaluator to obtain semi-private, authenticated shares $\langle y \rangle$ and $\langle sk \cdot y \rangle$ of the masked wire value $y = w \oplus r$ for some secret mask r known to the garbler, where the evaluator knows y . For the circuit input wires, the garbler can already define its authenticated shares ahead of time, thanks to the offline/online property of the HSS. Then, to evaluate a fan-in ℓ gate that computes some function f , if the masked values for the input wires are $y_1, \dots, y_\ell$, where $y_i = w_i \oplus r_i$, then the parties want to evaluate the following inside HSS:

$$((r_1, \dots, r_\ell, r), (y_1, \dots, y_\ell)) \mapsto f(y_1 \oplus r_1, \dots, y_\ell \oplus r_\ell) \oplus r \quad (1)$$

Here, the r_i ’s are private inputs initially chosen by the garbler, while the y_i ’s are semi-private, offline/online shared values that will be known to the evaluator. During the garbling phase, the garbler computes its authenticated shares of the gate output, namely, $\langle sk \cdot (f(w) \oplus r) \rangle_0$ and $\langle f(w) \oplus r \rangle_0$. Then, to preserve the invariant that the evaluator knows the masked value, the garbler includes $\langle f(w) \oplus r \rangle_0$ in the garbled circuit. This way, computation can continue through to the end of the circuit.

One problem with this sketch is that the r_i masks all need to be input into HSS as private input values (i.e. ciphertexts), which would lead to a very high communication cost. To avoid this, we have the garbler choose and input a key for a pseudorandom function, and derive all the r_i ’s pseudorandomly by evaluating the PRF as part of the HSS computation in eq. (1).

Overall, if the HSS is sufficiently expressive to evaluate eq. (1), for each gate, then the garbled circuit predominantly consists of a single, 1-bit wire mask for each fan-in ℓ gate. We

observe that our HSS is powerful enough to evaluate this for any gate whose function f has a polynomially sized truth table. Since a single such truth table can be used to capture a $\log \log(s)$ -depth chunk of a fan-in-two circuit of size s , we can convert any layered, size- s circuit with fan-in two gates into a circuit with large, truth table gates of size $s/\log \log s$ [Cou19]. Our scheme has a cost of 1 bit per truth table gate, leading to the result in Theorem 1.

To handle arithmetic circuits (Theorem 2), we generalize the above template to support arbitrary masking functions instead of just XOR. By instantiating this with additive (statistical) masking over the integers, we can build a garbling scheme where the garbler needs to send a single integer mask in order to evaluate a polynomial of degree $\log(s)$. This leads to a similar $\log \log(s)$ reduction in size for a general, layered arithmetic circuit of size s .

3 Preliminaries

Notation. We write $\mathbb{Z}/N\mathbb{Z}$ to mean the ring of integers modulo N , and denote its additive and multiplicative subgroups by $(\mathbb{Z}/N\mathbb{Z})^+$, $(\mathbb{Z}/N\mathbb{Z})^\times$, respectively. We write $[n]$ to denote the set of integers $\{1, \dots, n\}$, and $[a, b)$ the set $\{a, a+1, \dots, b-1\}$. We frequently refer to *subtractive shares* of a secret x , which means there are shares $\langle x \rangle_0, \langle x \rangle_1$ such that $\langle x \rangle_1 - \langle x \rangle_0 = x$, where the shares and subtraction operation are in $\mathbb{Z}$, unless otherwise specified.

RSA modulus generation. Let λ be a security parameter. We define RSA.Gen to be a polynomial-time algorithm which, on input λ , outputs (N, p, q) , where $N = pq$ and p, q are both ℓ -bit primes for some length parameter ℓ chosen to ensure λ -bit security.

3.1 Damgård-Jurik Cryptosystem

The Damgård-Jurik cryptosystem [DJ01] is a generalisation of the Paillier cryptosystem [Pai99] (which, with the notation of fig. 1, can be seen as the special case $\zeta = 1$).

Definition 3 (Decision Composite Residuosity Assumption (DCR), [Pai99]). Let λ be a security parameter. We say that the Decision Composite Residuosity (DCR) problem is hard relative to modulus-sampling algorithm $\text{RSA.Gen}(1^\lambda)$ if

$$\left\{ (N, x) : \begin{array}{l} (N, p, q) \xleftarrow{\$} \text{RSA.Gen} \\ x \xleftarrow{\$} (\mathbb{Z}/N^2\mathbb{Z})^\times \end{array} \right\} \stackrel{c}{\approx} \left\{ (N, x^N \bmod N) : \begin{array}{l} (N, p, q) \xleftarrow{\$} \text{RSA.Gen} \\ x \xleftarrow{\$} (\mathbb{Z}/N^2\mathbb{Z})^\times \end{array} \right\}$$

Theorem 4 (Damgård-Jurik Cryptosystem [DJ01]). For any choice of the parameter $\zeta \geq 1$, the construction of fig. 1 is a CPA-secure linearly homomorphic encryption scheme if and only if the DCR assumption holds.

LHE Damgård-Jurik Cryptosystem [DJ01]

Requires:

- $\zeta \geq 1$ is a constant defining the plaintext size.
- Functions $\exp: (\mathbb{Z}/N^\zeta\mathbb{Z})^+ \rightarrow 1 + N(\mathbb{Z}/N^{\zeta+1}\mathbb{Z})$ and $\log: 1 + N(\mathbb{Z}/N^{\zeta+1}\mathbb{Z}) \rightarrow (\mathbb{Z}/N^\zeta\mathbb{Z})^+$ are defined by the following expressions, as in [RS21]:

$$\exp(x) = \sum_{k=0}^{\zeta} \frac{(Nx)^k}{k!} \quad \text{and} \quad \log(1 + Nx) = \sum_{k=1}^{\zeta} \frac{(-N)^{k-1} x^k}{k}$$

DJ.KeyGen(1^λ): 1. Run $(N, p, q) \xleftarrow{\$} \text{RSA.Gen}(1^\lambda)$ 2. Compute $\varphi \leftarrow (p-1) \cdot (q-1)$ 3. Output (N, φ)	DJ.Enc$_{N,\zeta}(x)$: 1. Sample $r \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$ 2. Compute $c \leftarrow r^{N^\zeta} \cdot \exp(x)$ 3. Output c
DJ.Eval$_{N,\zeta}(c_1, c_2, \alpha)$: // Homomorph. eval. $(x, y) \mapsto \alpha \cdot x + y$ Compute and output $c \leftarrow c_1^\alpha \cdot c_2$	DJ.Dec$_{N,\zeta,\varphi}(c)$: 1. Compute $\psi = \varphi^{-1}$ in $\mathbb{Z}/N^\zeta\mathbb{Z}$ 2. Compute and output $x \leftarrow \psi \cdot \log(c^\varphi)$

Fig. 1: The Damgård-Jurik cryptosystem.

Definition 5 (KDM Security). A public-key encryption scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$, whose private-key and message spaces are denoted $\mathcal{K}$ and $\mathcal{M}$ respectively, is secure in the presence of key-dependent messages with respect to function class $\mathcal{F} \subseteq \mathcal{K} \rightarrow \mathcal{M}$ (or simply $\mathcal{F}$ -KDM-secure) if for every p.p.t. oracle algorithm $\mathcal{A}$ it holds that:

$$\left| \Pr \left[\mathcal{A}^{\mathcal{O}_{\mathcal{F}, \text{sk}, 0}^{\text{KDM}}}(1^\lambda, \text{pk}) = 1 : (\text{sk}, \text{pk}) \xleftarrow{\$} \text{KeyGen}(1^\lambda) \right] - \Pr \left[\mathcal{A}^{\mathcal{O}_{\mathcal{F}, \text{sk}, 1}^{\text{KDM}}}(1^\lambda, \text{pk}) = 1 : (\text{sk}, \text{pk}) \xleftarrow{\$} \text{KeyGen}(1^\lambda) \right] \right| \leq \text{negl}(\lambda),$$

where oracles $\mathcal{O}_{\mathcal{F}, \text{sk}, 0}^{\text{KDM}}$ and $\mathcal{O}_{\mathcal{F}, \text{sk}, 1}^{\text{KDM}}$ are defined as in fig. 2.

Experiment IND-CPA and IND-KDM Security Game	
$\frac{\mathcal{O}_{\mathcal{F}, \text{sk}, 0}^{\text{KDM}}(f)}{\text{if } f \notin \mathcal{F} \text{ return } \perp}$ else $c \xleftarrow{\$} \text{Enc}_{\text{pk}}(f(\text{sk}))$ return c	$\frac{\mathcal{O}_{\mathcal{F}, \text{sk}, 1}^{\text{KDM}}(f)}{\text{if } f \notin \mathcal{F} \text{ return } \perp}$ else $c \xleftarrow{\$} \text{Enc}_{\text{pk}}(0^{ f(\text{sk}) })$ return c

Fig. 2: Oracles used in IND-KDM security (definition 5).

3.2 Damgård-Jurik-ElGamal Cryptosystem

Theorem 6. Assuming DCR, Damgård-Jurik-ElGamal encryption fig. 3 is a public key encryption scheme satisfying correctness and KDM security for affine functions of the key. Specifically, the following properties hold:

Correctness: $\text{DJE.Dec}_{\text{sk}}(\text{DJE.Enc}_{\text{pk}}(x)) = x$, for any (sk, pk) in the support of DJE.KeyGen , and any $x \in \mathbb{Z}/N^\zeta\mathbb{Z}$.

KDM Security: Definition 5 for affine functions of the form $f_{a,b} : \text{sk} \mapsto a \cdot \text{sk} + b$, where $a, b \in \mathbb{Z}/N^\zeta\mathbb{Z}$.

Proof. See [BMO⁺25]. □

DJE Damgård-Jurik-ElGamal Cryptosystem

Requires:

- $\zeta \geq 1$ is a parameter defining the plaintext size.
- Group isomorphism $\exp: (\mathbb{Z}/N^\zeta\mathbb{Z})^+ \rightarrow 1 + N(\mathbb{Z}/N^{\zeta+1}\mathbb{Z})$ and its inverse $\log: 1 + N(\mathbb{Z}/N^{\zeta+1}\mathbb{Z}) \rightarrow (\mathbb{Z}/N^\zeta\mathbb{Z})^+$, as defined as in [RS21]:

$$\exp(x) = \sum_{k=0}^{\zeta} \frac{(Nx)^k}{k!} \quad \text{and} \quad \log(1 + Nx) = \sum_{k=1}^{\zeta} \frac{(-N)^{k-1} x^k}{k}$$

DJE.Setup(1^λ):

1. Sample $(N, p, q) \xleftarrow{\$} \text{RSA.Gen}(1^\lambda)^a$
2. Sample $g \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$
3. Output $\text{pp} = (g, N)$

^a Note that p and q are unused, so N could instead be a CRS.

DJE.KeyGen($1^\lambda, \text{pp}$):

1. Parse pp as $\text{pp} = (g, N)$
2. Sample $k \xleftarrow{\$} [0, N)$
3. Compute $h \leftarrow g^{-k}$
4. Output $(\text{sk} = (k, N), \text{pk} = (g, h, N))$

DJE.Enc_{pk}(x):

1. Parse $\text{pk} = (g, h, N)$
2. Sample $r \xleftarrow{\$} [0, N)$
3. Compute $c_0 \leftarrow g^r$
4. Compute $c_1 \leftarrow h^r \cdot \exp(x)$
5. Output $c = (c_0, c_1)$

DJE.Dec_{sk}($c = (c_0, c_1)$):

1. Parse $\text{sk} = (k, N)$
2. Assert $c_0^k \cdot c_1 \equiv 1 \pmod{N}$
3. Output $x \leftarrow \log(c_0^k \cdot c_1)$

Fig. 3: The Damgård-Jurik-ElGamal cryptosystem.

3.3 Distributed Discrete Logarithm

We use two distinct distributed discrete log procedures, for the Damgård-Jurik and Damgård-Jurik-ElGamal cryptosystems. The following two lemmas follow from [RS21, Theorem 18], and were also used in [MORS24].

DDLOG Damgård-Jurik Distance Function [RS21]

DDLog_N($h \in \mathbb{Z}/N^{\zeta+1}\mathbb{Z}$):

Compute and output $z \leftarrow \log\left(\frac{h}{h \bmod N}\right) \in \mathbb{Z}/N^\zeta\mathbb{Z}$

Fig. 4: [RS21]'s distributed discrete logarithm for Damgård-Jurik.

Lemma 7 (Damgård-Jurik Distributed Decryption). *For all $(N, \varphi) \in \text{Supp}(\text{DJ.KeyGen})$, for all exponents $\zeta \geq 1$, and for all ciphertexts $c \in (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$ and shares (over $\mathbb{Z}$) $\langle x\varphi \rangle_0, \langle x\varphi \rangle_1$ of some $x \in \mathbb{Z}$ times φ , we have*

$$\text{DDLog}(c^{\langle x\varphi \rangle_1}) - \text{DDLog}(c^{\langle x\varphi \rangle_0}) \equiv \text{DJ.Dec}_{N, \zeta, \varphi}(c) \cdot x \cdot \varphi \pmod{N^\zeta}.$$

Lemma 8 (Damgård-Jurik-ElGamal Distributed Decryption). *If we have shares over $\mathbb{Z}$ of $\langle x \rangle_1 - \langle x \rangle_0 = x$ and $\langle k \cdot x \rangle_1 - \langle k \cdot x \rangle_0 = k \cdot x$, then*

$$\text{DDLog}_N(c_0^{\langle k \cdot x \rangle_1} c_1^{\langle x \rangle_1}) - \text{DDLog}_N(c_0^{\langle k \cdot x \rangle_0} c_1^{\langle x \rangle_0}) \equiv x \cdot y \pmod{N^\zeta}$$

always holds, for every choice of plaintext size $\zeta \geq 1$, key pair $(\text{sk} = (k, N), \text{pk}) \in \text{Supp}(\text{DJE.KeyGen}(1^\lambda))$, plaintext $y \in \mathbb{Z}/N^\zeta\mathbb{Z}$, ciphertext $(c_0, c_1) \in \text{Supp}(\text{DJE.Enc}_{\text{pk}}(y))$, and scalar $x \in \mathbb{Z}/N^\zeta\mathbb{Z}$.

We also rely on the following, standard lemma stating that shares of some value x modulo M can be lifted to valid shares over the integers, with a failure probability of $|x|/M$.

Lemma 9 (From e.g. [RS21, Lemma 19]). *For all moduli $M > 1$ and all modulo M shares $\langle x \rangle_0, \langle x \rangle_1 \in \mathbb{Z}/M\mathbb{Z}$ of some $x \in \mathbb{Z}$, we have*

$$\Pr_{r \xleftarrow{\$} \mathbb{Z}/M\mathbb{Z}} \left[(\langle x \rangle_1 + r) \bmod M - (\langle x \rangle_0 + r) \bmod M = x \right] = \max \left(1 - \frac{|x|}{M}, 0 \right).$$

3.4 Models of computation

Arithmetic circuits. Unless otherwise specified, we consider fan-in two arithmetic circuits over the basis $\{+, \times\}$, with a single output gate.

Definition 10 (Arithmetic circuit). An *arithmetic circuit* C (over some ring $\mathcal{R}$) is a rooted, fan-in two, directed acyclic graph whose leaves are labelled by the constant $1_{\mathcal{R}}$ or input variables, whose fan-in two internal nodes are labelled by $+$ or $\times$, and whose fan-in one internal nodes are each labelled by $\cdot_\alpha$ for some $\alpha \in \mathcal{R}$.

Each node (or *gate*) of the circuit is associated with an n -variate polynomial (where n is the number of distinct input variables used as labels), defined recursively according to a topological order on the nodes of C (*i.e.* in a *gate-by-gate* fashion):

- a constant gate, labelled $1_{\mathcal{R}}$, computes the constant polynomial $1_{\mathcal{R}}$;
- an input gate, labelled by the i^{th} input variable x_i , computes the polynomial $P_i = (X_1, \dots, X_n) = X_i$;
- a linear gate, labelled $+$, computes the sum of the polynomials computed by its parents;
- a multiplication gate, labelled $\times$, computes the product of the polynomials computed by its parents;
- a scalar multiplication gate, labelled $\cdot_\alpha$, computes the product of the polynomial computed by its parent with the constant α .

The circuit computes the polynomial P associated with its root, *a.k.a.* the *output gate*. Evaluating C is done by evaluating the associated polynomial function.

We also consider restricted-multiplication circuits [Bar89, Cle90, BGI16, RS21] (sometimes called restricted multiplication straightline programs). This class of circuits is known to contain LOGSPACE (which includes NC1).

Definition 11 (Restricted-multiplication circuit). A *restricted-multiplication circuit* is an arithmetic circuit satisfying the following property: each node labelled $\times$ has at least one predecessor which is a leaf (that is, each multiplication gate has at least one input wire that is a circuit input).

Definition 12 (B -bounded circuit and C -admissible input). Let C be an n -input arithmetic circuit over $\mathbb{Z}$. An input (vector) $x \in \mathbb{Z}^n$ is *admissible for C with respect to bound B* , if the polynomial function computed by *each and every* gate is at most B when evaluated on input x .

If C and/or B is clear from context, we simply say C -admissible, B -admissible, or admissible. If C is an unbounded arithmetic circuit, any input x is C -admissible.

We use the following (non-standard) notion of multiplicative “pebbling” depth.

Definition 13 (Pebbling depth). Given an arithmetic circuit, the *pebbling depth* of a gate is defined recursively as follows:

- the pebbling depth of a constant or input gate is 0;
- the pebbling depth of an addition gate whose predecessors have pebbling depth i and j respectively is $\max(i, j)$;
- the pebbling depth of a scalar multiplication gate is the pebbling depth of its parent;
- the pebbling depth of a multiplication gate whose parents have pebbling depth i and j respectively is $\max(i, j, \min(i, j) + 1)$ (in other words, if $i = j$ this is $i + 1$, and if $i \neq j$ this is $\max(i, j)$).

The *pebbling depth of an arithmetic circuit* is then defined as the pebbling depth of its output gate. Note that pebbling depth is upper bounded by the (standard) multiplicative depth, but can be much smaller.

Circuit Classes VP and RMS. We denote by $\text{VP}_n[B, D]$ the class of B -bounded arithmetic circuits with n inputs, size $\text{poly}(n)$ and pebbling depth at most D . Similarly, we denote by $\text{RMS}_n[B]$ the class of B -bounded, restricted multiplication arithmetic circuits with n inputs and size $\text{poly}(n)$. When B and/or D are clear from context, we simply write VP_n and RMS_n .

Definition 14 (Tensor Product of Circuit Classes). Let $\mathcal{C}_0$ and $\mathcal{C}_1$ be two circuit classes over a ring $\mathcal{R}$. The tensor product of $\mathcal{C}_0$ and $\mathcal{C}_1$, denoted $\mathcal{C}_0 \otimes \mathcal{C}_1$, is defined as the class of all arithmetic circuits composed from some $F_1, \dots, F_r \in \mathcal{C}_0$ and $G_1, \dots, G_r \in \mathcal{C}_1$ in the following way: Let L_0 be the union of all input labels from all F_i , and similarly L_1 from all G_i . Then composed circuit has input labels

$$\{(j, l) \mid j \in \{0, 1\}, l \in L_j\}, \quad \text{and computes} \quad \sum_{i=1}^r F_i \cdot G_i. \quad (2)$$

More specifically, the composed circuit is union of the graphs of the circuits for all F_i and G_i , followed by r multiplication gates and $r - 1$ addition gates to compute (2). The integer r associated with this circuit is called the rank.

We write $\mathcal{C}_0 \otimes_r \mathcal{C}_1$ to denote the subset of tensor product circuits with rank at most r .

Remark 15. Note that the size of a tensor product circuit (definition 14) is lower bounded by its rank r . Therefore, when considering classes of circuits with sizes bounded by a polynomial, we can assume that r is bounded by a polynomial.

4 (Semi-Private) Offline-Online HSS

In section 4.1 we provide the definitions of semi-private HSS (definition 16) and its specialisation to offline/online sharing of semi-private inputs (definition 17). In section 4.2 we construct (fig. 6) semi-private offline-online HSS, assuming the KDM security of the Damgård-Jurik cryptosystem, for product circuits of the form $C \cdot C_{\text{rm}}$, where C is an arbitrary circuit applied to the semi-private inputs (known to one party), and C_{rm} is a restricted-multiplication circuit applied to the private inputs. In section 4.3, we present an alternative construction (fig. 9) that assumes only DCR, but achieving a weaker notion of compactness.

4.1 Definition

In HSS with setup, a setup algorithm is used to generate a secret key, used to share any input, in conjunction with a pair of evaluation keys, which can be used to evaluate a function of the shared inputs, obtaining a subtractive sharing of the output. Importantly, each input can be shared independently (as long as they are generated with respect to the same secret key) and need not be batched.

Below, we generalize the standard notion of HSS to allow for a special type of *semi-private input sharing algorithm*, which outputs a pair of shares of an input y such that only party 0's shares are guaranteed to hide y . In addition, we support the usual *private* input sharing algorithm, where each share (individually) reveals no information about the input. Each of the functions supported by the homomorphic evaluation algorithm take inputs partitioned into two sets: a set of n_{priv} private inputs, and a set of $n_{\text{s-priv}}$ semi-private inputs.

Definition 16 (Semi-Private HSS). Let $\mathcal{R} = \mathcal{R}(\lambda)$ be a ring and $n_{\text{priv}}, n_{\text{s-priv}} = \text{poly}(\lambda)$. A (two-party) semi-private HSS scheme (with setup) supporting a class of computations $\mathcal{C} \subseteq (\mathcal{R}^{n_{\text{priv}}} \times \mathcal{R}^{n_{\text{s-priv}}} \rightarrow \mathcal{R})$ is a tuple of *p.p.t.* algorithms $\text{HSS} = (\text{HSS.Setup}, \text{HSS.Share}_{\text{priv}}, \text{HSS.Share}_{\text{s-priv}}, \text{HSS.Eval})$ with the following syntax and properties:

- $\text{HSS.Setup}(1^\lambda)$: On input the security parameter 1^λ , the setup algorithm **Setup** outputs a secret key sk and a pair of evaluation keys $(\text{ek}_0, \text{ek}_1)$.
- $\text{HSS.Share}_{\text{priv}}(1^\lambda, \text{sk}, x)$: On input the security parameter 1^λ , secret key sk and a private input $x \in \mathcal{R}$, the input sharing algorithm **Share_{priv}** outputs a pair of input shares $(x^{(0)}, x^{(1)})$.
- $\text{HSS.Share}_{\text{s-priv}}(1^\lambda, \text{sk}, y)$: On input the security parameter 1^λ , secret key sk and a semi-private input $y \in \mathcal{R}$, the semi-private input sharing algorithm **Share_{s-priv}** outputs a pair of input shares $(y^{(0)}, y^{(1)})$.
- $\text{HSS.Eval}(\sigma, \text{ek}_\sigma, (x_i^{(\sigma)})_{i=1}^{n_{\text{priv}}}, (y_i^{(\sigma)})_{i=1}^{n_{\text{s-priv}}}, C)$: On input a party index σ , an evaluation key ek_σ , a vector of n_{priv} private input shares $(x_i^{(\sigma)})_{i=1}^{n_{\text{priv}}}$, a vector of $n_{\text{s-priv}}$ semi-private input shares $(y_i^{(\sigma)})_{i=1}^{n_{\text{s-priv}}}$, and a supported evaluation circuit $C \in \mathcal{C}$, the homomorphic evaluation algorithm **Eval** returns an output share $y_\sigma \in \mathcal{R}$.

It should satisfy the following properties (where the experiments are detailed in fig. 5):

- **Correctness.** For any sufficiently large $\lambda \in \mathbb{N}$, for any circuit $C \in \mathcal{C}$, any C -admissible inputs $((x_i)_{i \in [n_{\text{priv}}]}, (y_i)_{i \in [n_{\text{s-priv}}]}) \in \mathcal{R}^{n_{\text{priv}}} \times \mathcal{R}^{n_{\text{s-priv}}}$,

$$\Pr \left[y = C((x_i)_{i \in [n_{\text{priv}}]}, (y_i)_{i \in [n_{\text{s-priv}}]}) : y \xleftarrow{\$} \text{Exp}_{\text{corr}}^{\text{HSS}}(1^\lambda, (x_i)_{i \in [n_{\text{priv}}]}, (y_i)_{i \in [n_{\text{s-priv}}]}) \right] \geq 1 - \text{negl}(\lambda) .$$

- **Security.** For every $\sigma \in \{0, 1\}$, there exists a *p.p.t.* algorithm Sim_σ (a simulator), such that for any C -admissible inputs $((x_i)_{i \in [n_{\text{priv}}]}, (y_i)_{i \in [n_{\text{s-priv}}]}) \in \mathcal{R}^{n_{\text{priv}}} \times \mathcal{R}^{n_{\text{s-priv}}}$, the output of the experiments $\text{Real}_\sigma^{\text{HSS}}$ and $\text{Ideal}_\sigma^{\text{HSS}}$ (fig. 5) are computationally indistinguishable.

In our garbling application, we additionally require a linearity property of the semi-private input sharing algorithm. In particular, we require that the first party's share of a semi-private input y should be generated uniformly and independently of the input; the second party's input share can later be derived as a subtractive share of $y \cdot f(\text{sk})$, where f is some function that maps the secret key to a (vector of) elements of the underlying ring.

Together with this, we define an *authenticated evaluation* property, which requires a special form of homomorphic evaluation algorithm that produces subtractive shares of the output z , as well as $z \cdot f(\text{sk})$.

$\text{Exp}_{\text{corr}}^{\text{HSS}} / \text{Exp}_{\text{auth-corr}}^{\text{HSS}}(1^\lambda, (x_i)_{i \in [n_{\text{priv}}]}, (y_i)_{i \in [n_{\text{s-priv}}]}) :$	
$(\text{sk}, (\text{ek}_0, \text{ek}_1)) \xleftarrow{\$} \text{Setup}(1^\lambda)$	
for $i \in [n_{\text{priv}}]$, $(x_i^{(0)}, x_i^{(1)}) \xleftarrow{\$} \text{Share}_{\text{priv}}(1^\lambda, \text{sk}, x_i)$	
for $i \in [n_{\text{s-priv}}]$, $(y_i^{(0)}, y_i^{(1)}) \xleftarrow{\$} \text{Share}_{\text{s-priv}}(1^\lambda, \text{sk}, y_i)$	
$z_0 \xleftarrow{\$} \text{Eval/AuthEval}(0, \text{ek}_0, (x_i^{(0)})_{i=1}^{n_{\text{priv}}}, (y_i^{(0)})_{i=1}^{n_{\text{s-priv}}}, C);$	
$z_1 \xleftarrow{\$} \text{Eval/AuthEval}(1, \text{ek}_1, (x_i^{(1)})_{i=1}^{n_{\text{priv}}}, (y_i^{(1)})_{i=1}^{n_{\text{s-priv}}}, C);$	
return $z \leftarrow z_1 - z_0$	

$\text{Real}_\sigma^{\text{HSS}}(1^\lambda) :$	$\text{Ideal}_\sigma^{\text{HSS}}(1^\lambda) :$
$(\text{sk}, (\text{ek}_0, \text{ek}_1)) \xleftarrow{\$} \text{HSS.Setup}(1^\lambda)$	if $\sigma = 0$ then
for $i \in [n_{\text{priv}}]$ do	return $\text{Sim}_0(1^\lambda, n_{\text{priv}}, n_{\text{s-priv}})$
$(x_i^{(0)}, x_i^{(1)}) \xleftarrow{\$} \text{Share}_{\text{priv}}(1^\lambda, \text{sk}, x_i)$	else
for $i \in [n_{\text{s-priv}}]$ do	return $\text{Sim}_1(1^\lambda, n_{\text{priv}}, n_{\text{s-priv}}, (y_i)_{i \in [n_{\text{s-priv}}]})$
$(y_i^{(0)}, y_i^{(1)}) \xleftarrow{\$} \text{Share}_{\text{s-priv}}(1^\lambda, \text{sk}, y_i)$	
return $(\text{ek}_\sigma, (x_i^{(\sigma)})_{i \in [n_{\text{priv}}]}, (y_i^{(\sigma)})_{i \in [n_{\text{s-priv}}]})$	

Fig. 5: Experiments for correctness, authenticated correctness and security in semi-private HSS

Definition 17 (Additional Properties of Semi-Private HSS).

- **Linear, offline/online sharing of semi-private inputs.** Let $\mathcal{R} = \mathbb{Z}/M\mathbb{Z}$ for some modulus M . We say the semi-private input sharing algorithm $\text{HSS.Share}_{\text{s-priv}}$ is *offline/online* if it can be decomposed into two *p.p.t.* algorithms $\text{HSS.Share}_{\text{e0,s-priv}}$ and $\text{HSS.Share}_{\text{e1,s-priv}}$, parameterized by a dimension $d \geq 2$ and efficiently computable deterministic function $f : * \rightarrow (\mathbb{Z}/M\mathbb{Z})^d$, such that:
 - $\text{HSS.Share}_{\text{e0,s-priv}}(1^\lambda)$, on input the security parameter, outputs a uniformly random share $y^{(0)} \xleftarrow{\$} (\mathbb{Z}/M\mathbb{Z})^d$
 - $\text{HSS.Share}_{\text{e1,s-priv}}(1^\lambda, \text{sk}, y^{(0)}, y)$, on input the secret key sk , share $y^{(0)}$ and input y , computes and outputs the share $y^{(1)} \leftarrow (y, f(\text{sk}) \cdot y + y^{(0)})$.
 - $\text{HSS.Share}_{\text{s-priv}}(1^\lambda, \text{sk}, y)$ generates its output $(y^{(0)}, y^{(1)})$ in the natural way, by first sampling $y^{(0)} \xleftarrow{\$} \text{Share}_{\text{e0,s-priv}}(1^\lambda)$, and then $y^{(1)} \leftarrow \text{Share}_{\text{e1,s-priv}}(1^\lambda, \text{sk}, y)$.
- **Authenticated evaluation.** Furthermore, for a semi-private HSS scheme with linear offline-online sharing, we say that it admits *authenticated evaluation*, if there exists an algorithm AuthEval with the same syntax as HSS.Eval , such that:

$$\Pr \left[\begin{array}{c} z = (1, f(\text{sk})) \cdot C((x_i)_{i \in [n_{\text{priv}}]}, (y_i)_{i \in [n_{\text{s-priv}}]}) \\ : z \xleftarrow{\$} \text{Exp}_{\text{auth-corr}}^{\text{HSS}}(1^\lambda, (x_i)_{i \in [n_{\text{priv}}]}, (y_i)_{i \in [n_{\text{s-priv}}]}) \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

where $\text{Exp}_{\text{auth-corr}}^{\text{HSS}}$ is defined in fig. 5.

4.2 Semi-private HSS construction from KDM-security of Damgård-Jurik

Before describing the HSS construction, we introduce our core relinearisation technique.

Core Lemma 1 (Relinearisation, Damgård-Jurik variant). *For all $(N, \varphi) \in \text{Supp}(\text{DJ.KeyGen})$, $c_{\text{inv}} \in \text{Supp}(\text{DJ.Enc}_N(\varphi^{-1} \bmod N^\zeta))$, and $(x, y) \in \mathbb{Z}^2$, if*

1. $\langle \varphi \cdot x \rangle_1, \langle \varphi \cdot x \rangle_0$ denote shares over $\mathbb{Z}$ of $\varphi \cdot x$ (meaning they are integers satisfying $\langle \varphi \cdot x \rangle_1 - \langle \varphi \cdot x \rangle_0 = \varphi x$)
2. $\langle \varphi \cdot y \rangle_1, \langle \varphi \cdot y \rangle_0$ denote shares over $\mathbb{Z}$ of $\varphi \cdot y$

then $\langle \varphi \cdot xy \rangle_1 - \langle \varphi \cdot xy \rangle_0 \equiv \varphi \cdot xy \pmod{N^\zeta}$, where $\langle \varphi \cdot xy \rangle_1$ and $\langle \varphi \cdot xy \rangle_0$ are defined as follows:

$$\begin{cases} \langle \varphi \cdot xy \rangle_1 \leftarrow -\text{DDLog} \left(c_{\text{inv}}^{\langle \varphi \cdot x \rangle_1 \cdot \langle \varphi \cdot y \rangle_1} \right) + (x \cdot \langle \varphi \cdot y \rangle_1 + y \cdot \langle \varphi \cdot x \rangle_1) \\ \langle \varphi \cdot xy \rangle_0 \leftarrow -\text{DDLog} \left(c_{\text{inv}}^{\langle \varphi \cdot x \rangle_0 \cdot \langle \varphi \cdot y \rangle_0} \right) \end{cases}$$

Proof. Consider the following algebraic identity:

$$\begin{aligned} \langle \varphi \cdot x \rangle_0 \cdot \langle \varphi \cdot y \rangle_0 &= (\langle \varphi \cdot x \rangle_1 - \varphi \cdot x) \cdot (\langle \varphi \cdot y \rangle_1 - \varphi \cdot y) \\ &= \varphi^2 \cdot xy + \varphi \cdot (-x \cdot \langle \varphi \cdot y \rangle_1 - y \cdot \langle \varphi \cdot x \rangle_1) + \langle \varphi \cdot x \rangle_1 \cdot \langle \varphi \cdot y \rangle_1 \end{aligned}$$

After reorganising terms, we get that $\langle \varphi \cdot x \rangle_\sigma \cdot \langle \varphi \cdot y \rangle_\sigma$ is an authenticated share over $\mathbb{Z}$ of $-\varphi \cdot xy + (x \cdot \langle \varphi \cdot y \rangle_1 + y \cdot \langle \varphi \cdot x \rangle_1)$:

$$\langle \varphi \cdot x \rangle_1 \cdot \langle \varphi \cdot y \rangle_1 - \langle \varphi \cdot x \rangle_0 \cdot \langle \varphi \cdot y \rangle_0 = \varphi \cdot (-\varphi \cdot xy + (x \cdot \langle \varphi \cdot y \rangle_1 + y \cdot \langle \varphi \cdot x \rangle_1))$$

Therefore, by Lemma 7,

$$\begin{aligned} \text{DDLog} \left(c_{\text{inv}}^{\langle \varphi \cdot x \rangle_1 \cdot \langle \varphi \cdot y \rangle_1} \right) - \text{DDLog} \left(c_{\text{inv}}^{\langle \varphi \cdot x \rangle_0 \cdot \langle \varphi \cdot y \rangle_0} \right) \\ \equiv \underbrace{(\text{DJ.Dec}(c_{\text{inv}}) \cdot \varphi)}_{\equiv 1 \pmod{N^\zeta}} \cdot [-\varphi \cdot xy + (x \cdot \langle \varphi \cdot y \rangle_1 + y \cdot \langle \varphi \cdot x \rangle_1)] \pmod{N^\zeta} \end{aligned}$$

It follows that

$$\begin{aligned} \left(-\text{DDLog} \left(c_{\text{inv}}^{\langle \varphi \cdot x \rangle_1 \cdot \langle \varphi \cdot y \rangle_1} \right) + (x \cdot \langle \varphi \cdot y \rangle_1 + y \cdot \langle \varphi \cdot x \rangle_1) \right) \\ - \left(-\text{DDLog} \left(c_{\text{inv}}^{\langle \varphi \cdot x \rangle_0 \cdot \langle \varphi \cdot y \rangle_0} \right) \right) \equiv \varphi \cdot xy \pmod{N^\zeta} \end{aligned}$$

□

HSS Semi-private Offline-Online HSS (KDM-secure Damgård-Jurik variant)

Requires:

- DJ = (DJ.KeyGen, DJ.Enc, DJ.Dec) is the Damgård-Jurik cryptosystem of fig. 1.
- DDLog is the algorithm of fig. 4.
- $(\text{PRF}_{N,\zeta,s,t,\lambda})_{(N,\zeta,s,t,\lambda) \in \mathbb{N}^5}$ is an ε_{PRF} -secure family of pseudorandom functions such that $\text{PRF}_{N,\zeta,s,t,\lambda} : \{0, 1\}^\lambda \times [s + t] \rightarrow \mathbb{Z}/N^\zeta\mathbb{Z}$.

HSS.Setup(1^λ) :

1. $(N, \varphi) \xleftarrow{\$} \text{DJ.KeyGen}(1^\lambda)$
2. $c_{\text{inv}} \xleftarrow{\$} \text{DJ.Enc}_N(\varphi^{-1} \pmod{N^\zeta})$
3. $\text{sh}_{0,0} \xleftarrow{\$} [0, N] \parallel \langle \varphi \cdot 1 \rangle_0$ (over $\mathbb{Z}$)
4. $\text{sh}_{0,1} \leftarrow \varphi \cdot 1 + \text{sh}_{1,0} \parallel \langle \varphi \cdot 1 \rangle_1$ (over $\mathbb{Z}$)
5. $k_{\text{PRF}} \xleftarrow{\$} \{0, 1\}^\lambda$
6. $\text{sk} \leftarrow (\varphi, N)$, $\text{ek}_0 \leftarrow (N, c_{\text{inv}}, k_{\text{PRF}}, \text{sh}_{1,0})$, and $\text{ek}_1 = (N, c_{\text{inv}}, k_{\text{PRF}}, \text{sh}_{1,1})$
7. Output $(\text{sk}, (\text{ek}_0, \text{ek}_1))$

HSS.Share_{priv}($1^\lambda, \text{sk}, x$) :

1. Parse $\text{sk} = (\varphi, N)$
2. $c_{\text{priv}} \xleftarrow{\$} \text{DJ.Enc}_N(x)$
3. Output $(x_{\text{priv}}^{(0)}, x_{\text{priv}}^{(1)}) \leftarrow (c_{\text{priv}}, c_{\text{priv}})$

HSS.Share_{0,s-priv}($1^\lambda, \text{sk}$) :

1. $\text{sh}_0 \xleftarrow{\$} \mathbb{Z}/N^\zeta\mathbb{Z} \text{ // } \langle \varphi \cdot y \rangle_0 \text{ (over } \mathbb{Z}/N^\zeta\mathbb{Z})$
2. Output $y^{(0)} \leftarrow \text{sh}_0$

HSS.Share_{1,s-priv}($1^\lambda, \text{sk}, y, \text{st}$) :

1. Parse $\text{sk} = (\varphi, N)$
2. $\text{sh}_1 \leftarrow (\varphi \cdot y + y^{(0)}) \bmod N^\zeta \text{ // } \langle \varphi \cdot y \rangle_1 \text{ (over } \mathbb{Z}/N^\zeta\mathbb{Z})$
3. Output $y^{(1)} \leftarrow (y, \text{sh}_1)$

HSS.Eval($\sigma, \text{ek}_\sigma, (x_i^{(\sigma)})_{i \in [n_{\text{priv}}]}, (y_i^{(\sigma)})_{i \in [n_{\text{s-priv}}]}, C \cdot C_{\text{rm}}$) :

// C is a B -bounded arithmetic circuit, C_{rm} is a B' -bounded restricted-multiplication arithmetic circuit

1. Parse $\text{ek}_\sigma = (N, c_{\text{inv}}, k_{\text{PRF}}, \text{sh}_{1,\sigma})$
2. For $i = 1, \dots, n_{\text{s-priv}}$, parse $y_i^{(\sigma)}$ as

$$y_i^{(\sigma)} = \begin{cases} (y, \text{sh}_{i,1}) & \text{if } \sigma = 1 \\ \text{sh}_{i,0} & \text{if } \sigma = 0 \end{cases}$$

// First, evaluate C on semi-private inputs

3. Parse C as a sequence of gates $g_1, g_2, \dots, g_{|C|}$, where $g_1, \dots, g_{n_{\text{s-priv}}}$ are the input gates.
4. For $i = n_{\text{s-priv}} + 1, \dots, |C|$:

// Invariant: $\text{sh}_{i,\sigma}$ stores $\langle \varphi \cdot P_i(\vec{y}) \rangle_\sigma \text{ (over } \mathbb{Z})$, where P_i is the polynomial computed up to gate g_i

- If g_i is the constant gate 1,

$$\text{sh}_{i,\sigma} \leftarrow \text{sh}_{0,\sigma} \text{ // } \langle \varphi \cdot 1 \rangle_\sigma$$

- If g_i is an addition gate, whose parents are g_{i_L} and g_{i_R} :

$$\text{sh}_{i,\sigma} \leftarrow \text{sh}_{i_L,\sigma} + \text{sh}_{i_R,\sigma} \text{ // } \langle \varphi \cdot P_i(\vec{y}) \rangle_\sigma \leftarrow \langle \varphi \cdot (P_{i_L} + P_{i_R})(\vec{y}) \rangle_\sigma$$

- If g_i is a multiplication gate, whose parents are g_{i_L} and g_{i_R} :

$$\text{sh}_{i,\sigma} \leftarrow \begin{cases} -\text{DDLog} \left(c_{\text{inv}}^{\text{sh}_{i_L,\sigma} \cdot \text{sh}_{i_R,\sigma}} \right) + P_{i_L}(\vec{y}) \cdot \text{sh}_{i_R,\sigma} + P_{i_R}(\vec{y}) \cdot \text{sh}_{i_L,\sigma} & \text{if } \sigma = 1 \\ -\text{DDLog} \left(c_{\text{inv}}^{\text{sh}_{i_L,\sigma} \cdot \text{sh}_{i_R,\sigma}} \right) & \text{if } \sigma = 0 \end{cases}$$

$$\text{sh}_{i,\sigma} \leftarrow \text{sh}_{i,\sigma} + \text{PRF}(k_{\text{PRF}}, i) \bmod N^\zeta \text{ // } \langle \varphi \cdot P_i(\vec{y}) \rangle_\sigma = \langle \varphi \cdot (P_{i_L} P_{i_R})(\vec{y}) \rangle_\sigma$$

- If g_i is a scalar multiplication gate $\cdot_\alpha$, whose parent is g_j :

$$\text{sh}_{i,\sigma} \leftarrow \alpha \cdot \text{sh}_{j,\sigma} \text{ // } \langle \varphi \cdot \alpha \cdot P_j(\vec{y}) \rangle_\sigma \leftarrow \alpha \cdot \langle \varphi \cdot P_j(\vec{y}) \rangle_\sigma$$

// $\text{sh}_{|C|,\sigma}$ now stores $\langle \varphi \cdot C(\vec{y}) \rangle_\sigma$

5. For $i \in [n_{\text{priv}}]$, parse $x_i^{(\sigma)}$ as ciphertext c_i
 // Next, evaluate C_{rm} on the private inputs $\vec{x}$

6. Parse C_{rm} as a sequence of gates $g'_1, g'_2, \dots, g'_{|C_{\text{rm}}|}$.

7. For $i = 1, \dots, |C_{\text{rm}}|$:

// Invariant: $\text{sh}_{|C|+i, \sigma}$ stores $\langle \varphi \cdot C(\vec{y}) \cdot P_i(\vec{x}) \rangle_{\sigma}$, where P_i is the polynomial computed up to gate g'_i

- If g'_i is the constant 1:

$$\text{sh}_{|C|+i, \sigma} \leftarrow \text{sh}_{|C|, \sigma} \quad // \langle \varphi \cdot C(\vec{y}) \cdot 1 \rangle_{\sigma}$$

- If g'_i is the j^{th} input gate:

$$\begin{aligned} \text{sh}_{|C|+i, \sigma} &\leftarrow \text{DDLog} \left((c_j)^{\text{sh}_{|C|, \sigma}} \right) + \text{PRF}(k_{\text{PRF}}, |C| + i) \bmod N^{\zeta} \\ &// \langle \varphi \cdot C(\vec{y}) \cdot x_j \rangle_{\sigma} \end{aligned}$$

- If g'_i is an addition gate, whose parents are g'_{i_L} and g'_{i_R} :

$$\begin{aligned} \text{sh}_{|C|+i, \sigma} &\leftarrow \text{sh}_{|C|+i_L, \sigma} + \text{sh}_{|C|+i_R, \sigma} \\ &// \langle \varphi \cdot C(\vec{y}) \cdot (P_{i_L} + P_{i_R})(\vec{x}) \rangle_{\sigma} \end{aligned}$$

- If g'_i is a scalar multiplication gate $\cdot_{\alpha}$, whose parent is g'_j :

$$\begin{aligned} \text{sh}_{|C|+i, \sigma} &\leftarrow \alpha \cdot \text{sh}_{|C|+j, \sigma} \\ &// \langle \varphi \cdot C(\vec{y}) \cdot \alpha \cdot P_j(\vec{x}) \rangle_{\sigma} \leftarrow \alpha \cdot \langle \varphi \cdot C(\vec{y}) \cdot P_j(\vec{x}) \rangle_{\sigma} \end{aligned}$$

- If g'_i is a multiplication gate, whose parents are g'_{i_L} and g'_{i_R} , and g'_{i_L} is the j^{th} input:

$$\begin{aligned} \text{sh}_{|C|+i, \sigma} &\leftarrow \text{DDLog} \left((c_j)^{\text{sh}_{|C|+i_R, \sigma}} \right) + \text{PRF}(k_{\text{PRF}}, |C| + i) \bmod N^{\zeta} \\ &// \langle \varphi \cdot C(\vec{y}) \cdot x_j \cdot P_{i_R}(\vec{x}) \rangle_{\sigma} \end{aligned}$$

// $\text{sh}_{|C|+|C_{\text{rm}}|, \sigma}$ stores $\langle \varphi \cdot C(\vec{y}) \cdot C_{\text{rm}}(\vec{x}) \rangle_{\sigma}$

8. $z_{\sigma} \leftarrow \text{DDLog} \left((c_{\text{inv}})^{\text{sh}_{|C|+|C_{\text{rm}}|, \sigma}} \right) + \text{PRF}(k_{\text{PRF}}, |C| + |C_{\text{rm}}|) \bmod N^{\zeta} // \langle C(\vec{y}) \cdot C_{\text{rm}}(\vec{x}) \rangle_{\sigma}$

9. Output z_{σ}

HSS.AuthEval $(\sigma, \text{ek}_{\sigma}, (x_i^{(\sigma)})_{i \in [n_{\text{priv}}]}, (y_i^{(\sigma)})_{i \in [n_{\text{s-priv}}]}, C \cdot C_{\text{rm}})$:

1. Perform steps 1-8 of Eval
2. $\tilde{z}_{\sigma} \leftarrow (z_{\sigma}, \text{sh}_{|C|+|C_{\text{rm}}|, \sigma}) // \langle (1, \varphi) \cdot C(\vec{y}) \cdot C_{\text{rm}}(\vec{x}) \rangle_{\sigma}$
3. Output $\tilde{z}_{\sigma}$

Fig. 6: Semi-private offline-online HSS from KDM-security of the Damgård-Jurik encryption scheme.

Theorem 18 (Semi-private offline-online HSS for $\text{RMS} \otimes \text{VP}$ from KDM-security of Damgård-Jurik). *Let λ be a security parameter. Let $\zeta \geq 2$, and assume the KDM-security of the Damgård-Jurik cryptosystem with parameter ζ with respect to the function class $(\varphi, N) \mapsto \varphi^{-1}$; let ℓ_{DCR} denote the bit-length of the corresponding RSA modulus. For every $\varepsilon > 0$, every input sizes $n_{\text{priv}}, n_{\text{s-priv}} \in \mathbb{N}^*$, and every bound $B \leq 2^{(\zeta-1-\varepsilon) \cdot \ell_{\text{DCR}}/2}$, the construction of fig. 6 is a semi-private HSS scheme (definition 16) with linear offline/online sharing and*

authenticated evaluation (definition 17), supporting the class $\text{RMS}_{n_{\text{s-priv}}}[B] \otimes_1 \text{VP}_{n_{\text{priv}}}[B, D]$ (definition 14), for an arbitrary pebbling depth $D = \text{poly}(\lambda)$.

The size of the evaluation keys and of each share is $O(\zeta \cdot \lambda_{\text{DCR}})$ bits.

The proof is deferred to appendix A.1.

Remark 19 (Supporting Rank- r Tensoring). As described, the HSS construction of fig. 6 supports evaluating circuits of the form $C \cdot C_{\text{rm}}$, for $C \in \text{VP}[B]$ and $C_{\text{rm}} \in \text{RMS}[B]$, i.e., circuits in $\text{RMS}[B] \otimes_1 \text{VP}[B]$ as the first inputs go to C_{rm} . This easily extends to evaluating any (higher-rank) circuit in the tensor product class $\text{RMS}[B] \otimes \text{VP}[B]$. Any such circuit computes $\sum_{i=1}^r F_i \cdot G_i$, where $F_i \in \text{VP}$, $G_i \in \text{RMS}$, so we can evaluate each product separately and then sum up the r sets of shares to obtain the result.

4.3 (Weakly succinct) semi-private HSS construction from DCR

In this section, we present our second HSS construction, which avoids the KDM security assumption.

4.3.1 Relinearisation, without a circular security assumption. We first present a variant of the relinearisation technique, which avoids the need for the key-dependent encryption of φ^{-1} . Unlike our previous method, which multiplies two authenticated sharings $\langle k \cdot x \rangle, \langle k \cdot y \rangle$ to obtain a new sharing $\langle k \cdot xy \rangle$ under the same key, this method is slightly more restrictive: it starts with sharings $\langle k_1 \cdot x \rangle, \langle k_2 \cdot x \rangle$, for independent secret keys k_1, k_2 , and outputs a new sharing $\langle k_2 \cdot xy \rangle$.

Core Lemma 2 (Relinearisation, Damgård-Jurik-ElGamal variant). *For all $N, \zeta, x, y \in \mathbb{Z}$, $g \in (\mathbb{Z}/N^\zeta\mathbb{Z})^\times$, and $k_1, k_2 \in [0, N)$, if*

1. $\langle k_1 \cdot x \rangle_1, \langle k_1 \cdot x \rangle_0$ denote shares over $\mathbb{Z}$ of $k_1 \cdot x$ (meaning they are integers satisfying $\langle k_1 \cdot x \rangle_1 - \langle k_1 \cdot x \rangle_0 = k_1 x$)
2. $\langle k_2 \cdot y \rangle_1, \langle k_2 \cdot y \rangle_0$ denote shares over $\mathbb{Z}$ of $k_2 \cdot y$
3. c is the matrix $\begin{pmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{pmatrix} := \left(\frac{g}{g^{k_1}} \middle| \frac{g^{k_2}}{g^{k_1 k_2} \cdot \exp(k_2)} \right)$

then $\langle k_2 \cdot xy \rangle_1 - \langle k_2 \cdot xy \rangle_0 \equiv k_2 \cdot xy \pmod{N^\zeta}$, where $\langle k_2 \cdot xy \rangle_1$ and $\langle k_2 \cdot xy \rangle_0$ are defined as follows:

$$\begin{cases} \langle k_2 \cdot xy \rangle_1 & \leftarrow \text{DDLog} \left(c_{11}^{xy} c_{10}^{-x \langle k_2 \cdot y \rangle_1} c_{01}^{-y \langle k_1 \cdot x \rangle_1} c_{00}^{-\langle k_1 \cdot x \rangle_1 \langle k_2 \cdot y \rangle_1} \right) \\ \langle k_2 \cdot xy \rangle_0 & \leftarrow \text{DDLog} \left(c_{00}^{-\langle k_1 \cdot x \rangle_0 \langle k_2 \cdot y \rangle_0} \right) \end{cases}$$

Proof. Consider the following algebraic identity:

$$\begin{aligned} k_1 k_2 \cdot xy &= k_1 \cdot x \cdot \langle k_2 \cdot y \rangle_1 - \overbrace{k_1 \cdot x \cdot \langle k_2 \cdot y \rangle_0}^{= \langle k_1 \cdot x \rangle_1 \cdot \langle k_2 \cdot y \rangle_0 - \langle k_1 \cdot x \rangle_0 \cdot \langle k_2 \cdot y \rangle_0} \\ &= k_1 \cdot x \cdot \langle k_2 \cdot y \rangle_1 - \underbrace{\langle k_1 \cdot x \rangle_1 \cdot \langle k_2 \cdot y \rangle_0}_{= \langle k_1 \cdot x \rangle_1 \cdot \langle k_2 \cdot y \rangle_1 - k_2 \cdot y \cdot \langle k_1 \cdot x \rangle_1} + \langle k_1 \cdot x \rangle_0 \cdot \langle k_2 \cdot y \rangle_0 \\ &= k_1 \cdot x \cdot \langle k_2 \cdot y \rangle_1 - \langle k_1 \cdot x \rangle_1 \cdot \langle k_2 \cdot y \rangle_1 + k_2 \cdot y \cdot \langle k_1 \cdot x \rangle_1 + \langle k_1 \cdot x \rangle_0 \cdot \langle k_2 \cdot y \rangle_0 \end{aligned}$$

It follows that

$$\begin{aligned} c_{11}^{xy} &= g^{k_1 k_2 \cdot xy} \cdot \exp(k_2 \cdot xy) \\ &= c_{10}^{x \langle k_2 \cdot y \rangle_1} \cdot c_{00}^{\langle k_1 \cdot x \rangle_1 \langle k_2 \cdot y \rangle_1} \cdot c_{01}^{y \langle k_1 \cdot x \rangle_1} \cdot c_{00}^{\langle k_1 \cdot x \rangle_0 \langle k_2 \cdot y \rangle_0} \cdot \exp(k_2 \cdot xy) \end{aligned}$$

After reorganising terms,

$$\exp(k_2 \cdot xy) = \left[c_{11}^{xy} c_{10}^{-x\langle k_2 \cdot y \rangle_1} c_{01}^{-y\langle k_1 \cdot x \rangle_1} c_{00}^{-\langle k_1 \cdot x \rangle_1 \langle k_2 \cdot y \rangle_1} \right] / c_{00}^{-\langle k_1 \cdot x \rangle_0 \langle k_2 \cdot y \rangle_0}.$$

We get then get the desired result by [RS21, Theorem 18]. $\square$

Regarding security, note that we need to give out the matrix c , which contains $(g, g^{k_1}, g^{k_2}, g^{k_1 k_2} \cdot \exp(k_2))$. While this can be seen as a kind of ElGamal encryption of k_2 , where k_2 is also the randomness in the ciphertext, we observe that this is indistinguishable from an encryption of a random value, via an argument similar to that used for proving hardness of the extended-DDH assumption under DCR [HO12]. We prove the following in appendix A.2.2.

Lemma 20. *Let $N \xleftarrow{\$} \text{RSA.Gen}(1^\lambda)$, $g \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$, $x, y \xleftarrow{\$} [0, N)$ and $r \xleftarrow{\$} [0, N^\zeta)$. Then, the distributions*

$$(N, g, g^x, g^y, g^{xy} \cdot \exp(y)) \quad \text{and} \quad (N, g, g^x, g^y, g^{xy} \cdot \exp(r))$$

are computationally indistinguishable under the DCR assumption.

4.3.2 Key-switching. Since the circular-secure relinearisation technique changes the underlying key, our HSS construction also needs a *key-switching* mechanism, to ensure that shares input to an addition gate are encoded under the same key.

The **KeySwitch** algorithm, in fig. 7, uses public group elements $h = g^{-k'}$, $c = g^{kk'} \cdot \exp(k')$ to convert a sharing $\langle k' \cdot x \rangle$ into $\langle k \cdot x \rangle$, using **DDLog**. **RecKeySwitch**, shown in fig. 8, uses the same method with a chain of keys k_i , to iteratively convert the input sharing $\langle k_1 \cdot x \rangle$ into $\langle k_d \cdot x \rangle$. Regarding security, notice that starting with the key k_1 , we can apply Lemma 20 to replace the $\exp$ term in $c_1 = g^{k_1 k_2} \cdot \exp(k_2)$ with $\exp(r)$ for a random r . In general, in the i -th step of the proof, we will simulate $g^{k_{i-1} k_i}$ using g^{k_i} , and $g^{k_{i+1} k_{i+2}} \exp(k_{i+2})$ using $g^{k_{i+1}}$, and replace $\exp(k_{i+2})$ with a random element. We give the complete argument in the security proof of the HSS construction, shown in the next section.

Algorithm Key-Switching

KeySwitch $_{N,\zeta}(\langle x \rangle_\sigma, \langle k \cdot x \rangle_\sigma, h, c, r)$: // where $h = g^{-k'}$, $c = g^{kk'} \cdot \exp(k')$

1. Compute $\langle k' \cdot x \rangle_\sigma \leftarrow \text{DDLog}(c^{(x)_\sigma} h^{\langle k \cdot x \rangle_\sigma}) + r \bmod N^\zeta$
2. Output $\langle k' \cdot x \rangle_\sigma$

Fig. 7: Key-switching algorithm.

Algorithm Recursive Key-Switching

RecKeySwitch $_{N,\zeta}(\langle x \rangle_\sigma, \langle k_1 \cdot x \rangle_\sigma, d, (h_i)_{i \in [d]}, (c_i)_{i \in [d-1]}, (r_i)_{i \in [d-1]})$:
// where $h_i = g^{-k_i}$ and $c_i = g^{k_i k_{i+1}} \cdot \exp(k_{i+1})$

1. For $i = 1, \dots, d-1$:
 Compute $\langle k_{i+1} \cdot x \rangle_\sigma \leftarrow \text{DDLog}(c_i^{(x)_\sigma} h_{i+1}^{\langle k_i \cdot x \rangle_\sigma}) + r_i \bmod N^\zeta$
2. Output $\langle k_d \cdot x \rangle_\sigma$

Fig. 8: Recursive key-switching algorithm.

Lemma 21 (Correctness of key-switching). *Let $\zeta \geq 1$. For all $x \in \mathbb{Z}$, $(g, N) \in \text{Supp}(\text{DJE.Setup})$, $k, k' \in [0, N)$, if $\langle x \rangle_1$ and $\langle x \rangle_0$ are shares of x over $\mathbb{Z}$, $\langle k \cdot x \rangle_1$ and $\langle k \cdot x \rangle_0$ are shares of $k \cdot x$ over $\mathbb{Z}$, then*

$$\Pr_{r \xleftarrow{\$} \mathbb{Z}/N^\zeta \mathbb{Z}} [\langle y \rangle_1 - \langle y \rangle_0 = k' \cdot x] \geq 1 - \frac{|x|}{N^{\zeta-1}}$$

where $\langle y \rangle_\sigma \leftarrow \text{KeySwitch}(\langle x \rangle_\sigma, \langle k \cdot x \rangle_\sigma, g^{-k'}, g^{kk'} \exp(k'), r)$, for $\sigma \in \{0, 1\}$.

Moreover, for all $x \in \mathbb{Z}$, $(g, N) \in \text{Supp}(\text{DJE.Setup})$, $d \in \mathbb{N}$, $(k_i)_{i \in [1, d]} \in [0, N)^d$, if $\langle x \rangle_1$ and $\langle x \rangle_0$ are shares of x over $\mathbb{Z}$, $\langle k_1 \cdot x \rangle_1$ and $\langle k_1 \cdot x \rangle_0$ are shares of $k_1 \cdot x$ over $\mathbb{Z}$, then

$$\Pr_{r_1, \dots, r_{d-1} \xleftarrow{\$} \mathbb{Z}/N^\zeta \mathbb{Z}} [\langle y \rangle_1 - \langle y \rangle_0 = k_d \cdot x] \geq 1 - \frac{(d-1) \cdot |x|}{N^{\zeta-1}}$$

where $\langle y \rangle_\sigma = \text{RecKeySwitch}_{N, \zeta} \left(\langle x \rangle_\sigma, \langle k \cdot x \rangle_\sigma, d, (g^{-k_i})_{i \in [d]}, (g^{k_i k_{i+1}} \exp(k_i))_{i \in [d-1]}, (r_i)_{i \in [d-1]} \right)$, for $\sigma \in \{0, 1\}$.

Proof. Let $x \in \mathbb{Z}$, $(g, N) \in \text{Supp}(\text{DJE.Setup})$, $k, k' \in [0, N)$, and $\langle x \rangle_1, \langle x \rangle_0, \langle k \cdot x \rangle_1, \langle k \cdot x \rangle_0 \in \mathbb{Z}$ such that $\langle x \rangle_1 - \langle x \rangle_0 = x$ and $\langle k \cdot x \rangle_1 - \langle k \cdot x \rangle_0 = k \cdot x$.

Defining $h \leftarrow g^{-k'}$ and $c \leftarrow g^{kk'} \cdot \exp(k')$, $c^{\langle x \rangle_1} h^{\langle k \cdot x \rangle_1} / c^{\langle x \rangle_0} h^{\langle k \cdot x \rangle_0} = c^x h^{k \cdot x} = \exp(k' \cdot x)$. It follows by [RS21, theorem 18] that $\text{DDLog}(c^{\langle x \rangle_1} h^{\langle k \cdot x \rangle_1}) - \text{DDLog}(c^{\langle x \rangle_0} h^{\langle k \cdot x \rangle_0}) \equiv k' \cdot x \pmod{N^\zeta}$. Finally, by Lemma 9, we indeed get

$$\Pr_{r \xleftarrow{\$} \mathbb{Z}/N^\zeta \mathbb{Z}} \left[\text{KeySwitch}_{N, \zeta}(\langle x \rangle_1, \langle k \cdot x \rangle_1, g^{-k'}, g^{kk'} \exp(k'), r) - \text{KeySwitch}_{N, \zeta}(\langle x \rangle_0, \langle k \cdot x \rangle_0, g^{-k'}, g^{kk'} \exp(k'), r) = k' \cdot x \right] \geq 1 - \frac{|x|}{N^{\zeta-1}}. \quad (3)$$

Let $x \in \mathbb{Z}$, $(g, N) \in \text{Supp}(\text{DJE.Setup})$, $d \in \mathbb{N}$, $(k_i)_{i \in [1, d]} \in [0, N)^d$, and $\langle x \rangle_1, \langle x \rangle_0, \langle k \cdot x \rangle_1, \langle k \cdot x \rangle_0 \in \mathbb{Z}$ such that $\langle x \rangle_1 - \langle x \rangle_0 = x$ and $\langle k \cdot x \rangle_1 - \langle k \cdot x \rangle_0 = k \cdot x$. For $i \in [d]$, define $h_i \leftarrow g^{-k_i}$ and for $i \in [d-1]$, define $c_i \leftarrow g^{k_i k_{i+1}} \exp(k_{i+1})$.

For $i \in [d-1]$ and $\sigma \in \{0, 1\}$, let $\langle k_{i+1} \cdot x \rangle_\sigma \in \mathbb{Z}$ be defined recursively as $\langle k_{i+1} \cdot x \rangle_\sigma \leftarrow \text{DDLog}((c_i)^{\langle k_i \cdot x \rangle_\sigma} (h_i)^{\langle k_{i+1} \cdot x \rangle_\sigma}) + r_i \pmod{N^\zeta}$. For $i \in [d]$, consider the random variables $r_1, \dots, r_n$ all i.i.d. uniform in $\mathbb{Z}/N^\zeta \mathbb{Z}$ consider the event $E_i: \langle k_i \cdot x \rangle_1 - \langle k_i \cdot x \rangle_0 = k_i \cdot x$. By the first part of this lemma (eq. (3)), $\forall i \in [2, d], \Pr[E_i | E_{i-1}] \geq 1 - |x|/N^{\zeta-1}$. Note that $\Pr[E_1] = 1$ by assumption, and that $\Pr[E_d] \geq \Pr[E_d \wedge E_{d-1}] = \Pr[E_d | E_{d-1}] \cdot \Pr[E_{d-1}] \geq \dots \geq (\prod_{i=2}^d \Pr[E_i | E_{i-1}]) \cdot \Pr[E_1] \geq (1 - |x|/N^{\zeta-1})^{d-1} \geq 1 - (d-1)|x|/N^{\zeta-1}$. Since E_d is exactly the event whose probability we wished to lower bound, this concludes the proof. $\square$

4.3.3 HSS construction from DCR. During homomorphic evaluation of the semi-private part of the circuit, our HSS construction uses key-switching to align secret keys of the input wire shares. Due to this, the size of the evaluation keys, ek_σ , scales linearly with the circuit's pebbling depth. One other difference is that with this version, we can only achieve the regular evaluation procedure HSS.Eval , and not the authenticated evaluation HSS.AuthEval . This makes it slightly more difficult to use in our garbling construction.

HSS Semi-private Offline-Online HSS (Damgård-Jurik-ElGamal variant)

Requires:

- $\text{DJE} = (\text{DJE.KeyGen}, \text{DJE.Enc}, \text{DJE.Dec})$ is the Damgård-Jurik-ElGamal cryptosystem of fig. 3.
- DDLog is the algorithm of fig. 4.
- D is the pebbling depth (definition 13) of the online circuit
- $(\text{PRF}_{N,\zeta,s,t,\lambda})_{(N,\zeta,s,t,\lambda) \in \mathbb{N}^5}$ is an ε_{PRF} -secure family of pseudorandom functions such that $\text{PRF}_{N,\zeta,s,t,\lambda}: ([s] \times [D] \times \{0,1\}) \cup [s] \cup ([s+t+1, s+t] \times \{0,1\}) \rightarrow \mathbb{Z}/N^\zeta\mathbb{Z}$.

HSS.Setup(1^λ):

1. $(g, N) \xleftarrow{\$} \text{DJE.Setup}(1^\lambda)$
2. For $i \in [0, D]$, $(\text{sk}_i = (k_i, N), \text{pk}_i = (g, h_i, N)) \xleftarrow{\$} \text{DJE.KeyGen}(1^\lambda, \text{pp})$
3. For $i \in [0, D]$, set $c_i \leftarrow g^{k_i k_{i+1}} \cdot \exp(k_{i+1})$
4. $\text{sh}_{1,0}^{\text{auth}} \xleftarrow{\$} \mathbb{Z}/N^\zeta\mathbb{Z} // \langle k_0 \cdot 1 \rangle_0$ (over $\mathbb{Z}/N^\zeta\mathbb{Z}$)
5. $\text{sh}_{1,1}^{\text{auth}} \leftarrow (k_0 + \text{sh}_{1,0}^{\text{auth}}) \bmod N^\zeta // \langle k_0 \cdot 1 \rangle_1$ (over $\mathbb{Z}/N^\zeta\mathbb{Z}$)
6. $k_{\text{PRF}} \xleftarrow{\$} \{0,1\}^\lambda$
7. For $\sigma \in \{0,1\}$, $\text{ek}_\sigma \leftarrow (N, g, (h_i)_{i=1}^D, (c_i)_{i=0}^{D-1}, \text{sh}_{1,\sigma}^{\text{auth}}, k_{\text{PRF}})$
8. $\text{sk} \leftarrow (g, (k_i)_{i=0}^D, N)$
9. Output $(\text{sk}, (\text{ek}_0, \text{ek}_1))$

HSS.Share_{priv}($1^\lambda, \text{sk}, x$):

1. Parse $\text{sk} = (g, (k_i)_{i=0}^D, N)$, and compute $\text{pk}_D \leftarrow (g, h_D, N)$
2. $c \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}_D}(x)$
3. $c^{\text{auth}} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}_D}(k_D \cdot x)$
4. For $\sigma \in \{0,1\}$, set $x^{(\sigma)} \leftarrow (c, c^{\text{auth}})$
5. Output $(x^{(0)}, x^{(1)})$

HSS.Share_{0,s-priv}($1^\lambda, \text{sk}$):

1. Parse $\text{sk} = (g, (k_i)_{i=0}^D, N)$
2. $\text{sh}_{\text{s-priv},0}^{\text{auth}} \xleftarrow{\$} \mathbb{Z}/N^\zeta\mathbb{Z} // \langle k_0 \cdot y \rangle_0$ (over $\mathbb{Z}/N^\zeta\mathbb{Z}$)
3. Set $y^{(0)} \leftarrow \text{sh}_{\text{s-priv},0}^{\text{auth}}$ and $\text{st} \leftarrow \text{sh}_{\text{s-priv},0}^{\text{auth}}$
4. Output $(y^{(0)}, \text{st})$

HSS.Share_{1,s-priv}($1^\lambda, \text{sk}, y, \text{st}$):

1. Parse $\text{sk} = (g, (k_i)_{i=0}^D, N)$
2. $\text{sh}_{\text{s-priv},1}^{\text{auth}} \leftarrow (k_0 \cdot y + \text{st}) \bmod N^\zeta // \langle k_0 \cdot y \rangle_1$ (over $\mathbb{Z}/N^\zeta\mathbb{Z}$)
3. Output $y^{(1)} \leftarrow (y, \text{sh}_{\text{s-priv},1}^{\text{auth}})$

HSS.Eval($\sigma, \text{ek}_\sigma, (x_i^{(\sigma)})_{i \in [n_{\text{priv}}]}, (y_i^{(\sigma)})_{i \in [n_{\text{s-priv}}]}, (C, C_{\text{rm}})$):

// C is a B -bounded arithmetic circuit of pebbling depth D , and C_{rm} is a B' -bounded restricted-multiplication arithmetic circuit.

1. Parse $\text{ek}_\sigma \leftarrow (N, g, (h_i)_{i=1}^D, (c_i)_{i=0}^{D-1}, \text{sh}_{1,\sigma}^{\text{auth}}, k_{\text{PRF}})$
2. Parse $y_i^{(\sigma)}$ as

$$y_i^{(\sigma)} = \begin{cases} (\text{sh}_{\text{s-priv},i,1}, \text{sh}_{\text{s-priv},i,1}^{\text{auth}}, y) & \text{if } \sigma = 1 \\ (\text{sh}_{\text{s-priv},i,0}, \text{sh}_{\text{s-priv},i,0}^{\text{auth}}) & \text{if } \sigma = 0 \end{cases}$$

3. Parse C as a sequence of gates $g_1, g_2, \dots, g_{|C|}$.
4. For $i = 1, \dots, |C|$:

// Invariant: $\text{sh}_{i,\sigma}^{\text{auth}}$ stores $\langle k_{d_i} \cdot P_i(\vec{y}) \rangle_\sigma$ (over $\mathbb{Z}$), where P_i is the polynomial computed by gate g_i and d_i is the pebbling depth of g_i

 - If g_i is the constant gate 1,

$$\text{sh}_{i,\sigma}^{\text{auth}} \leftarrow \text{sh}_{1,\sigma}^{\text{auth}} // \langle k_0 \cdot 1 \rangle_\sigma$$
 - If g_i is the j^{th} input gate,

$$\text{sh}_{i,\sigma}^{\text{auth}} \leftarrow \text{sh}_{\text{s-priv},j,\sigma}^{\text{auth}} // \langle k_0 \cdot y_j \rangle_\sigma$$
 - If g_i is an addition gate at pebbling depth d_i , whose parents are g_{i_L} (at pebbling depth d_L) and g_{i_R} (at pebbling depth d_{i_R}):

$$\begin{aligned} \text{sh}_{i,L,\sigma}^{\text{auth},d_i} &\leftarrow \text{RecKeySwitch}_{N,\zeta}(\text{sh}_{i_L,\sigma}, \text{sh}_{i_L,\sigma}^{\text{auth}}, d_i - d_{i_L}, \\ &\quad ((h_i)_{i \in [d_{i_L}, d_i]}, (c_j)_{j \in [d_{i_L}, d_i-1]}, \\ &\quad (\text{PRF}(k_{\text{PRF}}, (i, j, 0)))_{j \in [d_{i_L}, d_i-1]}) \\ &// \langle k_{\ell_i} \cdot P_{i_L}(\vec{y}) \rangle_\sigma \\ \text{sh}_{i,R,\sigma}^{\text{auth},d_i} &\leftarrow \text{RecKeySwitch}_{N,\zeta}(\text{sh}_{i_R,\sigma}, \text{sh}_{i_R,\sigma}^{\text{auth}}, d_i - d_{i_R}, \\ &\quad (h_i)_{i \in [d_{i_R}, d_i]}, (c_j)_{j \in [d_{i_R}, d_i-1]}, \\ &\quad (\text{PRF}(k_{\text{PRF}}, (i, j, 1)))_{j \in [d_{i_R}, d_i-1]}) \\ &// \langle k_{\ell_i} \cdot P_{i_R}(\vec{y}) \rangle_\sigma \\ \text{sh}_{i,\sigma}^{\text{auth}} &\leftarrow \text{sh}_{i,L,\sigma}^{\text{auth},d_i} + \text{sh}_{i,R,\sigma}^{\text{auth},d_i} \\ &// \langle k_{\ell_i} \cdot P_i(\vec{y}) \rangle_\sigma \leftarrow \langle k_{\ell_i} \cdot (P_{i_L} + P_{i_R})(\vec{y}) \rangle_\sigma \end{aligned}$$
 - If g_i is a multiplication gate at pebbling depth d_i , whose parents are g_{i_L} (at

pebbling depth d_L) and g_{i_R} (at pebbling depth d_{i_R}):

$$\begin{aligned}
\text{sh}_{i_L, \sigma} &\leftarrow \begin{cases} P_{i_L}(\vec{y}) & \text{If } \sigma = 1 \\ 0 & \text{If } \sigma = 0 \end{cases} \\
\text{sh}_{i_R, \sigma} &\leftarrow \begin{cases} P_{i_R}(\vec{y}) & \text{If } \sigma = 1 \\ 0 & \text{If } \sigma = 0 \end{cases} \\
\text{sh}_{i_L, \sigma}^{\text{auth}, d_i-1} &\leftarrow \text{RecKeySwitch}_{N, \zeta}(\text{sh}_{i_L, \sigma}, \text{sh}_{i_L, \sigma}^{\text{auth}}, d_i - d_{i_L} - 1, \\
&\quad (h_j)_{j \in [d_{i_L}, d_i-1]}, (c_j)_{j \in [d_{i_L}, d_i-2]}, \\
&\quad (\text{PRF}(k_{\text{PRF}}, (i, j, 0)))_{j \in [d_{i_L}, d_i-1]}) \\
\text{sh}_{i_R, \sigma}^{\text{auth}, d_i} &\leftarrow \text{RecKeySwitch}_{N, \zeta}(\text{sh}_{i_R, \sigma}, \text{sh}_{i_R, \sigma}^{\text{auth}}, d_i - d_{i_R}, \\
&\quad (h_j)_{j \in [d_{i_R}, d_i]}, (c_j)_{j \in [d_{i_R}, d_i-1]}, \\
&\quad (\text{PRF}(k_{\text{PRF}}, (i, j, 1)))_{j \in [d_{i_R}, d_i-1]}) \\
&\quad // \langle k_{d_i} \cdot P_{i_R}(\vec{y}) \rangle_{\sigma} \\
\text{sh}_{i, \sigma}^{\text{auth}} &\leftarrow \begin{cases} -\text{DDLog} \left((c_{d_i-1})^{P_{i_L}(\vec{y}) \cdot P_{i_R}(\vec{y})} \right. & \text{if } \sigma = 1 \\
\quad \cdot (h_i)^{P_{i_L}(\vec{y}) \cdot \text{sh}_{i, R, 1}^{\text{auth}, d_i}} \\
\quad \cdot (h_{i+1})^{P_{i_R}(\vec{y}) \cdot \text{sh}_{i, L, 1}^{\text{auth}, d_i-1}} \\
\quad \left. \cdot g^{-\text{sh}_{i, L, 1}^{\text{auth}, d_i-1} \cdot \text{sh}_{i, R, 1}^{\text{auth}, d_i}} \right) & \\
-\text{DDLog} \left(g^{-\text{sh}_{i, L, 0}^{\text{auth}, d_i-1} \cdot \text{sh}_{i, R, 0}^{\text{auth}, d_i}} \right) & \text{if } \sigma = 0 \end{cases} \\
\text{sh}_{i, \sigma}^{\text{auth}} &\leftarrow \text{sh}_{i, \sigma}^{\text{auth}} + \text{PRF}(k_{\text{PRF}}, i) \bmod N^{\zeta} \\
&\quad // \langle k_{d_i} \cdot P_i(\vec{y}) \rangle_{\sigma} = \langle k_{d_i} \cdot (P_{i_L} P_{i_R})(\vec{y}) \rangle_{\sigma}
\end{aligned}$$

– If g_i is a scalar multiplication gate $\cdot_{\alpha}$, whose parent is g_j :

$$\text{sh}_{i, \sigma}^{\text{auth}} \leftarrow \alpha \cdot \text{sh}_{j, \sigma}^{\text{auth}} \quad // \langle k_{d_i} \cdot \alpha \cdot P_j(\vec{y}) \rangle_{\sigma} \leftarrow \alpha \cdot \langle k_{d_i} \cdot P_j(\vec{y}) \rangle_{\sigma}$$

// $\text{sh}_{|C|, \sigma}^{\text{auth}}$ stores $\langle k_D \cdot C(\vec{y}) \rangle_{\sigma}$

// Now, evaluate C_{rm} on private inputs

5. For $i \in [n_{\text{priv}}]$, parse $x_i^{(\sigma)}$ as $(\text{sh}_{x_i, \sigma}^{\text{auth}}, c_i = (c_{i,0}, c_{i,1}), c_i^{\text{auth}} = (c_{i,0}^{\text{auth}}, c_{i,1}^{\text{auth}}))$

6. Parse C_{rm} as a sequence of gates $g'_1, g'_2, \dots, g'_{|C_{\text{rm}}|}$.

7. For $i = 1, \dots, |C_{\text{rm}}|$:

// Invariant: $\text{sh}_{|C|+i, \sigma}$ stores $\langle C(\vec{y}) \cdot P_i(\vec{x}) \rangle_{\sigma}$ and $\text{sh}_{|C|+i, \sigma}^{\text{auth}}$ stores $\langle k_D \cdot C(\vec{y}) \cdot P_i(\vec{x}_{\text{priv}}) \rangle_{\sigma}$, where P_i is the polynomial computed by gate g'_i

– If g'_i is the constant 1:

$$\begin{aligned}
\text{sh}_{|C|+i, \sigma} &\leftarrow \begin{cases} C(\vec{y}) & \text{if } \sigma = 1 \\ 0 & \text{if } \sigma = 0 \end{cases} // \langle C(\vec{y}) \cdot 1 \rangle_{\sigma} \\
\text{sh}_{|C|+i, \sigma}^{\text{auth}} &\leftarrow \text{sh}_{|C|, \sigma}^{\text{auth}} // \langle k_D \cdot C(\vec{y}) \cdot 1 \rangle_{\sigma} \leftarrow \langle k_D \cdot C(\vec{y}) \rangle_{\sigma}
\end{aligned}$$

– If g'_i is the j^{th} input gate:

$$\begin{aligned}
\text{sh}_{|C|+i, \sigma} &\leftarrow \text{DDLog} \left((c_{j,0})^{\text{sh}_{|C|, \sigma}^{\text{auth}}} (c_{j,1})^{\text{sh}_{|C|, \sigma}} \right) + \text{PRF}_{k_{\text{PRF}}}(|C| + i, 0) \bmod N^{\zeta} \\
&\quad // \langle C(\vec{y}) \cdot x_j \rangle_{\sigma} \\
\text{sh}_{|C|+i, \sigma}^{\text{auth}} &\leftarrow \text{DDLog} \left((c_j^{\text{auth}})^{\text{sh}_{|C|, \sigma}^{\text{auth}}} \right) + \text{PRF}(k_{\text{PRF}}, (|C| + i, 1)) \bmod N^{\zeta} \\
&\quad // \langle k_D \cdot C(\vec{y}) \cdot x_j \rangle_{\sigma}
\end{aligned}$$

- If g'_i is an addition gate, whose parents are g'_{i_L} and g'_{i_R} :

$$\begin{aligned} \text{sh}_{|C|+i,\sigma} &\leftarrow \text{sh}_{|C|+i_L,\sigma} + \text{sh}_{|C|+i_R,\sigma} \\ // \langle C(\vec{y}) \cdot (P_{i_L} + P_{i_R})(\vec{x}) \rangle_\sigma \\ \text{sh}_{|C|+i,\sigma}^{\text{auth}} &\leftarrow \text{sh}_{|C|+i_L,\sigma}^{\text{auth}} + \text{sh}_{|C|+i_R,\sigma}^{\text{auth}} \\ // \langle k_D \cdot C(\vec{y}) \cdot (P_{i_L} + P_{i_R})(\vec{x}) \rangle_\sigma \end{aligned}$$

- If g'_i is a scalar multiplication gate $\cdot_\alpha$, whose parent is g'_j :

$$\begin{aligned} \text{sh}_{|C|+i,\sigma} &\leftarrow \alpha \cdot \text{sh}_{|C|+j,\sigma} \\ // \langle C(\vec{y}) \cdot \alpha \cdot P_j(\vec{x}) \rangle_\sigma &\leftarrow \alpha \cdot \langle C(\vec{y}) \cdot P_j(\vec{x}) \rangle_\sigma \\ \text{sh}_{|C|+i,\sigma}^{\text{auth}} &\leftarrow \alpha \cdot \text{sh}_{|C|+j,\sigma}^{\text{auth}} \\ // \langle k_D \cdot C(\vec{y}) \cdot \alpha \cdot P_j(\vec{x}) \rangle_\sigma &\leftarrow \alpha \cdot \langle k_D \cdot C(\vec{y}) \cdot P_j(\vec{x}) \rangle_\sigma \end{aligned}$$

- If g'_i is a multiplication gate, whose parents are g'_{i_L} and g'_{i_R} , and g'_{i_L} is the j^{th} input:

$$\begin{aligned} \text{sh}_{|C|+i,\sigma} &\leftarrow \text{DDLog} \left((c_{j,0})^{\text{sh}_{|C|+i_R,\sigma}^{\text{auth}}} (c_{j,1})^{\text{sh}_{|C|+i_R,\sigma}} \right) \\ &\quad + \text{PRF}_{k_{\text{PRF}}}(|C| + i, 0) \bmod N^\zeta \\ // \langle C(\vec{y}) \cdot P_{i_R}(\vec{x}) \rangle_\sigma \\ \text{sh}_{|C|+i,\sigma}^{\text{auth}} &\leftarrow \text{DDLog} \left((c_{j,0}^{\text{auth}})^{\text{sh}_{|C|+i_R,\sigma}^{\text{auth}}} (c_{j,1}^{\text{auth}})^{\text{sh}_{|C|+i_R,\sigma}} \right) \\ &\quad + \text{PRF}_{k_{\text{PRF}}}(|C| + i, 1) \bmod N^\zeta \\ // \langle k_D \cdot C(\vec{y}) \cdot P_{i_R}(\vec{x}) \rangle_\sigma \end{aligned}$$

8. $y_\sigma \leftarrow \text{sh}_{|C|+|C_{\text{rm}}|,\sigma}^{\text{auth}} // \langle C(\vec{y}) \cdot C_{\text{rm}}(\vec{x}) \rangle_\sigma$
9. Output y_σ

Fig. 9: Semi-private offline-online HSS from DCR, using the Damgård-Jurik-ElGamal encryption scheme.

Theorem 22 (Semi-private offline-online HSS for $\text{RMS} \otimes \text{VP}$ from DCR). *Let λ be a security parameter and assume the DCR assumption with λ_{DCR} -bit RSA modulus. For every $\zeta \geq 1$, every $\varepsilon = \omega(1)$,² every input sizes $n_{\text{priv}}, n_{\text{s-priv}} \in \mathbb{N}^*$, every bound $B \leq 2^{(\zeta-1-\varepsilon) \cdot \lambda_{\text{DCR}}/2}$ and every pebbling depth $D = \text{poly}(\lambda)$, the construction of fig. 9 is a semi-private offline-online HSS scheme (definition 16) with linear online-input shares (definition 17), supporting the class $\text{RMS}_{n_{\text{s-priv}}}[B] \otimes_1 \text{VP}_{n_{\text{priv}}}[B, D]$ (definition 14).*

The size of the evaluation keys is $O(D \cdot \zeta \cdot \lambda_{\text{DCR}})$ bits, and the share size is $O(\zeta \cdot \lambda_{\text{DCR}})$ bits.

As in the previous construction (see remark 19), this also extends to support evaluating the tensor product class $\text{RMS} \otimes \text{VP}$, beyond the rank 1 products in $\text{RMS} \otimes_1 \text{VP}$.

5 Sublinear-Size Garbled Circuits

Our garbling constructions are designed to garble some class of circuits consisting of higher fan-in gates (which we call wide-gates), with constant rate, i.e., a constant size of plaintext values for every gate. Since layered fan-in 2 circuits can be written as smaller wide-gate circuits, with size sublinear in the size of the fan-in 2 circuit, this implies sublinear garbling

² ε could be chosen as a constant, provided $(1 - 2^{-\varepsilon \cdot \lambda_{\text{DCR}}})^D$ were still negligible.

for fan-in 2 circuits.

We base our constant rate garbling schemes on semi-private HSS with offline/online sharing. At a high level, they work as follows: The garbler chooses masks for every wire in the circuit, and gives the evaluator semi-private shares of its masked inputs. These masks and masked values work similarly to the “permute bits” and “color bits” of point-permute garbling, and ensure that the masked values do not leak anything about the plaintext computation. Each (masked) wide-gate is garbled using semi-private HSS, allowing the evaluator to learn its masked output given its masked inputs. This garbled gate also outputs semi-private shares of the masked output, so that the evaluator can evaluate further gates.

In section 5.1 we discuss our requirements on the class of wide-gates. In section 5.2 we present our full construction of garbling of wide-gate circuits. Finally, section 5.3 presents two suitable classes of wide-gates – boolean truth tables and arithmetic multivariate polynomials – and applies them to fan-in two garbling of boolean and bounded arithmetic circuits.

5.1 Wide-gates and Masking Schemes

Definition 23 (Wide-gates). Let $\mathcal{X}$ be the plaintext space. A class of wide-gates over $\mathcal{X}$ is a class of functions $\mathcal{G} \subseteq \bigcup_{\ell} (\mathcal{X}^{\ell} \rightarrow \mathcal{X})$, possibly with multiple different arities ℓ .

The inputs and outputs of every wide-gate in our garbling scheme must be masked, so that semi-private shares do not leak anything important. We define masking schemes as essentially one-time pads for the plaintext wire values.

Definition 24 (Masking Scheme). Let λ be a security parameter, and let $(\mathcal{K}_{\lambda})_{\lambda \in \mathbb{N}}, (\mathcal{X}_{\lambda})_{\lambda \in \mathbb{N}}, (\mathcal{Y}_{\lambda})_{\lambda \in \mathbb{N}}$ be infinite families of finite sets. A masking scheme (w.r.t. $(\mathcal{K}_{\lambda})_{\lambda \in \mathbb{N}}, (\mathcal{X}_{\lambda})_{\lambda \in \mathbb{N}}, (\mathcal{Y}_{\lambda})_{\lambda \in \mathbb{N}}$) is a pair of p.p.t. algorithms $(\text{Mask}, \text{UnMask})$ with the following syntax and properties.

- **Mask** takes as input the security parameter 1^{λ} , a mask $k \in \mathcal{K}$, and a plaintext value $x \in \mathcal{X}$, and outputs a masked value $y \in \mathcal{Y}$; when convenient, we may use $\text{Mask}_k(x)$ as shorthand for $\text{Mask}(1^{\lambda}, k, x)$.
- **UnMask** takes as input the security parameter 1^{λ} , a masking key $k \in \mathcal{K}$, and a masked value $y \in \mathcal{Y}$, and outputs a plaintext value $x \in \mathcal{X}$; when convenient, we may use $\text{UnMask}_k(y)$ as shorthand for $\text{UnMask}(1^{\lambda}, k, y)$.
- **Correctness.** There exists a negligible function negl such that

$$\forall x \in \mathcal{X}_{\lambda}, \Pr [\text{UnMask}_k(\text{Mask}_k(x)) = x] \geq 1 - \text{negl}(\lambda) .$$

- **One-Time Privacy.** There exists a PPT simulator Sim such that for all $x \in \mathcal{X}_{\lambda}$, the following distributions are computationally indistinguishable:

$$[y \xleftarrow{\$} \text{Sim}(1^{\lambda})] \text{ and } [y \leftarrow \text{Mask}_k(x) \mid k \xleftarrow{\$} \mathcal{K}]$$

Next, while evaluating each gate we must be able to unmask, evaluate some gate, then mask its output, all inside semi-private HSS. We formalize this by saying that the masking scheme and wide-gate class must be admissible for the circuit class supported by the HSS.

Definition 25. A pair of a masking scheme $(\text{Mask}, \text{UnMask})$ and a class $\mathcal{G}$ of wide-gates is admissible for a circuit class $\mathcal{C}$ if, for all $g \in \mathcal{G}$, the function

$$(k_1, \dots, k_{\ell}, k', y_1, \dots, y_{\ell}) \mapsto \text{Mask}_{k'}(g(\text{UnMask}_{k_1}(y_1), \dots, \text{UnMask}_{k_{\ell}}(y_{\ell})))$$

is in $\mathcal{C}$. Additionally, a triplet of masking scheme, wide-gate class, and pseudorandom function F is admissible for $\mathcal{C}$ if, for all $i_1, \dots, i_{\ell}, i' \in \text{Dom}(F_k)$,

$$(k, y_1, \dots, y_{\ell}) \mapsto \text{Mask}_{F_k(i')} (g(\text{UnMask}_{F_k(i_1)}(y_1), \dots, \text{UnMask}_{F_k(i_{\ell})}(y_{\ell})))$$

is in $\mathcal{C}$.

We need this stronger version of admissibility because we need to compress the masking keys using a pseudorandom function, similarly to hybrid encryption. However, for the circuit class $\text{RMS} \otimes \text{VP}$ supported by our offline-online HSS constructions, there's no difference.

Lemma 26. *Let F be a pseudorandom function in NC^1 , and let $((\text{Mask}, \text{UnMask}), \mathcal{G})$ be admissible for $\text{RMS} \otimes \text{VP}$. Then $((\text{Mask}, \text{UnMask}), \mathcal{G}, F)$ is admissible for $\text{RMS} \otimes \text{VP}$.*

Proof. The only difference between the two notions of admissibility is setting the mask keys k_j to be PRF evaluations $F_k(i_j)$. This is local to just the RMS side of the tensor product. Since F is in NC^1 , it can be evaluated by an RMS program of size $\text{poly}(\lambda)$. One can compose two RMS programs, with the size being at most the product of the two original sizes, so after composing with F the RMS side of the tensor product will still be in RMS. $\square$

5.2 General Template for Rate-1 Garbling from Offline-Online HSS

Definition 27. *A garbling scheme AGC for a class of n -input m -output arithmetic functions $\mathcal{C} \subseteq (\mathbb{Z}^m)^{\mathbb{Z}^n}$ and bounded input set $\mathcal{B} \subseteq \mathbb{Z}^n$ is a label group $\mathcal{L}$ together with p.p.t. algorithms:*

- $(\widehat{C}, (K_i)_{i \in [n]}) \leftarrow \text{Garble}(C)$: *garble a circuit $C \in \mathcal{C}$ to get a garbled circuit $\widehat{C} \in \{0, 1\}^*$ and input wire label encoding functions $K_i : \mathbb{Z} \rightarrow \mathcal{L}$ for all $i \in [n]$. Every K_i must be an affine function.*
- $(y_j)_{j \in [m]} \leftarrow \text{Eval}(\widehat{C}, (L_i)_{i \in [n]})$: *evaluate a garbled circuit $\widehat{C}$ on input wire labels $L_i \in \mathcal{L}$ to get outputs $y_j \in \mathbb{Z}$.*

Correctness. *AGC is ε -correct if for all $C \in \mathcal{C}$ and $(x_i)_{i \in [n]} \in \mathcal{B}$,*

$$\Pr \left[\text{Eval}(\widehat{C}, (L_i)_{i \in [n]}) \neq_{\mathbb{Z}} C(x_1, \dots, x_n) \mid \begin{array}{l} (\widehat{C}, (K_i)_{i \in [n]}) \leftarrow \text{Garble}(C) \\ L_i \leftarrow K_i(x_i) \end{array} \right] \leq \varepsilon.$$

Privacy. *AGC is private if there exists a p.p.t. simulator Sim such that for all $C \in \mathcal{C}$ and $(x_i)_{i \in [n]} \in \mathcal{B}$, the following distributions are indistinguishable.*

$$\left\{ ((L_i)_{i \in [n]}, \widehat{C}) : \begin{array}{l} (\widehat{C}, (K_i)_{i \in [n]}) \leftarrow \text{Garble}(C) \\ L_i \leftarrow K_i(x_i) \end{array} \right\} \stackrel{c}{\approx} \{ \mathcal{S}(C, y) : y \leftarrow C(x) \}.$$

5.2.1 From semi-private HSS. In this section, we provide a general template for rate-1 garbling schemes, given a masking scheme and a semi-private HSS (with linear, offline/online sharing of semi-private inputs and authenticated evaluation) that is expressive enough to capture “unmask, evaluating a single gate, then re-mask”.

Our garbling scheme works by having the evaluator iteratively compute a masked representation $\text{Mask}_{R_w}(w)$ of each wire w . The garbled circuit will then contain the mask of the output wire, allowing the evaluator to retrieve its desired output.

We use semi-private homomorphic sharing, where the masked wires are the semi-private inputs and the masks are the private inputs, in order to allow the evaluator to go from masked representations of the input wires of a gate to a masked representation of its output wire. Moreover, the additional properties we require of the HSS scheme (namely linear, offline/online sharing of the semi-private inputs and authenticated evaluation) will guarantee a form of “self reducibility” which will guarantee each gate’s output wire is ready to be used as an input wire to the next gate.

At a high level, a semi-private HSS scheme satisfies linear, offline/online sharing of the semi-private inputs if the first share of a semi-private input y consists of $\langle \text{sk} \cdot y \rangle_0$ while the second share consists of $(y, \langle \text{sk} \cdot y \rangle_1)$. After performing authenticated evaluation, the parties hold $\langle f(x, y) \rangle_\sigma$ and $\langle \text{sk} \cdot f(x, y) \rangle_\sigma$ (where x is some private input). If the first party sends $\langle f(x, y) \rangle_0$ to the second, they now hold $\langle \text{sk} \cdot f(x, y) \rangle_0$ and $(f(x, y), \langle \text{sk} \cdot f(x, y) \rangle_1)$. These values are now *almost* consistent with the sharing of a semi-private input, except there is no guarantee that the $\langle \text{sk} \cdot f(x, y) \rangle_\sigma$ are sampled uniformly at random. However, all is well if garbler and evaluator offset these shares by a pseudorandom value.³

Combining all of the above ideas yields a garbling scheme, but which is not rate-1:

1. The garbler initially performs the HSS setup, samples the masks for each of the wires, shares the masks (treated as private inputs), samples their offline shares of the masked inputs (treated as public inputs), and samples the PRF key used for share re-randomisation.
2. The garbler then proceeds gate by gate to compute $\langle \text{Mask}_{R_w}(w) \rangle_0$ and $\langle \text{sk} \cdot \text{Mask}_{R_w}(w) \rangle_0$ for each wire w of the circuit. Specifically, if gate f takes as input the wires $w_1, \dots, w_\ell$ and produces as output the wire w , then the garbler can convert private-input shares of $R_1, \dots, R_\ell, R$ (the masks of the ℓ inputs to the gate as well as that of its output) and semi-private-input shares of $\text{Mask}_{R_{w_1}}(w_1), \dots, \text{Mask}_{R_{w_\ell}}(w_\ell)$ into $\langle \text{Mask}_{R_w}(w) \rangle_0, \langle \text{sk} \cdot \text{Mask}_{R_w}(w) \rangle_0$ by homomorphically evaluating

$$\left(\underbrace{((X_i)_{i=1}^\ell, X)}_{\text{Private Inputs}}, \underbrace{(Y_i)_{i=1}^\ell}_{\text{Semi-private Inputs}} \right) \mapsto \text{Mask}_X(f(\text{UnMask}_{X_1}(Y_1), \dots, \text{UnMask}_{X_\ell}(Y_\ell))) .$$

3. The garbled circuit is then comprised of the PRF key, the second party's HSS evaluation key, $\langle \text{Mask}_{R_w}(w) \rangle_0$ and the second party's private-input-shares of R_w for each wire w , and the masks for the output wires.
4. An input label for input x is computed as $(\text{Mask}_{R_x}(x), \langle \text{sk} \cdot \text{Mask}_{R_x}(x) \rangle_1)$.
5. At evaluation time, the evaluator proceeds gate-by-gate, computing $\langle \text{Mask}_{R_w}(w) \rangle_1$ and $\langle \text{sk} \cdot \text{Mask}_{R_w}(w) \rangle_1$ analogously to the garbler, and then using $\langle \text{Mask}_{R_w}(w) \rangle_0$ (which is part of the garbled circuit) to reconstruct $\text{Mask}_{R_w}(w)$.
6. Finally, having computed $\text{Mask}_{R_w}(w)$ for the output wire w , the evaluator uses R_w (which is part of the garbled circuit) to retrieve the output value.

The first observation towards making this scheme rate-1 is that the garbled circuit need not contain $\langle \text{Mask}_{R_w}(w) \rangle_0$, only $\langle \text{Mask}_{R_w}(w) \rangle_0 \bmod |\mathcal{Y}|$, where $\text{Mask}_{R_w}(w) \in \mathcal{Y}$ outputs in an interval $[0, |\mathcal{Y}|)$. If the masking scheme is rate-1 then this value is only about as large as the wire value. The second observation is that if all the masks are generated using a PRF F to be evaluated inside the HSS, the garbled circuit only needs to contain the private-input-shares of the bits of the PRF key, not the private-input-shares of each mask. This means the HSS scheme now needs to support the following evaluation:

$$\left(\underbrace{((X_i)_{i=1}^\lambda, X)}_{\text{Private Inputs}}, \underbrace{(Y_i)_{i=1}^\ell}_{\text{Semi-private Inputs}} \right) \mapsto \text{Mask}_X \left(f \left(\text{UnMask}_{F_{X_1 \dots X_\lambda}(i_1)}(Y_1), \dots, \text{UnMask}_{F_{X_1 \dots X_\lambda}(i_\ell)}(Y_\ell) \right) \right)$$

where $i_1, \dots, i_\ell$ are the indices of the gate's input wires. This is what admissibility (definition 25) guarantees.

³ Garbler and evaluator can know the PRF key: in the reduction to PRF security, the adversary is the next call to `HSS.AuthEval`.

Arithmetic Garbling AGC from HSS (for wide-gate circuits)

Requires:

- Bound S on the circuit size.
- A semi-private HSS with modulus M linear, offline/online sharing of semi-private inputs, and supporting authenticated evaluation.
- An admissible triplet for the HSS's circuit class, consisting of:
 1. A class $\mathcal{G}$ of gates over $\mathcal{X}$.
 2. A masking scheme $(\mathcal{K}, \mathcal{X}, \mathcal{Y}, \text{Mask}, \text{UnMask})$, where the space $\mathcal{Y}$ of masked values is an integer interval $[0, |\mathcal{Y}|)$.
 3. A pseudorandom function $F: \{0, 1\}^\lambda \times [S] \rightarrow \mathcal{K}$.
- A pseudorandom function $\text{PRF}: \{0, 1\}^\lambda \times [S] \rightarrow \mathbb{Z}/M\mathbb{Z}$.

The Garbling Scheme (gate-by-gate description):

Initialisation (performed by the garbler).

1. $k_{\text{PRF}} \xleftarrow{\$} \{0, 1\}^\lambda$
2. $(\text{sk}, (\text{ek}_0, \text{ek}_1)) \xleftarrow{\$} \text{HSS.Setup}(1^\lambda)$
3. $\vec{r} = (r_1, \dots, r_\lambda) \xleftarrow{\$} \{0, 1\}^\lambda$
4. For $j \in [\lambda]$, $(r_j^{(0)}, r_j^{(1)}) \xleftarrow{\$} \text{HSS.Share}_{\text{priv}}(1^\lambda, \text{sk}, r_j)$
5. For each $i \in [n]$, $w_i^{(0)} \xleftarrow{\$} \text{HSS.Share}_{0, \text{s-priv}}(1^\lambda, \text{sk})$

HSS circuits (computed by both parties).

Garbling the i^{th} gate $f \in \mathcal{G}$, whose input wires are indexed $i_1, \dots, i_\ell$.

1. Let C_i be the circuit (in the circuit class of HSS) evaluating the function

$$g_i(\vec{r}, y_1, \dots, y_\ell) = \text{Mask}_{F_{\vec{r}}(i)}(f(\text{UnMask}_{F_{\vec{r}}(i_1)}(y_1), \dots, \text{UnMask}_{F_{\vec{r}}(i_\ell)}(y_\ell))).$$
2. $(y_i^{(0)}, \tilde{w}_i^{(0)}) \xleftarrow{\$} \text{HSS.AuthEval}(0, \text{ek}_0, (r_j^{(0)})_{j \in [\lambda]}, (w_{i_j}^{(0)})_{j \in [\ell]}, C_i)$
3. $w_i^{(0)} \leftarrow (\tilde{w}_i^{(0)} + \text{PRF}(k_{\text{PRF}}, i)) \bmod M$

Garbler output. The garbler outputs the garbled circuit, comprised of $k_{\text{PRF}}, \text{ek}_1, r_j^{(1)}$ for $j \in [\lambda]$, $\bar{y}_i^{(0)} \leftarrow (y_i^{(0)} \bmod |\mathcal{Y}|)$ for each of the gates (indexed by i), and $R_j := F_{\vec{r}}(j)$ for each output gate index j .

Label generation. For each $i \in [n]$, given input x_i , compute the label $(y_i, w_i^{(1)}) \xleftarrow{\$} \text{HSS.Share}_{1, \text{s-priv}}(1^\lambda, \text{sk}, w_i^{(0)}, \text{Mask}_{F_{\vec{r}}(i)}(x_i))$.

Evaluating the i^{th} gate $f \in \mathcal{G}$, whose input wires are indexed $i_1, \dots, i_\ell$.

1. Parse the garbled circuit as $(k_{\text{PRF}}, \text{ek}_1, (r_j^{(1)})_{j \in [\lambda]}, (\bar{y}_i^{(0)})_i, (R_j)_j)$
2. Define C_i as at garbling time.
3. $(y_i^{(1)}, \tilde{w}_i^{(1)}) \xleftarrow{\$} \text{HSS.AuthEval}(1, \text{ek}_1, (r_j^{(1)})_{j \in [\lambda]}, (y_{i_j}, w_{i_j}^{(1)})_{j \in [\ell]}, C_i)$
4. $y_i \leftarrow (y_i^{(1)} - \bar{y}_i^{(0)}) \bmod |\mathcal{Y}|$
5. $w_i^{(1)} \leftarrow \tilde{w}_i^{(1)} + \text{PRF}(k_{\text{PRF}}, i) \bmod M$

Evaluator output. For each output gate index j , output $\text{UnMask}_{R_j}(y_j)$.

Fig. 10: Template for (arithmetic or boolean) garbled circuits from semi-private HSS with linear offline/online sharing of semi-private inputs and supporting authenticated evaluation.

Theorem 28 (Template for (arithmetic or boolean) garbled circuits from semi-private HSS with linear offline/online sharing of semi-private inputs and supporting authenticated evaluation). *Let $\mathcal{G}$ be a set of wide-gates over $\mathcal{X}$, $(\text{Mask}, \text{UnMask})$ be a masking scheme with plaintext space $\mathcal{X}$ and masked value space $\mathcal{Y} = [0, |\mathcal{Y}|)$, and F be a PRF. If this triplet is admissible for some circuit class for which we have a semi-private HSS scheme with linear, offline/online sharing of semi-private inputs and supporting authenticated evaluation, then the construction of fig. 10 is an arithmetic garbling scheme for $\mathcal{G}$ wide-gate circuits. For a size s wide-gate circuit with n inputs and m outputs, the size of the garbled circuit and labels is $s \log(|\mathcal{Y}|) + m \log(|\mathcal{K}|) + (n + 1)\text{poly}(\lambda)$.*

Proof.

- **Strong correctness.** Let us prove a stronger notion of correctness, namely that with all but negligible probability the following invariants are maintained for every wire:

$$\begin{aligned} A_i: & \quad "y_i = \text{Mask}_{F_{\vec{r}(i)}}(w_i)" \quad (\text{for } i > n \text{ only}) \\ B_i: & \quad "w_i^{(1)} - w_i^{(0)} = \text{sk} \cdot \text{Mask}_{F_{\vec{r}(i)}}(w_i)" \quad (\text{for } i \geq 1) \end{aligned}$$

First observe that $\forall i \in [n]$, $\Pr[B_i] = 1$ by definition of $\text{Share}_{0, \text{s-priv}}$ and $\text{Share}_{1, \text{s-priv}}$. Now, assume towards a contradiction that there is a non-negligible probability that there exists a wire index $i \geq n$ such that $(y_i \neq \text{Mask}_{F_{\vec{r}(i)}}(w_i)) \vee (w_i^{(1)} - w_i^{(0)} \neq \text{sk} \cdot \text{Mask}_{F_{\vec{r}(i)}}(w_i))$. By a union bound, because there are only polynomially many wires, there exists a wire index $i \geq n$ such that $(y_i \neq \text{Mask}_{F_{\vec{r}(i)}}(w_i)) \vee (w_i^{(1)} - w_i^{(0)} \neq \text{sk} \cdot \text{Mask}_{F_{\vec{r}(i)}}(w_i))$ with non-negligible probability (indeed $1/\text{poly} \leq \Pr[\cup_i (\neg A_i \vee \neg B_i)] \leq \sum_i \Pr[\neg A_i \vee \neg B_i]$). Let i^* be the smallest such index. It follows from its minimality that there exists a negligible function ε and a non-negligible function δ such that $\Pr[\cap_{i=1}^{i^*-1} (A_i \cap B_i)] \geq 1 - \varepsilon$ and $\Pr[(\neg A_{i^*} \vee \neg B_{i^*})] \geq \delta$. Let $i_1, \dots, i_\ell$ be the indices of the wires which are inputs to gate i^* . With all but negligible probability, $\forall j \in [\ell]$, $\tilde{w}_{i_j}^{(1)} - \tilde{w}_{i_j}^{(0)} = w_{i_j}^{(1)} - w_{i_j}^{(0)} = \text{sk} \cdot w_{i_j}$ and $y_{i_j} = \text{Mask}_{F_{r_1 \dots r_\lambda}(i_j)}(w_{i_j})$.

Consider the following variant of the PRF security game for PRF: (1) the challenger samples k_{PRF} , (2) the adversary gets the PRF evaluation on inputs $[i^* - 1] \setminus \{i_1, \dots, i_\ell\}$, (3) the challenger samples $b \xleftarrow{\$} \{0, 1\}$, and sends to the adversary $(\text{PRF}(k_{\text{PRF}}, i_j))_{j \in [\ell]}$ if $b = 0$, and random values instead if $b = 1$, (4) the adversary outputs b' and wins if $b = b'$. Clearly, any *p.p.t.* adversary must have at most negligible advantage in this game, by PRF security.

Now, consider the following adversarial strategy: (1) run the garbler's initialisation for the Garbled circuit (which includes sampling sk), with the exception of sampling k_{PRF} , (2) garble the circuit up to gate i^* , using the PRF queries on inputs $[i^* - 1] \setminus \{i_1, \dots, i_\ell\}$ for all gates except the inputs to the i^* -th, (3) receive the challenge $(Y_1, \dots, Y_\ell)$, (4) output the boolean value of $(y_i = \text{Mask}_{F_{\vec{r}(i)}}(w_i)) \wedge (\tilde{w}_{i^*}^{(1)} - \tilde{w}_{i^*}^{(0)} = \text{sk} \cdot w_{i^*})$. Observe that if $b = 1$ (*i.e.* the challenges $R_1, \dots, R_\ell$ are uniformly random) then this adversary must output 1 with all but negligible probability by correctness of AuthEval . Least it constitute an efficient distinguisher, it must therefore also output 1 with all but negligible probability conditioned on $b = 0$. This means that $\Pr[\neg A_i \vee \neg B_i]$ is negligible, which is a contradiction.

- **Privacy.** We must show that the evaluator's view can be simulated using only the circuit

and its output. We present a hybrid argument starting from the real garbled circuit and wire labels. The final hybrid will be independent of the inputs x_i , depending only on the circuit C and its outputs z_j , so we can make it be the simulator.

- $\mathcal{H}_1$. This is the real world. The evaluator sees the garbled circuit: $k_{\text{PRF}}, \text{ek}_1, r_j^{(1)}$ for $j \in [\lambda]$, $\bar{y}_i^{(0)} \leftarrow (y_i^{(0)} \bmod |\mathcal{Y}|)$, and $R_j := F_{\vec{r}}(j)$ for each output gate index j . The evaluator also see its wire labels: $(y_i, w_i^{(1)}) \xleftarrow{\$} \text{HSS.Share}_{1, \text{s-priv}}(1^\lambda, \text{sk}, w_i^{(0)}, \text{Mask}_{F_{\vec{r}}(i)}(x_i))$.
- $\mathcal{H}_2$. By strong correctness, with all but negligible probability every masked wire y_i computed by the evaluator will be correct. I.e., $y_i = \text{Mask}_{F_{\vec{r}}(i)}(w_i)$ for all i , where w_i is the plaintext value on the wire. Now, y_i is calculated from shares $y_i^{(1)}$ and $\bar{y}_i^{(0)}$, so we rewrite the garbled gates $\bar{y}_i^{(0)}$ in terms of y_i and $y_i^{(1)}$:
- $$\begin{aligned} y_i &\leftarrow (y_i^{(1)} - \bar{y}_i^{(0)}) \bmod |\mathcal{Y}| \\ &\Downarrow \\ \bar{y}_i^{(0)} &\leftarrow (y_i^{(1)} - y_i) \bmod |\mathcal{Y}| \end{aligned}$$
- $\mathcal{H}_3$. Notice that the garbler's HSS evaluation key ek_0 is no longer used, since we replaced the garbler's computation of $\bar{y}_i^{(0)}$. Also, the HSS secret key sk is only used to run $\text{HSS.Share}_{\text{priv}}$ and $\text{HSS.Share}_{\text{s-priv}}$. Therefore, the semi-private security of HSS against party 1 implies that we can simulate ek_1 , and the shares $r_j^{(1)}$ and $(y_i, w_i^{(1)})$ from just the semi-private inputs $y_i = \text{Mask}_{F_{\vec{r}}(i)}(x_i)$.
- $\mathcal{H}_4$. Since we are now using the HSS simulator, the key $\vec{r} = (r_1, \dots, r_\lambda)$ of F is only used to evaluate F . Therefore, we can replace $F_{\vec{r}}$ by a uniformly random function R from $[S] \rightarrow \mathcal{K}$. This is indistinguishable by the PRF security of F .
- $\mathcal{H}_5$. The evaluator's view is now being generated using only the masked values $y_i = \text{Mask}_{R(i)}(w_i)$ of every wire in the circuit. Note that for non-output wires, each $R(i)$ is used exactly once.⁴ For non-output wires i , use the one-time privacy (definition 24) of the masking scheme to replace $\text{Mask}_{R(i)}(w_i)$ by a simulation independent of w_i . The adversary's view now only depends on the values of the circuit outputs, so this hybrid is a valid simulator for garbled circuit privacy. $\square$

5.2.2 From semi-private HSS with linear offline/online shares.

Definition 29. *Additionally, a triplet of masking scheme, wide-gate class, and pseudorandom function F is authentication admissible for $\mathcal{C}$ if, for all $i_1, \dots, i_\ell, i' \in \text{Dom}(F_k)$,*

$$(k, s, y_1, \dots, y_\ell) \mapsto s \cdot \text{Mask}_{F_k(i')} (g(\text{UnMask}_{F_k(i_1)}(y_1), \dots, \text{UnMask}_{F_k(i_\ell)}(y_\ell)))$$

is in $\mathcal{C}$.

Remark 30. Similarly to Lemma 26, for our supported HSS circuit class $\text{RMS} \otimes \text{VP}$, authentication admissibility is equivalent to admissibility.

Arithmetic Garbling AGC from HSS (for wide-gate circuits)

Requires:

- Bound S on the circuit size.
- A semi-private HSS with modulus M linear, offline/online sharing of semi-private inputs.
- An admissible (and authentication admissible) triplet for the HSS's circuit class,

⁴ Recall that for output wires $R(j)$ is also sent to the evaluator to allow it unmask the output.

consisting of:

1. A class $\mathcal{G}$ of gates over $\mathcal{X}$.
2. A masking scheme $(\mathcal{K}, \mathcal{X}, \mathcal{Y}, \text{Mask}, \text{UnMask})$, where the space $\mathcal{Y}$ of masked values is an integer interval $[0, |\mathcal{Y}|)$.
3. A pseudorandom function $F: \{0, 1\}^\lambda \times [S] \rightarrow \mathcal{K}$.
- A pseudorandom function $\text{PRF}: \{0, 1\}^\lambda \times [S] \rightarrow \mathbb{Z}/M\mathbb{Z}$.

The Garbling Scheme (gate-by-gate description):

We assume that the wires of the circuit (whose depth is D) are ordered in some fixed topological order such that the n input wires are indexed $1, \dots, n$.

Initialisation (performed by the garbler).

1. $k_{\text{PRF}} \xleftarrow{\$} \{0, 1\}^\lambda$
2. For $d \in [0, D]$, $(\text{sk}_d, (\text{ek}_{0,d}, \text{ek}_{1,d})) \xleftarrow{\$} \text{HSS.Setup}(1^\lambda)$
3. $\vec{r} = (r_1, \dots, r_\lambda) \xleftarrow{\$} \{0, 1\}^\lambda$
4. For $d \in [0, D]$ and $j \in [\lambda]$, $(r_{j,d}^{(0)}, r_{j,d}^{(1)}) \xleftarrow{\$} \text{HSS.Share}_{\text{priv}}(1^\lambda, \text{sk}_d, r_j)$
5. For $d \in [1, D]$, $(s_d^{(0)}, s_d^{(1)}) \xleftarrow{\$} \text{HSS.Share}_{\text{priv}}(1^\lambda, \text{sk}_{d-1}, \text{sk}_d)$ // Key-switching gadget
6. For each $i \in [n]$, $w_i^{(0)} \xleftarrow{\$} \text{HSS.Share}_{0,\text{s-priv}}(1^\lambda, \text{sk}_0)$ // $\langle \text{sk}_0 \cdot \text{Mask}_{F_{\vec{r}}(i)}(w_i) \rangle_0$
7. Let $C_\times$ be the a circuit for the function $(s, y) \mapsto s \cdot y$.
8. For each $i \in [n]$, for $d = 1, \dots, D$:

$$w_{i,d}^{(0)} \xleftarrow{\$} (\text{HSS.Eval}(0, \text{ek}_{0,d-1}, s_d^{(0)}, w_{i,d-1}^{(0)}, \widehat{C}_{\text{id}}) + \text{PRF}(k_{\text{PRF}}, (i, d))) \bmod M$$

Garbling the i^{th} gate f at depth d , whose input wires are indexed $i_1, \dots, i_\ell$.

1. Let $C_i, \widehat{C}_i$ be the circuits (in the circuit class of HSS) for the functions

$$\begin{aligned} g_i(\vec{r}, y_1, \dots, y_\ell) &= \text{Mask}_{F_{\vec{r}}(i)}(f(\text{UnMask}_{F_{\vec{r}}(i_1)}(y_1), \dots, \text{UnMask}_{F_{\vec{r}}(i_\ell)}(y_\ell))) \\ \widehat{g}_i(\vec{r}, s, y_1, \dots, y_\ell) &= s \cdot g_i(\vec{r}, y_1, \dots, y_\ell). \end{aligned}$$

2. $y_i^{(0)} \xleftarrow{\$} \text{HSS.Eval}(0, \text{ek}_{0,d}, (r_{j,d}^{(0)})_{j \in [\lambda]}, (w_{i_j,d}^{(0)})_{j \in [\ell]}, C_i)$ // $\langle \text{Mask}_{F_{\vec{r}}(i)}(w_i) \rangle_0$
3. For $d' = d, \dots, D$:

$$\begin{aligned} w_{i,d'}^{(0)} &\xleftarrow{\$} \left(\text{HSS.Eval}(0, \text{ek}_{0,d'-1}, ((r_{j,d'-1}^{(0)})_{j \in [\lambda]}, s_{d'}^{(0)}), (w_{i_j,d'-1}^{(0)})_{j \in [\ell]}, \widehat{C}_i) \right. \\ &\quad \left. + \text{PRF}(k_{\text{PRF}}, (i, d')) \right) \bmod M \quad // \langle \text{sk}_{d'} \cdot \text{Mask}_{F_{\vec{r}}(i)}(w_i) \rangle_0 \end{aligned}$$

Garbler output. The garbler outputs the garbled circuit, comprised of $k_{\text{PRF}}, \text{ek}_{1,d}$ for $d \in [0, D]$, $r_j^{(1)}$ for $j \in [\lambda]$, $s_d^{(1)}$ for $d \in [1, D]$, $\bar{y}_i^{(0)} \leftarrow (y_i^{(0)} \bmod |\mathcal{Y}|)$ for each of the gates (indexed by i), and $R_j := F_{\vec{r}}(j)$ for each output gate index j .

Label generation. For each $i \in [n]$, given input $x_i \in [0, |\mathcal{Y}|)$, compute $y_i \leftarrow \text{Mask}_{F_{\vec{r}}(i)}(x_i)$ and $w_i^{(1)} \xleftarrow{\$} \text{HSS.Share}_{1,\text{s-priv}}(1^\lambda, \text{sk}_0, w_i^{(0)}, \text{Mask}_{F_{\vec{r}}(i)}(x_i))$, and output the label $(y_i, w_i^{(1)})$.

Evaluating the i^{th} input. For $d = 1, \dots, D$:

$$w_{i,d}^{(1)} \xleftarrow{\$} (\text{HSS.Eval}(1, \text{ek}_{1,d-1}, s_d^{(1)}, (y_i, w_{i,d-1}^{(1)}), \widehat{C}_{\text{id}}) + \text{PRF}(k_{\text{PRF}}, (i, d))) \bmod M$$

Evaluating the i^{th} gate f , whose input wires are indexed $i_1, \dots, i_\ell$.

1. Define C_i and $\widehat{C}_i$ as at garbling time.
 2. $y_i^{(1)} \xleftarrow{\$} \text{HSS.Eval}(1, \text{ek}_{1,d}, (r_{j,d}^{(1)})_{j \in [\lambda]}, (y_{ij}, w_{ij,d}^{(1)})_{j \in [\ell]}, C_i) // \langle \text{Mask}_{F_{\overline{r}}(i)}(w_i) \rangle_1$
 3. $y_i \leftarrow (y_i^{(1)} - \overline{y}_i^{(0)}) \bmod |\mathcal{Y}|$
 4. For $d' = d, \dots, D$:
$$w_{i,d'}^{(1)} \xleftarrow{\$} \left(\text{HSS.Eval}(1, \text{ek}_{1,d'-1}, ((r_{j,d'-1}^{(1)})_{j \in [\lambda]}, s_{d'}^{(1)}), (y_{ij}, w_{ij,d'-1}^{(1)})_{j \in [\ell]}, \widehat{C}_i) \right. \\ \left. + \text{PRF}(k_{\text{PRF}}, (i, d')) \right) \bmod M // \langle \text{sk}_{d'} \cdot \text{Mask}_{F_{\overline{r}}(i)}(w_i) \rangle_1$$
- Evaluator output.** For each output gate index j , output $\text{UnMask}_{R_j}(y_j)$.

Fig. 11: Template for (arithmetic or boolean) garbled circuits from semi-private HSS with linear offline/online sharing of semi-private inputs.

Theorem 31 (Template for (arithmetic or boolean) garbled circuits from semi-private HSS). *Let $\mathcal{G}$ be a set of wide-gates over $\mathcal{X}$, $(\text{Mask}, \text{UnMask})$ be a masking scheme with plaintext space $\mathcal{X}$ and masked value space $\mathcal{Y} = [0, |\mathcal{Y}|)$, and F be a PRF. If this triplet is admissible (and multiplication admissible) for some circuit class for which we have a semi-private HSS scheme with linear, offline/online sharing of semi-private inputs, then the construction of fig. 11 is an arithmetic garbling scheme for $\mathcal{G}$ wide-gate circuits. For a size s , depth D wide-gate circuit with n inputs and m outputs, the size of the garbled circuit and labels is $s \log(|\mathcal{Y}|) + m \log(|\mathcal{K}|) + (n + D + 1) \text{poly}(\lambda)$.*

Proof.

- **Strong correctness.** Let us prove a stronger notion of correctness, namely that with all but negligible probability the following invariants are maintained for every wire w_i (whose depth is denoted d_i):

$$A_i: \quad "y_i = \text{Mask}_{F_{\overline{r}}(i)}(w_i)" \quad (\forall i \geq 1) \\ B_{i,d'}: \quad "w_{i,d'}^{(1)} - w_{i,d'}^{(0)} = \text{sk}_{d'} \cdot \text{Mask}_{F_{\overline{r}}(i)}(w_i)" \quad (\forall i \geq 1, \forall d' \in [d_i, D])$$

The proof is completely analogous to that of theorem 28. For all $i \in [n]$, $\Pr[A_i] = 1$ by definition of $y_i \leftarrow \text{Mask}_{F_{\overline{r}}(i)}(x_i)$, and for all $d' \in [d_i, D]$ $\Pr[B_{i,d'}] = 1$ by definition of $\text{Share}_{0,\text{s-priv}}$ and $\text{Share}_{1,\text{s-priv}}$. Assume towards a contradiction that there is a non-negligible probability that there exists a wire index $i \geq n$ such that $(\neg A_i) \vee \bigvee_{d'=d_i}^D (\neg B_{i,d'})$. By a union bound, because there are only polynomially many wires, there exists a wire index $i \geq n$ such that $(\neg A_i) \vee \bigvee_{d'=d_i}^D (\neg B_{i,d'})$ with non-negligible probability. Let i^* be the smallest such index. It follows from its minimality that there exists a negligible function ε and a non-negligible function δ such that $\Pr \left[\bigwedge_{i=1}^{i^*-1} \left(A_i \wedge \bigwedge_{d'=d_i}^D B_{i,d'} \right) \right] \leq \varepsilon$ and $\Pr \left[\neg A_{i^*} \vee \bigvee_{d'=d_{i^*}}^D \neg B_{i^*,d'} \right] \geq \delta$. By essentially the same reduction to PRF security as used in the proof of strong correctness of theorem 28, because $\forall j \in [\ell], \tilde{w}_{i,d_i}^{(1)} - \tilde{w}_{i,d_i}^{(0)} = \text{sk}_{d_i} \cdot \text{Mask}_{F_{\overline{r}}(i_j)}(w_{ij})$, it follows that $(y_{i^*}^{(1)}, \tilde{w}_{i^*,d_{i^*}}^{(1)}) - (y_{i^*}^{(0)}, \tilde{w}_{i^*,d_{i^*}}^{(0)}) = (1, \text{sk}_{d_{i^*}}) \cdot \text{Mask}_{F_{\overline{r}}(i^*)}(w_{i^*})$ (and in turn $A_{i^*} \wedge B_{i^*,d_{i^*}}$) with all but negligible probability. Similarly, by PRF security, for every $d' \in [d_{i^*}, D-1]$, if $\bigwedge_{j \in [\ell]} (A_j \wedge B_{j,d'})$ then with all but negligible probability $A_{i^*} \wedge B_{i^*,d'+1}$. By combining all of these polynomially many hybrids, $(\neg A_i) \vee \bigvee_{d'=d_i}^D (\neg B_{i,d'})$ is negligible, which constitutes a contradiction.

- **Security.** We must show that the evaluator’s view can be simulated using only the circuit and its output. We present a hybrid argument starting from the real garbled circuit and wire labels, with the goal of removing all dependence on the circuit inputs x_i from the view. The final hybrid will become the simulator.

$\mathcal{H}_1$. This is the real world. The evaluator sees the garbled circuit: k_{PRF} , $\text{ek}_{1,d}$ for $d \in [0, D]$, $r_j^{(1)}$ for $j \in [\lambda]$, $s_d^{(1)}$ for $d \in [1, D]$, $\bar{y}_i^{(0)} \leftarrow (y_i^{(0)} \bmod |\mathcal{Y}|)$, and $R_j := F_{\bar{r}}(j)$ for each output gate index j . The evaluator also see its wire labels: $(y_i, w_i^{(1)}) \xleftarrow{\$} \text{HSS.Share}_{1,\text{s-priv}}(1^\lambda, \text{sk}_0, w_i^{(0)}, \text{Mask}_{F_{\bar{r}}(i)}(x_i))$.

$\mathcal{H}_2$. By strong correctness, with all but negligible probability every masked wire y_i satisfies computed by the evaluator satisfies $y_i = \text{Mask}_{F_{\bar{r}}(i)}(w_i)$ for all i , where w_i is the plaintext value on the wire. Similarly to the proof of theorem 28, use strong correctness to rewrite each garbled gates $\bar{y}_i^{(0)}$ in terms of y_i and $y_i^{(1)}$:

$$\begin{aligned} y_i &\leftarrow (y_i^{(1)} - \bar{y}_i^{(0)}) \bmod |\mathcal{Y}| \\ &\Downarrow \\ \bar{y}_i^{(0)} &\leftarrow (y_i^{(1)} - y_i) \bmod |\mathcal{Y}| \end{aligned}$$

After this change, none of the garbler’s HSS evaluation keys $\text{ek}_{0,d}$ are used.

$\mathcal{H}_3$. Perform a sequence of hybrids $\mathcal{H}_3^0 \dots, \mathcal{H}_3^{D+1}$, with $\mathcal{H}_3^0 = \mathcal{H}_2$ being the previous hybrid. In the change from $\mathcal{H}_3^d$ to $\mathcal{H}_3^{d+1}$, replace $\text{ek}_{1,d}$ and all calls to $\text{HSS.Share}_{\text{priv}}(1^\lambda, \text{sk}_d, \dots)$ or $\text{HSS.Share}_{\text{s-priv}}(1^\lambda, \text{sk}_d, \dots)$ with their simulations from the semi-private security of HSS against party 1. These shares include $\text{HSS.Share}_{\text{priv}}(1^\lambda, \text{sk}_d, r_j)$ and $\text{HSS.Share}_{\text{priv}}(1^\lambda, \text{sk}_d, \text{sk}_{d+1})$ (for $d < D$). For $d = 0$, these shares also include $\text{HSS.Share}_{1,\text{s-priv}}(1^\lambda, \text{sk}_0, w_i^{(0)}, \text{Mask}_{F_{\bar{r}}(i)}(x_i))$, and because these shares are semi-private their simulation will still be based on $y_i = \text{Mask}_{F_{\bar{r}}(i)}(x_i)$.

We now argue that the change from $\mathcal{H}_3^d$ to $\mathcal{H}_3^{d+1}$ is indistinguishable. In $\mathcal{H}_3^d$, the key sk_d is only used to generate shares, since the share $s_d^{(1)}$ of sk_d under key sk_{d-1} was replaced with a simulation in the $\mathcal{H}_3^{d-1}$. Therefore, we can apply the semi-private security of HSS to sk_d , which takes us to $\mathcal{H}_3^{d+1}$.

$\mathcal{H}_4$. The last two changes are the same as in theorem 28: replace $F_{\bar{r}}$ by a random function, and then use one-time privacy of the masking scheme to replace non-output y_i with simulations. They work exactly as in theorem 28. After these changes, the distribution depends only on the output values of the garbled circuit, so it is a valid simulator. $\square$

5.3 Instantiating the Template and Applications to Sublinear-Size Garbling

The garbling schemes in section 5.2 are parameterized by a class of wide-gates and a masking scheme. In any instantiation, the wide-gate class and masking scheme must be admissible; that is, the function that unmask, evaluates, then remark must be in $\text{RMS} \otimes \text{VP}$.

Due to space constraints we only state the main theorems here, and defer the details to appendix B.

Boolean Garbling For the Boolean case, putting appendix B.1 together with theorems 18, 22, 28 and 31, we get rate 1 garbling for truth-table circuits.

Theorem 32. *Assuming either DCR or KDM-security of Damgård-Jurik, there exists a garbling scheme for boolean truth table wide-gate circuits. The size of the garbled circuit and labels is $s + m + (n + D + 1) \cdot \text{poly}(\lambda)$ for DCR, or $s + m + (n + 1) \cdot \text{poly}(\lambda)$ for KDM-security, if the truth table circuit has size s , depth D , n inputs, and m outputs.*

This result can also be applied to fan-in 2 layered boolean circuits. A fan-in 2 circuit of depth $\log \log(s)$ has at most $\log(s)$ inputs, and so can be represented in truth tables of size $\text{poly}(s)$. Deep, but layered boolean circuits can then be chopped into chunks of depth $\log \log(s)$, and each bit of output of each chunk can be represented as a truth table. Therefore, m -output layered boolean circuits with size s can equivalently be written as fan-in $\log \log(s)$ boolean truth table circuits of size $n + m + s/\log \log(s)$ [Cou19].

Theorem 33. *Assuming either DCR or KDM-security of Damgård-Jurik, there exists a sub-linear communication garbling scheme for layered boolean circuits. The size of the garbled circuit and labels is $O(s/\log \log(s) + m) + (n + D + 1) \cdot \text{poly}(\lambda)$ for DCR, or $O(s/\log \log(s) + m) + (n + 1) \cdot \text{poly}(\lambda)$ for KDM-security, if the boolean circuit has size s , depth D , n inputs, and m outputs.*

This result can be extended beyond *layered* circuits using the fact that an *arbitrary* m -output boolean circuit with size s can be written as a fan-in k ($\leq \log \log s$) boolean truth table circuit of size $n + m + s(\frac{2}{5} + \frac{3}{5k})$ [CCMN24].

Arithmetic Garbling. For the arithmetic case, putting together appendix B.2 with theorems 18, 22, 28 and 31, we get rate 1 arithmetic garbling for B -bounded arithmetic circuits with multivariate polynomial gates, if $\log(B) = \omega(B)$.

Theorem 34. *Assuming either DCR or KDM-security of Damgård-Jurik, there exists a garbling scheme for B -bounded arithmetic circuits with multivariate polynomial wide-gates of logarithmic degree and polynomial number of terms. The size of the garbled circuit and labels is $(s + m)(\lambda + \log B + 2) + (n + D + 1) \cdot \text{poly}(\lambda, \log B)$ for DCR, or $(s + m)(\lambda + \log B + 2) + (n + 1) \cdot \text{poly}(\lambda, \log B)$ for KDM-security, if the wide-gate circuit has size s , depth D , n inputs, and m outputs.*

And again, we can use a standard technique to convert to sublinear communication garbling of fan-in 2 layered arithmetic circuits.

Theorem 35. *Assuming either DCR or KDM-security of Damgård-Jurik, there exists a sub-linear communication garbling scheme for B -bounded layered arithmetic circuits. The size of the garbled circuit and labels is $O((s/\log \log(s) + m)(\lambda + \log B)) + (n + D + 1) \cdot \text{poly}(\lambda, \log B)$ for DCR, or $O((s/\log \log(s) + m)(\lambda + \log B)) + (n + 1) \cdot \text{poly}(\lambda, \log B)$ for KDM-security, if the arithmetic circuit has size s , depth D , n inputs, and m outputs.*

Proof. By [Cou19], any fan-in 2 layered arithmetic circuit can be divided into layers of depth $\log \log(s)$, giving an equivalent wide-gate circuit of size $O(s/\log \log(s) + n + m)$. Each gate of this circuit evaluates a multivariate polynomial of degree at most $2^{\log \log(s)} = \log(s)$ and at most $2^{\log \log(s)} = \log(s)$ variables. By stars-and-bars counting, this polynomial has at most $\binom{2^{\log(s)}}{\log(s)} \leq s^2$ monomials. Therefore, gates of this circuit can be evaluated by the garbling scheme of theorem 34. $\square$

Acknowledgments

This research was supported by the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme under grant agreement number 101124977 (DECrypSIS) and the Danish Independent Research Council under Grant-IDs DFF-3103-00077B (CryptoDigi) and DFF-0165-00107B (C3PO).

References

- AIK11. Benny Applebaum, Yuval Ishai, and Eyal Kushilevitz. How to garble arithmetic circuits. In Rafail Ostrovsky, editor, *52nd FOCS*, pages 120–129. IEEE Computer Society Press, October 2011.

- Bar89. David A. Barrington. Bounded-width polynomial-size branching programs recognize exactly those languages in nc1 . *Journal of Computer and System Sciences*, 38(1):150–164, 1989.
- BCM23. Elette Boyle, Geoffroy Couteau, and Pierre Meyer. Sublinear-communication secure multiparty computation does not require FHE. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 159–189. Springer, Cham, April 2023.
- BGG⁺14. Dan Boneh, Craig Gentry, Sergey Gorbunov, Shai Halevi, Valeria Nikolaenko, Gil Segev, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit ABE and compact garbled circuits. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 533–556. Springer, Berlin, Heidelberg, May 2014.
- BGI16. Elette Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under DDH. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part I*, volume 9814 of *LNCS*, pages 509–539. Springer, Berlin, Heidelberg, August 2016.
- BLLL23. Marshall Ball, Hanjun Li, Huijia Lin, and Tianren Liu. New ways to garble arithmetic circuits. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 3–34. Springer, Cham, April 2023.
- BMO⁺25. Amik Raj Behera, Pierre Meyer, Claudio Orlandi, Lawrence Roy, and Peter Scholl. Privately constrained PRFs from DCR: Puncturing and bounded waring rank. 2025. Cryptology ePrint Archive.
- CCMN24. Pierre Charbit, Geoffroy Couteau, Pierre Meyer, and Reza Naserasr. A note on low-communication secure multiparty computation via circuit depth-reduction. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part IV*, volume 15367 of *LNCS*, pages 167–199. Springer, Cham, December 2024.
- Cle90. Richard Cleve. Towards optimal simulations of formulas by bounded-width programs. In *22nd ACM STOC*, pages 271–277. ACM Press, May 1990.
- CMPR23. Geoffroy Couteau, Pierre Meyer, Alain Passelègue, and Mahshid Riahinia. Constrained pseudo-random functions from homomorphic secret sharing. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part III*, volume 14006 of *LNCS*, pages 194–224. Springer, Cham, April 2023.
- Cou19. Geoffroy Couteau. A note on the communication complexity of multiparty computation in the correlated randomness model. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part II*, volume 11477 of *LNCS*, pages 473–503. Springer, Cham, May 2019.
- DIO21. Samuel Dittmer, Yuval Ishai, and Rafail Ostrovsky. Line-point zero knowledge and its applications. In *2nd Conference on Information-Theoretic Cryptography (ITC 2021)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2021.
- DJ01. Ivan Damgård and Mats Jurik. A generalisation, a simplification and some applications of Paillier’s probabilistic public-key system. In Kwangjo Kim, editor, *PKC 2001*, volume 1992 of *LNCS*, pages 119–136. Springer, Berlin, Heidelberg, February 2001.
- FNO15. Tore Kasper Frederiksen, Jesper Buus Nielsen, and Claudio Orlandi. Privacy-free garbled circuits with applications to efficient zero-knowledge. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 191–219. Springer, Berlin, Heidelberg, April 2015.
- GKP⁺13. Shafi Goldwasser, Yael Tauman Kalai, Raluca A. Popa, Vinod Vaikuntanathan, and Nickolai Zeldovich. Reusable garbled circuits and succinct functional encryption. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th ACM STOC*, pages 555–564. ACM Press, June 2013.
- HK20. David Heath and Vladimir Kolesnikov. Stacked garbling - garbled circuit proportional to longest execution path. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 763–792. Springer, Cham, August 2020.
- HK21a. David Heath and Vladimir Kolesnikov. One hot garbling. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 574–593. ACM Press, November 2021.
- HK21b. David Heath and Vladimir Kolesnikov. LogStack: Stacked garbling with $O(b \log b)$ computation. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part III*, volume 12698 of *LNCS*, pages 3–32. Springer, Cham, October 2021.
- HKN24. David Heath, Vladimir Kolesnikov, and Lucien K. L. Ng. Garbled circuit lookup tables with logarithmic number of ciphertexts. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part V*, volume 14655 of *LNCS*, pages 185–215. Springer, Cham, May 2024.
- HLL23. Yao-Ching Hsieh, Huijia Lin, and Ji Luo. Attribute-based encryption for circuits of unbounded depth from lattices. In *64th FOCS*, pages 415–434. IEEE Computer Society Press, November 2023.
- HO12. Brett Hemenway and Rafail Ostrovsky. Extended-DDH and lossy trapdoor functions. In Marc Fischlin, Johannes Buchmann, and Mark Manulis, editors, *PKC 2012*, volume 7293 of *LNCS*, pages 627–643. Springer, Berlin, Heidelberg, May 2012.

- ILL24. Yuval Ishai, Hanjun Li, and Huijia Lin. Succinct partial garbling from groups and applications. Cryptology ePrint Archive, Paper 2024/2073, 2024.
- ILL25. Yuval Ishai, Hanjun Li, and Huijia Lin. A unified framework for succinct garbling from homomorphic secret sharing. Cryptology ePrint Archive, Paper 2025/442, 2025.
- IW14. Yuval Ishai and Hoeteck Wee. Partial garbling schemes and their applications. In Javier Esparza, Pierre Fraigniaud, Thore Husfeldt, and Elias Koutsoupias, editors, *ICALP 2014, Part I*, volume 8572 of *LNCS*, pages 650–662. Springer, Berlin, Heidelberg, July 2014.
- LWYY24. Hanlin Liu, Xiao Wang, Kang Yang, and Yu Yu. Garbled circuits with 1 bit per gate. Cryptology ePrint Archive, Paper 2024/1988, 2024.
- MORS24. Pierre Meyer, Claudio Orlandi, Lawrence Roy, and Peter Scholl. Rate-1 arithmetic garbling from homomorphic secret sharing. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part IV*, volume 15367 of *LNCS*, pages 71–97. Springer, Cham, December 2024.
- OSY21. Claudio Orlandi, Peter Scholl, and Sophia Yakoubov. The rise of paillier: Homomorphic secret sharing and public-key silent OT. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 678–708. Springer, Cham, October 2021.
- Pai99. Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *EUROCRYPT’99*, volume 1592 of *LNCS*, pages 223–238. Springer, Berlin, Heidelberg, May 1999.
- QWW18. Willy Quach, Hoeteck Wee, and Daniel Wichs. Laconic function evaluation and applications. In Mikkel Thorup, editor, *59th FOCS*, pages 859–870. IEEE Computer Society Press, October 2018.
- RR21. Mike Rosulek and Lawrence Roy. Three halves make a whole? Beating the half-gates lower bound for garbled circuits. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 94–124, Virtual Event, August 2021. Springer, Cham.
- RS21. Lawrence Roy and Jaspal Singh. Large message homomorphic secret sharing from DCR and applications. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part III*, volume 12827 of *LNCS*, pages 687–717, Virtual Event, August 2021. Springer, Cham.
- Yao82. Andrew Chi-Chih Yao. Protocols for secure computations (extended abstract). In *23rd FOCS*, pages 160–164. IEEE Computer Society Press, November 1982.
- YSWW21. Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2986–3001. ACM Press, November 2021.

—Supplementary material—

A Offline-Online HSS Proofs

A.1 From KDM-security of Damgård-Jurik

In this section, we prove theorem 18, to show that fig. 6 is a Semi-Private HSS scheme (definition 16) with linear offline/online sharing of semi-private inputs and supporting authenticated evaluation (definition 17). We prove each property of the semi-private HSS scheme separately.

A.1.1 Correctness and authenticated correctness. Let $\lambda \in \mathbb{N}$ be a security parameter, $B \leq 2^{(\zeta-1-\varepsilon) \cdot \lambda_{\text{DCR}}/2}$ a bound, let C be an $n_{\text{s-priv}}$ -input circuit over $\mathbb{Z}$, let C_{rm} be a restricted-multiplication n_{priv} -input circuit over $\mathbb{Z}$, let $(x_{\text{s-priv},i})_{i \in [n_{\text{s-priv}}]}$ and $(x_{\text{priv},i})_{i \in [n_{\text{priv}}]}$ be B -admissible inputs for C and C_{rm} respectively.

Let us prove that correctness (definition 16) holds with probability at least $p \cdot (1 - N^{-\varepsilon}) - \varepsilon_{\text{PRF}}$ and that authenticated correctness (definition 17) holds with probability at least $p - \varepsilon_{\text{PRF}}$, where

$$p := 1 - (n_{\text{s-priv}} + 2n_{\text{priv}} + |C_{\times}| + |C'_{\times}|) \cdot N^{-\varepsilon}$$

(with $|C_{\times}|$ and $|C'_{\times}|$ denoting the number of multiplication gates in C and C_{rm} respectively) and ε_{PRF} is tied to the security of PRF.

The proof boils down to showing that all of the comments in the pseudocode of fig. 6 are invariants holding with all but negligible probability.

Idealised PRF. For simplicity, we first consider an idealised version of the HSS scheme of fig. 6 where PRF is replaced with a truly random function (*i.e.* the evaluation keys contain the same polynomial-size random string—because PRF has a polynomial-sized domain—instead of the short PRF key). We will later remove this restriction by a reduction to the security of PRF.

Consider the following random variables: (where $\text{sh}_{|C|+|C_{\text{rm}}|,0}$ and $\text{sh}_{|C|+|C_{\text{rm}}|,1}$ are random variables implicitly defined by Eval)

- | | |
|---|---|
| <ol style="list-style-type: none"> 1. $(\text{sk}, (\text{ek}_0, \text{ek}_1)) \xleftarrow{\\$} \text{Setup}(1^\lambda);$ 2. $(x_{\text{priv},i}^{(0)}, x_{\text{priv},i}^{(1)}) \xleftarrow{\\$} \text{Share}_{\text{priv}}(1^\lambda, \text{sk}, x_{\text{priv},i})$ 3. $y_{\text{s-priv},i}^{(0)} \xleftarrow{\\$} \text{Share}_{0,\text{s-priv}}(1^\lambda, \text{sk})$ 4. $y_{\text{s-priv},i}^{(1)} \xleftarrow{\\$} \text{Share}_{1,\text{s-priv}}(1^\lambda, \text{sk}, y_{\text{s-priv},i}, \text{st})$ 5. $z_0 \xleftarrow{\\$} \text{Eval}(0, \text{ek}_0, (x_{\text{priv},i}^{(0)})_{i=1}^{n_{\text{priv}}}, (y_{\text{s-priv},i}^{(0)})_{i=1}^{n_{\text{s-priv}}}, \mathcal{A})$ | <ol style="list-style-type: none"> 6. $z_1 \xleftarrow{\\$} \text{Eval}(1, \text{ek}_1, (x_{\text{priv},i}^{(1)})_{i=1}^{n_{\text{priv}}}, (y_{\text{s-priv},i}^{(1)})_{i=1}^{n_{\text{s-priv}}}, (C, C_{\text{rm}}))$ 7. $\tilde{z}_0 \leftarrow (z_0, \text{sh}_{ C + C_{\text{rm}} ,0})$ 8. $\tilde{z}_1 \leftarrow (z_1, \text{sh}_{ C + C_{\text{rm}} ,1})$ 9. $z \xleftarrow{\\$} z_1 - z_0$ 10. $\tilde{z} \xleftarrow{\\$} \tilde{z}_1 - \tilde{z}_0$ |
|---|---|

Observe that if we show that $\Pr[z = C(\vec{y}_{\text{s-priv}}) \cdot C_{\text{rm}}(\vec{x}_{\text{priv}})] \geq 1 - \text{negl}(\lambda)$ and $\Pr[\tilde{z} = (1, \varphi) \cdot C(\vec{y}_{\text{s-priv}}) \cdot C_{\text{rm}}(\vec{x}_{\text{priv}})] \geq 1 - \text{negl}(\lambda)$ then we will have proven both correctness and authenticated correctness.

Consider the following events:

$$\begin{aligned}
E_{\text{corr}} : & \quad \text{“Dec}_\varphi(c_{\text{inv}}) \equiv \varphi^{-1} \bmod N^\zeta\text{”} \\
E_{\text{corr},j} : & \quad \text{“Dec}_\varphi(c_j) \equiv x_{\text{priv},j} \bmod N^\zeta\text{”} \quad (\text{for } j \in [n_{\text{priv}}]) \\
E_0 : & \quad \text{“sh}_{0,1} - \text{sh}_{0,0} = \varphi \cdot 1\text{”} \\
E_i : & \quad \text{“sh}_{i,1} - \text{sh}_{i,0} = \varphi \cdot y_{\text{s-priv},i}\text{”} \quad (\text{for } i \in [n_{\text{s-priv}}]) \\
E_i : & \quad \text{“sh}_{i,1} - \text{sh}_{i,0} = \varphi \cdot P_i(\vec{y}_{\text{s-priv}})\text{”} \quad (\text{for } i \in [n_{\text{s-priv}} + 1, |C|]) \\
E'_i : & \quad \text{“sh}_{|C|+i,1} - \text{sh}_{|C|+i,0} = \varphi \cdot C(\vec{y}_{\text{s-priv}}) \cdot Q_i(\vec{x}_{\text{priv}})\text{”} \quad (\text{for } i \in [|C_{\text{rm}}|])
\end{aligned}$$

where for $i \in [n_{\text{s-priv}} + 1, |C|]$, P_i is the polynomial computed by the i^{th} gate of C , while for $i \in [|C_{\text{rm}}|]$, Q_i is the polynomial computed by the i^{th} gate of C_{rm} .

Lemma 36. $\Pr[E_0] = 1$.

Lemma 36 follows immediately from the definition of $\text{sh}_{0,1}$ and $\text{sh}_{0,0}$.

Lemma 37. $\forall i \in [n_{\text{s-priv}}], \Pr[E_i] \geq 1 - N^{-\varepsilon}$.

Proof of lemma Lemma 37. By definition of the $\text{sh}_{i,1}$ and $\text{sh}_{i,0}$ ($i \in [n_{\text{s-priv}}]$), the following relation holds with probability 1:

$$\forall i \in [n_{\text{s-priv}}], \text{sh}_{i,1} - \text{sh}_{i,0} \equiv \varphi \cdot x_{\text{s-priv},i} \bmod N^\zeta$$

Because the offline and the online inputs are all B -admissible, and because $\varphi \cdot B \leq N \cdot B = N^{\zeta-\varepsilon} \ll N^\zeta$, we get the desired results by invoking Lemma 9 (in particular because the $\text{sh}_{i,0}$ for $i \in [n_{\text{s-priv}}]$ are sampled uniformly at random from $\mathbb{Z}/N^\zeta\mathbb{Z}$). $\square$

Lemma 38. Let $i \in [n_{\text{s-priv}} + 1, |C|]$. If g_i (the i^{th} gate of C) is the constant gate, then $\Pr[E_i] = 1$.

Proof of Lemma 38. If g_i is the constant gate 1 then by definition $E_i \equiv E_0$ so $\Pr[E_i] = 1$ by Lemma 36. $\square$

Lemma 39. Let $i \in [n_{\text{s-priv}} + 1, |C|]$. If g_i (the i^{th} gate of C) is the j^{th} input gate, then $\Pr[E_i|E_j] = 1$.

Proof of Lemma 39. If g_i is the j^{th} input then by definition $E_i \equiv E_j$. By Lemma 37, $\Pr[E_j] \neq 0$ so $\Pr[E_i|E_j] = 1$ (because any non-impossible event is certain when conditioned on itself). $\square$

Lemma 40. Let $i \in [n_{\text{s-priv}} + 1, |C|]$. If g_i (the i^{th} gate of C) is an addition whose parents are g_{i_L} and g_{i_R} , then $\Pr[E_i|E_{i_L}, E_{i_R}] = 1$.

Proof of Lemma 40. If g_i is an addition gate, whose parents we denote i_R and i_L , then,

conditioned on E_{i_L} and E_{i_R} :

$$\begin{aligned}
\text{sh}_{i,1} - \text{sh}_{i,0} &= (\text{sh}_{i_R,1} + \text{sh}_{i_L,1}) - (\text{sh}_{i_R,0} + \text{sh}_{i_L,0}) \\
&= (\text{sh}_{i_L,1} - \text{sh}_{i_L,0}) + (\text{sh}_{i_R,1} - \text{sh}_{i_R,0}) \\
&= \varphi \cdot P_{i_L}(\vec{y}_{\text{s-priv}}) + \varphi \cdot P_{i_R}(\vec{y}_{\text{s-priv}}) \\
&= \varphi \cdot P_i(\vec{y}_{\text{s-priv}})
\end{aligned}$$

Therefore $\Pr[E_i|E_{i_L}, E_{i_R}] = 1$. $\square$

Lemma 41. *Let $i \in [n_{\text{s-priv}} + 1, |C|]$. If g_i (the i^{th} gate of C) is a scalar multiplication gate whose parent is g_j , then $\Pr[E_i|E_j] = 1$.*

Proof of Lemma 41. If g_i is the scalar multiplication $\cdot_\alpha$ with predecessor g_j , then, conditioned on E_j :

$$\begin{aligned}
\text{sh}_{i,1} - \text{sh}_{i,0} &= \alpha \cdot \text{sh}_{j,1} - \alpha \cdot \text{sh}_{j,0} \\
&= \alpha \cdot (\text{sh}_{j,1} - \text{sh}_{j,0}) \\
&= \alpha \cdot \varphi \cdot P_j(\vec{y}_{\text{s-priv}}) \\
&= \varphi \cdot P_i(\vec{y}_{\text{s-priv}})
\end{aligned}$$

Therefore $\Pr[E_i|E_j] = 1$. $\square$

Lemma 42. *Let $i \in [n_{\text{s-priv}} + 1, |C|]$. If g_i (the i^{th} gate of C) is a multiplication whose parents are g_{i_L} and g_{i_R} , then $\Pr[E_i|E_{i_L}, E_{i_R}, E_{\text{corr}}] \geq 1 - N^{-\varepsilon}$.*

Proof of Lemma 42. Let $i \in [n_{\text{s-priv}} + 1, |C|]$ such that g_i is a multiplication gate whose two operands are g_{i_L} and g_{i_R} (with $i_L, i_R < i$). Denote P_{i_L} and P_{i_R} the polynomials computed by g_{i_L} and g_{i_R} respectively. By core lemma 1, conditioned on E_{i_L} , E_{i_R} , and E_{corr} :

$$\begin{aligned}
&\left(-\text{DDLog} \left(c_{\text{inv}}^{\text{sh}_{i_L,1} \cdot \text{sh}_{i_R,1}} \right) + P_{i_L}(\vec{y}_{\text{s-priv}}) \cdot \text{sh}_{i_R,1} + P_{i_R}(\vec{x}_{\text{priv}}) \cdot \text{sh}_{i_L,1} \right) \\
&\quad - \left(-\text{DDLog} \left(c_{\text{inv}}^{\text{sh}_{i_L,0} \cdot \text{sh}_{i_R,0}} \right) \right) \equiv \varphi \cdot P_i(\vec{y}_{\text{s-priv}}) \pmod{N^\zeta}
\end{aligned}$$

We conclude, by Lemma 9 (because $\varphi \cdot P_i(\vec{y}_{\text{s-priv}}) \leq N \cdot B \leq N^{\zeta-\varepsilon}$), that $\Pr[E_i|E_{i_L}, E_{i_R}, E_{\text{corr}}] \geq 1 - N^{-\varepsilon}$. $\square$

Lemma 43. *Let $i \in [|C_{\text{rm}}|]$. If g'_i (the i^{th} gate of C_{rm}) is a constant gate then $\Pr[E'_i|E_{|C|}] = 1$.*

Proof of Lemma 43. If g'_i is the constant gate 1, then by definition $E'_i \equiv E_{|C|}$ so $\Pr[E'_i|E_{|C|}] = 1$. $\square$

Lemma 44. *Let $i \in [|C_{\text{rm}}|]$. If g'_i (the i^{th} gate of C_{rm}) is an addition whose parents are g'_{i_L} and g'_{i_R} , then $\Pr[E'_i|E'_{i_L}, E'_{i_R}] = 1$.*

Proof of Lemma 44. If g'_i is an addition gate, whose parents are indexed i_R and i_L ,

then, conditioned on E'_{i_L} and E'_{i_R} :

$$\begin{aligned}
\text{sh}_{|C|+i,1} - \text{sh}_{|C|+i,0} &= (\text{sh}_{|C|+i_R,1} + \text{sh}_{|C|+i_L,1}) - (\text{sh}_{|C|+i_R,0} + \text{sh}_{|C|+i_L,0}) \\
&= (\text{sh}_{|C|+i_L,1} - \text{sh}_{|C|+i_L,0}) + (\text{sh}_{|C|+i_R,1} - \text{sh}_{|C|+i_R,0}) \\
&= \varphi \cdot C(\vec{y}_{\text{s-priv}}) \cdot Q_{|C|+i_L}(\vec{x}_{\text{priv}}) + \varphi \cdot C(\vec{y}_{\text{s-priv}}) \cdot Q_{|C|+i_R}(\vec{x}_{\text{priv}}) \\
&= \varphi \cdot C(\vec{y}_{\text{s-priv}}) \cdot Q_i(\vec{x}_{\text{priv}})
\end{aligned}$$

Therefore $\Pr[E'_i | E'_{i_L}, E'_{i_R}] = 1$. $\square$

Lemma 45. *Let $i \in [|C_{\text{rm}}|]$. If g'_i (the i^{th} gate of C_{rm}) is a scalar multiplication gate whose parent is g'_j , then $\Pr[E'_i | E'_j] = 1$.*

Proof of Lemma 45. If g'_i is the scalar multiplication $\cdot_\alpha$ with predecessor g'_j , then, conditioned on E'_j :

$$\begin{aligned}
\text{sh}_{|C|+i,1} - \text{sh}_{|C|+i,0} &= \alpha \cdot \text{sh}_{|C|+j,1} - \alpha \cdot \text{sh}_{|C|+j,0} \\
&= \alpha \cdot (\text{sh}_{|C|+j,1} - \text{sh}_{|C|+j,0}) \\
&= \alpha \cdot \varphi \cdot C(\vec{y}_{\text{s-priv}}) \cdot Q_j(\vec{x}_{\text{priv}}) \\
&= \varphi \cdot C(\vec{y}_{\text{s-priv}}) \cdot Q_i(\vec{x}_{\text{priv}})
\end{aligned}$$

Therefore $\Pr[E'_i | E'_j] = 1$. $\square$

Lemma 46. *Let $i \in [|C_{\text{rm}}|]$. If g'_i (the i^{th} gate of C_{rm}) is the j^{th} input gate, then $\Pr[E'_i | E_{|C|}, E_{\text{priv},j}] \geq 1 - N^{-\varepsilon}$.*

Proof of Lemma 46. If g'_i is the j^{th} input gate $x_{\text{priv},j}$ and we condition on $E_{|C|}$, then by Lemma 7

$$\text{DDLog}((c_j)^{\text{sh}_{|C|,1}}) - \text{DDLog}((c_j)^{\text{sh}_{|C|,0}}) \equiv \text{DJ.Dec}_{N,\zeta,\varphi}(c_j) \cdot C(\vec{y}_{\text{s-priv}}) \cdot \varphi \pmod{N^\zeta}$$

If we further condition on $E_{\text{priv},j}$, then

$$\text{DDLog}((c_j)^{\text{sh}_{|C|,1}}) - \text{DDLog}((c_j)^{\text{sh}_{|C|,0}}) \equiv \varphi \cdot C(\vec{y}_{\text{s-priv}}) \cdot x_{\text{priv},j} \pmod{N^\zeta}$$

Finally, because $C(\vec{y}_{\text{s-priv}}) \cdot x_{\text{priv},j} \leq N \cdot B^2 \leq N^{\zeta-\varepsilon}$, (still conditioned on $E_{|C|}$ and $E_{\text{priv},j}$) E'_i holds with probability at least $1 - N^{-\varepsilon}$ by Lemma 9. $\square$

Lemma 47. *Let $i \in [|C_{\text{rm}}|]$. If g'_i (the i^{th} gate of C_{rm}) is a multiplication gate whose parents are the j^{th} input gate and g'_{i_R} , then $\Pr[E'_i | E'_{i_R}, E_{\text{corr},j}] \geq 1 - N^{-\varepsilon}$.*

Proof of Lemma 47. If g'_i is the multiplication of the j^{th} input gate $x_{\text{priv},j}$ and gate g'_{i_R} , then, conditioning on E'_{i_R} , by Lemma 7

$$\begin{aligned}
\text{DDLog}((c_j)^{\text{sh}_{|C|+i_R,1}}) - \text{DDLog}((c_j)^{\text{sh}_{|C|+i_R,0}}) \\
\equiv \text{DJ.Dec}_{N,\zeta,\varphi}(c_j) \cdot C(\vec{y}_{\text{s-priv}}) \cdot Q_{i_R}(\vec{x}_{\text{priv}}) \cdot \varphi \pmod{N^\zeta}
\end{aligned}$$

If we further condition on $E_{\text{corr},j}$, then

$$\text{DDLog}((c_j)^{\text{sh}_{|C|+i_R,1}}) - \text{DDLog}((c_j)^{\text{sh}_{|C|+i_R,0}}) \equiv \varphi \cdot C(\vec{y}_{\text{s-priv}}) \cdot Q_i(\vec{x}_{\text{priv}}) \pmod{N^\zeta}$$

Finally, because $C(\vec{y}_{\text{s-priv}}) \cdot Q_i(\vec{x}_{\text{priv}}) \leq N \cdot B^2 \leq N^{\zeta-\varepsilon}$, (still conditioned on $E_{|C|+i_R}$ and $E_{\text{priv},j}$) E'_i holds with probability at least $1 - N^{-\varepsilon}$ by Lemma 9. $\square$

Lemma 48. *Authenticated correctness with an idealised PRF (i.e. the event “ $\tilde{y} = \varphi \cdot C(\vec{y}_{\text{s-priv}}) \cdot C_{\text{rm}}(\vec{x}_{\text{priv}})$ ”) holds with probability at least $1 - (n_{\text{s-priv}} + n_{\text{priv}} + |C_{\times}| + |C'_{\times}|) \cdot N^{-\varepsilon}$, where $|C_{\times}|$ and $|C'_{\times}|$ are the number of multiplication gates in C and C_{rm} respectively.*

Proof of Lemma 48. To simplify notations, we rename/order the events $E_{\text{corr}}, (E_{\text{corr},j})_{j \in [n_{\text{priv}}]}, (E_i)_{i \in [0,|C|]}, (E'_i)_{i \in [|C_{\text{rm}}|]}$ as $A_1, \dots, A_{n_{\text{priv}}+|C|+|C_{\text{rm}}|+2}$:

1. $A_1 := E_{\text{corr}}$
2. For $j \in [n_{\text{priv}}]$, $A_{j+1} := E_{\text{corr},j}$
3. For $j \in [0, |C|]$, $A_{n_{\text{s-priv}}+2+j} := E_j$
4. For $j \in [|C_{\text{rm}}|]$, $A_{n_{\text{s-priv}}+2+|C|+j} := E'_j$

Observe that

$$\begin{aligned} \Pr[E'_{|C_{\text{rm}}|}] &= \Pr[A_{n_{\text{priv}}+|C|+|C_{\text{rm}}|+2}] \\ &\geq \Pr[\bigcap_{i=1}^{n_{\text{priv}}+|C|+|C_{\text{rm}}|+2} A_i] \\ &= 1 - \Pr[\bigcup_{i=1}^{n_{\text{priv}}+|C|+|C_{\text{rm}}|+2} \overline{A_i}] \\ &= 1 - \Pr[\bigcup_{i=1}^{n_{\text{priv}}+|C|+|C_{\text{rm}}|+2} (\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j)] \\ &= 1 - \sum_{i=1}^{n_{\text{priv}}+|C|+|C_{\text{rm}}|+2} \Pr[\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j] \end{aligned}$$

Let us now upper bound each of the $\Pr[\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j]$, which in turn allows us to lower bound $\Pr[E'_{|C_{\text{rm}}|}]$.

1. $\Pr[\overline{A_1}] = \Pr[\overline{E_{\text{corr}}}] = 0$ by perfect correct of the Damgård-Jurik encryption scheme.
2. For $i \in [2, n_{\text{priv}}+1]$, $\Pr[\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j] \leq \Pr[\overline{A_i}] = \Pr[\overline{E_{\text{corr},i-1}}] = 0$ by perfect correct of the Damgård-Jurik encryption scheme.
3. $\Pr[\overline{A_{n_{\text{priv}}+2}} \cap \bigcap_{1 \leq j < n_{\text{priv}}+2} A_j] \leq \Pr[\overline{A_{n_{\text{priv}}+2}}] = \Pr[\overline{E_0}] = 0$ by Lemma 36.
4. For $i \in [n_{\text{priv}}+3, n_{\text{priv}}+n_{\text{s-priv}}+2]$, $\Pr[\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j] \leq \Pr[\overline{A_i}] = \Pr[\overline{E_{j-n_{\text{priv}}-2}}] \leq N^{-\varepsilon}$ by Lemma 37.
5. For $i \in [n_{\text{priv}}+n_{\text{s-priv}}+3, n_{\text{priv}}+n_{\text{s-priv}}+|C|+2]$,
 - If $g_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}$ is the constant gate, then $\Pr[\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j] \leq \Pr[\overline{A_i}] = \Pr[\overline{E_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}}] = 0$ by Lemma 38.
 - If $g_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}$ is the j^{th} input gate, then $\Pr[\overline{A_i} \cap \bigcap_{1 \leq k < i} A_k] \leq \Pr[\overline{A_i} \cap A_{n_{\text{priv}}+j+2}] = \Pr[\overline{E_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}} \cap E_j] = \Pr[\overline{E_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}} | E_j] \cdot \Pr[E_{\text{s-priv},j}] = 0$ by Lemma 39.
 - If $g_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}$ is an addition gate whose parents are g_{i_L} and g_{i_R} , then $\Pr[\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j] \leq \Pr[\overline{A_i} \cap A_{n_{\text{s-priv}}+2+i_L} \cap A_{n_{\text{s-priv}}+2+i_R}] = \Pr[\overline{E_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}} \cap E_{i_L} \cap E_{i_R}] = \Pr[\overline{E_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}} | E_{i_L}, E_{i_R}] \cdot \Pr[E_{i_L} \cap E_{i_R}] \leq \Pr[\overline{E_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}} | E_{i_L}, E_{i_R}] = 0$ by Lemma 40.
 - If $g_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}$ is a scalar multiplication gate whose parent is g_j , then $\Pr[\overline{A_i} \cap \bigcap_{1 \leq k < i} A_k] \leq \Pr[\overline{A_i} \cap A_{n_{\text{s-priv}}+2+j}] = \Pr[\overline{E_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}} \cap E_j] = \Pr[\overline{E_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}} | E_j] \cdot \Pr[E_j] \leq \Pr[\overline{E_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}} | E_j] = 0$ by Lemma 41.
 - If $g_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}$ is a multiplication gate whose parents are g_{i_L} and g_{i_R} , then $\Pr[\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j] \leq \Pr[\overline{A_i} \cap A_{n_{\text{priv}}+2+i_L} \cap A_{n_{\text{priv}}+2+i_R} \cap A_1] = \Pr[\overline{E_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}} \cap E_{i_L} \cap E_{i_R} \cap E_{\text{corr}}] = \Pr[\overline{E_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}} | E_{i_L}, E_{i_R}, E_{\text{corr}}] \cdot \Pr[E_{i_L} \cap E_{i_R} \cap E_{\text{corr}}] \leq \Pr[\overline{E_{i-n_{\text{s-priv}}-n_{\text{priv}}-2}} | E_{i_L}, E_{i_R}, E_{\text{corr}}] \leq N^{-\varepsilon}$ by Lemma 42.
6. For $i \in [n_{\text{s-priv}}+n_{\text{priv}}+|C|+3, n_{\text{s-priv}}+n_{\text{priv}}+|C|+|C_{\text{rm}}|+2]$,
 - If $g'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}$ is the constant gate, then $\Pr[\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j] \leq \Pr[\overline{A_i}] = \Pr[\overline{E'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}}] = 0$ by Lemma 43.

- If $g'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}$ is an addition gate whose parents are g'_{i_L} and g'_{i_R} , then $\Pr[\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j] \leq \Pr[\overline{A_i} \cap A_{n_{\text{s-priv}}+n_{\text{priv}}+|C|+2+i_L} \cap A_{n_{\text{s-priv}}+n_{\text{priv}}+|C|+2+i_R}] = \Pr[\overline{E'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}} \cap E'_{i_L} \cap E'_{i_R}] = \Pr[\overline{E'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}} | E'_{i_L}, E'_{i_R}] \cdot \Pr[E'_{i_L} \cap E'_{i_R}] \leq \Pr[\overline{E'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}} | E'_{i_L}, E'_{i_R}] = 0$ by Lemma 44.
- If $g'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}$ is a scalar multiplication gate whose parent is g'_j , then $\Pr[\overline{A_i} \cap \bigcap_{1 \leq k < i} A_k] \leq \Pr[\overline{A_i} \cap A_j] = \Pr[\overline{E'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}} \cap E'_j] = \Pr[\overline{E'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}} | E'_j] \cdot \Pr[E'_j] \leq \Pr[\overline{E'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}} | E'_j] = 0$ by Lemma 45.
- If $g'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}$ is the j^{th} input gate, then $\Pr[\overline{A_i} \cap \bigcap_{1 \leq k < i} A_k] \leq \Pr[\overline{A_i} \cap A_{n_{\text{s-priv}}+n_{\text{priv}}+|C|+2, A_{j+1}, A_1}] = \Pr[\overline{E'_i} \cap E_{|C|} \cap E_{\text{corr},j} \cap E_{\text{corr}}] = \Pr[\overline{E'_i} | E_{|C|}, E_{\text{corr},j}, E_{\text{corr}}] \cdot \Pr[E_{|C|} \cap E_{\text{corr},j} \cap E_{\text{corr}}] \leq \Pr[\overline{E'_i} | E_{|C|}, E_{\text{corr},j}, E_{\text{corr}}] \leq N^{-\varepsilon}$ by Lemma 46.
- If $g'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}$ is a multiplication gate whose parents are the j^{th} input and g'_{i_R} , then $\Pr[\overline{A_i} \cap \bigcap_{1 \leq k < i} A_k] \leq \Pr[\overline{A_i} \cap A_{j+1} \cap A_{n_{\text{s-priv}}+n_{\text{priv}}+i_R+2} \cap A_1] = \Pr[\overline{E'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}} \cap E_{\text{corr},j} \cap E'_{i_R} \cap E_{\text{corr}}] = \Pr[\overline{E'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}} | E_{\text{corr},j}, E'_{i_R}, E_{\text{corr}}] \cdot \Pr[E_{\text{corr},j} \cap E'_{i_R} \cap E_{\text{corr}}] \leq \Pr[\overline{E'_{i-n_{\text{s-priv}}-n_{\text{priv}}-|C|-2}} | E_{\text{corr},j}, E'_{i_R}, E_{\text{corr}}] \leq N^{-\varepsilon}$ by Lemma 47.

By combining all of the above, $\Pr[E'_{|C_{\text{rm}}|}] \geq 1 - (n_{\text{s-priv}} + n_{\text{priv}} + |C_{\times}| + |C'_{\times}|) \cdot N^{-\varepsilon}$.^a Because $E'_{|C_{\text{rm}}|}$ coincides with authenticated correctness, this concludes the proof. $\square$

^a This bound is obtained by making the assumption that there are no duplicate input gates, i.e. that there are at most $n_{\text{s-priv}}$ input gates in C and at most n_{priv} in C_{rm} .

Lemma 49. *Correctness with an idealised PRF (i.e. the event “ $y = C(\vec{y}_{\text{s-priv}}) \cdot C_{\text{rm}}(\vec{x}_{\text{priv}})$ ”) holds with probability at least $(1 - N^{-\varepsilon}) \cdot (1 - (n_{\text{s-priv}} + n_{\text{priv}} + |C_{\times}| + |C'_{\times}|) \cdot N^{-\varepsilon})$, where $|C_{\times}|$ and $|C'_{\times}|$ are the number of multiplication gates in C and C_{rm} respectively.*

Proof of Lemma 49. By Lemma 9, $\Pr[“y = C(\vec{y}_{\text{s-priv}}) \cdot C_{\text{rm}}(\vec{x}_{\text{priv}})” | E'_{|C_{\text{rm}}|}, E_{\text{corr}}] \geq 1 - N^{-\varepsilon}$ (using the fact that $C(\vec{y}_{\text{s-priv}}) \cdot C_{\text{rm}}(\vec{x}_{\text{priv}}) \leq B^2 \leq N^{\zeta-\varepsilon}$).

$$\begin{aligned} \Pr[“y = C(\vec{y}_{\text{s-priv}}) \cdot C_{\text{rm}}(\vec{x}_{\text{priv}})”] &\geq \Pr[“y = C(\vec{y}_{\text{s-priv}}) \cdot C_{\text{rm}}(\vec{x}_{\text{priv}})” | E'_{|C_{\text{rm}}|}, E_{\text{corr}}] \\ &= \Pr[“y = C(\vec{y}_{\text{s-priv}}) \cdot C_{\text{rm}}(\vec{x}_{\text{priv}})” | E'_{|C_{\text{rm}}|}, E_{\text{corr}}] \cdot \Pr[E'_{|C_{\text{rm}}|}, E_{\text{corr}}] \end{aligned}$$

Because $\Pr[E_{\text{corr}}] = 0$ by perfect correctness of the Damgård-Jurik encryption scheme, and $\Pr[E'_{|C_{\text{rm}}|}] \geq 1 - (n_{\text{s-priv}} + n_{\text{priv}} + |C_{\times}| + |C'_{\times}|) \cdot N^{-\varepsilon}$ by Lemma 48, $\Pr[“y = C(\vec{y}_{\text{s-priv}}) \cdot C_{\text{rm}}(\vec{x}_{\text{priv}})”] \geq (1 - N^{-\varepsilon}) \cdot 1 - (n_{\text{s-priv}} + n_{\text{priv}} + |C_{\times}| + |C'_{\times}|) \cdot N^{-\varepsilon}$, which concludes the proof. $\square$

Real PRF. We now no longer assume that PRF is instantiated as a truly random function. Consider the adversary which, tasked with distinguishing outputs of PRF and a random oracle, runs the “HSS correctness experiment” and outputs 1 if and only if correctness holds. By security of PRF, the advantage of this adversary can be at most ε_{PRF} , and since we have already established that correctness (respectively authenticated correctness) in the ideal world holds with probability at least $p \cdot (1 - N^{-\varepsilon})$ (respectively p), where

$$p := \left[(1 - N^{-\varepsilon})^{|C_{\times}|} \cdot (1 - n_{\text{s-priv}} \cdot N^{-\varepsilon} - \varepsilon_{\text{corr}}) - n_{\text{priv}} \cdot N^{-\varepsilon} \right] \cdot (1 - N^{-\varepsilon})^{|C'_{\times}| + n_{\text{priv}}},$$

it follows that correctness (respectively authenticated correctness) in the real world holds with probability at least $p \cdot (1 - N^{-\varepsilon}) - \varepsilon_{\text{PRF}}$ (respectively $p - \varepsilon_{\text{PRF}}$).

A.1.2 Security against party $\sigma = 0$. We need to simulate the view of party $\sigma = 0$ using just the numbers $n_{\text{priv}}, n_{\text{s-priv}}$. We present a hybrid argument, in which the last world the view of party 0 will depend only on these values. This last hybrid world becomes the simulation.

- $\mathcal{H}_1$. This is the real world. The adversary sees $\text{ek}_0 \leftarrow (N, c_{\text{inv}}, k_{\text{PRF}}, \text{sh}_{1,0})$, $x_i^{(0)}$ (output by $\text{Share}_{\text{priv}}$) for $i \in [n_{\text{priv}}]$, and $y_i^{(0)}$ (output by $\text{Share}_{1,\text{s-priv}}$) for $i \in n_{\text{s-priv}}$. Notice that the only thing in the adversary's view that depends on private information that the simulator will not know are the Damgård-Jurik ciphertexts $x_i^{(0)}$.
- $\mathcal{H}_2$. Notice that the only part of the adversary's view that φ is used for is $c_{\text{inv}} \xleftarrow{\$} \text{DJ.Enc}_N(\varphi^{-1} \bmod N^\zeta)$. Replace this with $c_{\text{inv}} \xleftarrow{\$} \text{DJ.Enc}_N(0)$. This is indistinguishable assuming KDM security of Damgård-Jurik.
- $\mathcal{H}_3$. Now that the secret key φ is unused, we can simulate the encryptions $x_i^{(0)} \xleftarrow{\$} \text{DJ.Enc}_N(x)$ as encryptions of zero $x_i^{(0)} \xleftarrow{\$} \text{DJ.Enc}_N(0)$. This is indistinguishable by CPA security of Damgård-Jurik (which is implied by KDM security). The adversary's view is now being simulated using only the number of inputs, n_{priv} and $n_{\text{s-priv}}$.

A.1.3 Security against party $\sigma = 1$. We need to simulate the view of party $\sigma = 1$ using just the numbers $n_{\text{priv}}, n_{\text{s-priv}}$ of shares, and the values of the semi-private shares y_i . We present a hybrid argument, in which the last world the view of party 1 will depend only on these values. This last hybrid world becomes the simulation.

- $\mathcal{H}_1$. This is the real world. The adversary sees $\text{ek}_1 = (N, c_{\text{inv}}, k_{\text{PRF}}, \text{sh}_{1,1})$, $x_i^{(1)}$ (output by $\text{Share}_{\text{priv}}$) for $i \in [n_{\text{priv}}]$, and $y_i^{(1)} = (y_i, \text{sh}_{i,1})$ (output by $\text{Share}_{1,\text{s-priv}}$) for $i \in n_{\text{s-priv}}$.
- $\mathcal{H}_2$. Instead of sampling $\text{sh}_{i,0} \xleftarrow{\$} \mathbb{Z}/N^\zeta\mathbb{Z}$ and setting $\text{sh}_{i,1} \leftarrow (\varphi \cdot y_i + \text{sh}_{i,0}) \bmod N^\zeta$, sample $\text{sh}_{i,1} \xleftarrow{\$} \mathbb{Z}/N^\zeta\mathbb{Z}$. This is an identical distribution because $\text{sh}_{i,0}$ is unused.
- $\mathcal{H}_3$. Make a similar change to $\text{sh}_{1,1}$. Instead of sampling $\text{sh}_{1,0} \xleftarrow{\$} [0, N]$ and setting $\text{sh}_{1,1} \leftarrow \varphi + \text{sh}_{1,0}$, sample $\text{sh}_{1,1} \xleftarrow{\$} [N, 2N]$ (and $\text{sh}_{1,0}$ is unused). This shifts the distribution of $\text{sh}_{1,1}$ from uniform on $[\varphi, N + \varphi]$ to uniform on $[N, 2N]$. Therefore, this change has statistical distance of

$$\frac{N - \varphi}{N + 1} = \frac{N - (pq - p - q + 1)}{N + 1} = \frac{p + q - 1}{N + 1} = O(N^{-\frac{1}{2}}).$$

where p and q are the factors of the semi-prime N , each of size roughly $N^{\frac{1}{2}}$.

- $\mathcal{H}_4$. Notice that φ is now used in only one place: $c_{\text{inv}} \xleftarrow{\$} \text{DJ.Enc}_N(\varphi^{-1} \bmod N^\zeta)$. Instead sample $c_{\text{inv}} \xleftarrow{\$} \text{DJ.Enc}_N(0)$. This is indistinguishable assuming KDM security of Damgård-Jurik.
- $\mathcal{H}_5$. Replace the encryptions $x_i^{(1)} \xleftarrow{\$} \text{DJ.Enc}_N(x)$ with encryptions of zero $x_i^{(1)} \xleftarrow{\$} \text{DJ.Enc}_N(0)$. This is indistinguishable by CPA security of Damgård-Jurik (which is implied by KDM security). The adversary's view now only depends on the semi-private inputs y_i , so this hybrid is a valid simulator for the HSS security game.

A.2 From DCR

In this section, we prove theorem 22. That is, we show that fig. 9 is a Semi-Private HSS scheme (definition 16) with linear offline/online sharing of semi-private inputs. We prove each property of the Offline-Online HSS separately.

A.2.1 Correctness. Let $\lambda \in \mathbb{N}$ be a security parameter and $B \leq 2^{(\zeta-1-\varepsilon) \cdot \lambda_{\text{DCR}}/2}$ a bound, let C be an $n_{\text{s-priv}}$ -input circuit over $\mathbb{Z}$ of pebbling depth D , let C_{rm} be a restricted-multiplication n_{priv} -input circuit over $\mathbb{Z}$, let $(y_{\text{s-priv},i})_{i \in [n_{\text{s-priv}}]}$ and $(x_{\text{priv},i})_{i \in [n_{\text{priv}}]}$ be B -admissible inputs for C and C_{rm} respectively.

Let us prove that correctness (definition 16) holds with all but negligible probability. The proof boils down to showing that all of the comments in the pseudocode of fig. 9 (*e.g.* “ $\text{sh}_{y_{\text{s-priv},0}}^{\text{auth}}$ and $\text{sh}_{y_{\text{s-priv},1}}^{\text{auth}}$ are subtractive shares of $k_0 \cdot y_{\text{s-priv}}$ over $\mathbb{Z}/N^\zeta\mathbb{Z}$ ”) are invariants holding with all but negligible probability.

Idealised PRF. For simplicity, we first consider an idealised version of the HSS scheme of fig. 9 where PRF is replaced with a truly random function (*i.e.* the evaluation keys contain the same polynomial-size random string—because PRF has a polynomial-sized domain—instead of the short PRF key). We will later remove this restriction by a reduction to the security of PRF.

Consider the following random variables: (where $\text{sh}_{|C|+|C_{\text{rm}}|,0}^{\text{auth}}$ and $\text{sh}_{|C|+|C_{\text{rm}}|,1}^{\text{auth}}$ are random variables implicitly defined by Eval)

1. $(\text{sk}, (\text{ek}_0, \text{ek}_1)) \xleftarrow{\$} \text{Setup}(1^\lambda);$
2. $(x_{\text{priv},i}^{(0)}, x_{\text{priv},i}^{(1)}) \xleftarrow{\$} \text{Share}_{\text{priv}}(1^\lambda, \text{sk}, x_{\text{priv},i});$
3. $x_{\text{s-priv},i}^{(0)} \xleftarrow{\$} \text{Share}_{0,\text{s-priv}}(1^\lambda, \text{sk});$
4. $x_{\text{s-priv},i}^{(1)} \xleftarrow{\$} \text{Share}_{1,\text{s-priv}}(1^\lambda, \text{sk}, y_{\text{s-priv},i}, \text{st});$
5. $y_0 \xleftarrow{\$} \text{Eval}(0, \text{ek}_0, (x_{\text{priv},i}^{(0)})_{i=1}^{n_{\text{priv}}}, (y_{\text{s-priv},i}^{(0)})_{i=1}^{n_{\text{s-priv}}}, C);$
6. $y_1 \xleftarrow{\$} \text{Eval}(1, \text{ek}_1, (x_{\text{priv},i}^{(1)})_{i=1}^{n_{\text{priv}}}, (y_{\text{s-priv},i}^{(1)})_{i=1}^{n_{\text{s-priv}}}, (C, C_{\text{rm}}));$
7. $y \xleftarrow{\$} \text{Rec}(y_0, y_1)$

Observe that if we show that $\Pr[y = C(\vec{y}_{\text{s-priv}}) \cdot C_{\text{rm}}(\vec{x}_{\text{priv}})] \geq 1 - \text{negl}(\lambda)$ then we will have proven correctness.

Further consider all the random variables introduced by Eval (we identify the algorithmic variables and the corresponding random variables), and consider the following events:

$$\begin{aligned}
E_{\text{cst}} : & \quad \text{“sh}_{1,1}^{\text{auth}} - \text{sh}_{1,0}^{\text{auth}} = k_0 \cdot 1\text{”} \\
E_{\text{s-priv},i}^{\text{auth}} : & \quad \text{“sh}_{\text{s-priv},i,1}^{\text{auth}} - \text{sh}_{\text{s-priv},i,0}^{\text{auth}} = k_0 \cdot y_{\text{s-priv},i}\text{”} \quad (\text{for } i \in [n_{\text{s-priv}}]) \\
E_i^{\text{auth}} : & \quad \text{“sh}_{i,1}^{\text{auth}} - \text{sh}_{i,0}^{\text{auth}} = k_{d_i} \cdot P_i(\vec{y}_{\text{s-priv}})\text{”} \quad (\text{for } i \in [|C|]) \\
E'_i : & \quad \text{“sh}_{|C|+i,1} - \text{sh}_{|C|+i,0} = C(\vec{y}_{\text{s-priv}}) \cdot Q_i(\vec{x}_{\text{priv}})\text{”} \quad (\text{for } i \in [|C_{\text{rm}}|]) \\
E'_{i,\text{auth}} : & \quad \text{“sh}_{|C|+i,1}^{\text{auth}} - \text{sh}_{|C|+i,0}^{\text{auth}} = k_{d'_i} \cdot C(\vec{y}_{\text{s-priv}}) \cdot Q_i(\vec{x}_{\text{priv}})\text{”} \quad (\text{for } i \in [|C_{\text{rm}}|])
\end{aligned}$$

where for $i \in [n_{\text{s-priv}}]$, P_i is the polynomial computed by the i^{th} gate of C (and whose pebbling depth we denote d_i), while for $i \in [n_{\text{priv}}]$, Q_i is the polynomial computed by the i^{th} gate of C_{rm} (and whose pebbling depth we denote d'_i).

Let $C_+ \subseteq [|C|]$ denote the set of indices of the addition gates of C (*i.e.* $i \in C_+$ iff g_i is an addition). For all $i \in C_+$ we consider the events $E_{i,L}^{\text{KS}}$ and $E_{i,R}^{\text{KS}}$

$$\begin{aligned}
E_{i,L}^{\text{KS}} : & \quad \text{“sh}_{i,L,1}^{\text{auth},d_i} - \text{sh}_{i,L,0}^{\text{auth},d_i} = k_{d_i} \cdot P_{i_L}(\vec{y}_{\text{s-priv}})\text{”} \\
E_{i,R}^{\text{KS}} : & \quad \text{“sh}_{i,R,1}^{\text{auth},d_i} - \text{sh}_{i,R,0}^{\text{auth},d_i} = k_{d_i} \cdot P_{i_R}(\vec{y}_{\text{s-priv}})\text{”}
\end{aligned}$$

where i_L and i_R are the indices of the parents of gate g_i , and P_i, P_{i_L}, P_{i_R} are the polynomials

computed by gates g_i, g_{i_L}, g_{i_R} (recall that d_i denotes the pebbling depth of gate g_i).

Let $C_\times \subseteq [|C|]$ denote the set of indices of the multiplication gates of C (i.e. $i \in C_\times$ iff g_i is a multiplication). For all $i \in C_\times$ we consider the events $E_{i,L}^{\text{KS}}$ and $E_{i,R}^{\text{KS}}$

$$E_{i,L}^{\text{KS}}: \text{“sh}_{i,L,1}^{\text{auth},d_i-1} - \text{sh}_{i,L,0}^{\text{auth},d_i-1} = k_{d_i-1} \cdot P_{i_L}(\vec{y}_{\text{s-priv}})”$$

$$E_{i,R}^{\text{KS}}: \text{“sh}_{i,R,1}^{\text{auth},d_i} - \text{sh}_{i,R,0}^{\text{auth},d_i} = k_{d_i} \cdot P_{i_R}(\vec{y}_{\text{s-priv}})”$$

where i_L and i_R are the indices of the parents of gate g_i , and P_i, P_{i_L}, P_{i_R} are the polynomials computed by gates g_i, g_{i_L}, g_{i_R} (recall that d_i denotes the pebbling depth of gate g_i).

Lemma 50. $\Pr[E_{\text{cst}}] = 1$.

Lemma 36 follows immediately from the definition of $\text{sh}_{0,1}^{\text{auth}}$ and $\text{sh}_{0,1}^{\text{auth}}$.

Lemma 51. $\forall i \in [n_{\text{s-priv}}], \Pr[E_{\text{s-priv},i}^{\text{auth}}] \geq 1 - N^{-\varepsilon}$.

Proof of Lemma 51. By definition of the $\text{sh}_{\text{s-priv},i,1}^{\text{auth}}$ ($i \in [n_{\text{s-priv}}]$) the following relation holds with probability 1:

$$\forall i \in [n_{\text{s-priv}}], \text{sh}_{\text{s-priv},i,1}^{\text{auth}} - \text{sh}_{\text{s-priv},i,0}^{\text{auth}} \equiv k_0 \cdot y_{\text{s-priv},i} \text{ mod } N^\zeta$$

Because the offline inputs are all B -admissible, and because $k_0 \cdot B \cdot B \leq N \cdot B = N^{\zeta-\varepsilon} < N^\zeta$, we get the desired results by invoking Lemma 9 (because the $\text{sh}_{\text{s-priv},i,0}^{\text{auth}}$, $\text{sh}_{\text{priv},j,0}$, and $\text{sh}_{\text{priv},j,0}^{\text{auth}}$ are sampled uniformly at random from $\mathbb{Z}/N^\zeta\mathbb{Z}$). $\square$

Lemma 52. Let $i \in [|C|]$. If g_i (the i^{th} gate of C) is the constant gate then $\Pr[E_i^{\text{auth}}] = 1$.

Proof of Lemma 52. If g_i is the constant gate 1, then by definition $E_i^{\text{auth}} \equiv E_{\text{cst}}$ so $\Pr[E_i^{\text{auth}}] = 1$ by Lemma 50. $\square$

Lemma 53. Let $i \in [|C|]$. If g_i (the i^{th} gate of C) is the j^{th} input then $\Pr[E_i^{\text{auth}} | E_{\text{s-priv},j}^{\text{auth}}] = 1$.

Proof of Lemma 53. If g_i is the j^{th} input gate $y_{\text{s-priv},j}$, then by definition $E_i^{\text{auth}} \equiv E_{\text{s-priv},j}^{\text{auth}}$ so $\Pr[E_i^{\text{auth}} | E_{\text{s-priv},j}^{\text{auth}}] = 1$ because any event is certain when conditioned on itself. $\square$

Lemma 54. Let $i \in [|C|]$. If g_i (the i^{th} gate of C) is a scalar multiplication with parent g_j , then $\Pr[E_i^{\text{auth}} | E_j^{\text{auth}}] = 1$.

Proof of Lemma 54. If g_i is the scalar multiplication $\cdot_\alpha$ with predecessor g_j , note that by definition of the pebbling depth g_j and g_i have the same pebbling depth d_i . Therefore, conditioned on E_j :

$$\begin{aligned} \text{sh}_{i,1}^{\text{auth}} - \text{sh}_{i,0}^{\text{auth}} &= \alpha \cdot \text{sh}_{j,1}^{\text{auth}} - \alpha \cdot \text{sh}_{j,0}^{\text{auth}} \\ &= \alpha \cdot (\text{sh}_{j,1}^{\text{auth}} - \text{sh}_{j,0}^{\text{auth}}) \\ &= \alpha \cdot k_{d_i} \cdot P_j(\vec{y}_{\text{s-priv}}) \\ &= k_{d_i} \cdot P_i(\vec{y}_{\text{s-priv}}) \end{aligned}$$

Therefore $\Pr[E_i^{\text{auth}} | E_j^{\text{auth}}] = 1$. $\square$

Lemma 55. For all $i \in C_+$, if the parents of (the addition) g_i are g_{i_L} and g_{i_R} , $\Pr[E_{i_L}^{\text{KS}} | E_{i_L}^{\text{auth}}] \geq 1 - (d_i - d_{i_L}) \cdot N^{-\varepsilon}$ and $\Pr[E_{i_R}^{\text{KS}} | E_{i_R}^{\text{auth}}] \geq 1 - (d_i - d_{i_R}) \cdot N^{-\varepsilon}$ (where i_L and i_R are the indices of the parents of multiplication g_i , and d_i, d_{i_L}, d_{i_R} are the pebbling depths of g_i, g_{i_L}, g_{i_R}).

Proof of Lemma 55. Let $i \in C_+$, and let i_L and i_R be the indices of the parents of addition g_i . Let d_i, d_{i_L}, d_{i_R} be the pebbling depths of g_i, g_{i_L}, g_{i_R} . By definition, $\text{sh}_{i_L,1} - \text{sh}_{i_L,0} = P_{i_L}(\vec{y}_{\text{s-priv}})$ and $\text{sh}_{i_R,1} - \text{sh}_{i_R,0} = P_{i_R}(\vec{y}_{\text{s-priv}})$, so, by Lemma 21:

$$\begin{aligned} \Pr \left[\text{sh}_{i_L,1}^{\text{auth},d_i} - \text{sh}_{i_L,0}^{\text{auth},d_i} = k_{d_i} \cdot P_{i_L}(\vec{y}_{\text{s-priv}}) | E_{i_L}^{\text{auth}} \right] &\geq 1 - (d_i - d_{i_L}) \cdot N^{-\varepsilon} \\ \Pr \left[\text{sh}_{i_R,1}^{\text{auth},d_i} - \text{sh}_{i_R,0}^{\text{auth},d_i} = k_{d_i} \cdot P_{i_R}(\vec{y}_{\text{s-priv}}) | E_{i_R}^{\text{auth}} \right] &\geq 1 - (d_i - d_{i_R}) \cdot N^{-\varepsilon} . \end{aligned}$$

□

Lemma 56. For all $i \in C_+$, $\Pr[E_i^{\text{auth}} | E_{i_L}^{\text{KS}}, E_{i_R}^{\text{KS}}] = 1$.

Proof of Lemma 56. Let $i \in C_+$. If $E_{i_L}^{\text{KS}}$ and $E_{i_R}^{\text{KS}}$ then

$$\begin{aligned} \text{sh}_{i,1}^{\text{auth}} - \text{sh}_{i,0}^{\text{auth}} &= (\text{sh}_{i_R,1}^{\text{auth},d_i} + \text{sh}_{i_L,1}^{\text{auth},d_i}) - (\text{sh}_{i_R,0}^{\text{auth},d_i} + \text{sh}_{i_L,0}^{\text{auth},d_i}) \\ &= (\text{sh}_{i_L,1}^{\text{auth},d_i} - \text{sh}_{i_L,0}^{\text{auth},d_i}) + (\text{sh}_{i_R,1}^{\text{auth},d_i} - \text{sh}_{i_R,0}^{\text{auth},d_i}) \\ &= k_{d_i} \cdot P_{i_L}(\vec{y}_{\text{s-priv}}) + k_{d_i} \cdot P_{i_R}(\vec{y}_{\text{s-priv}}) \\ &= k_{d_i} \cdot P_i(\vec{y}_{\text{s-priv}}) \end{aligned}$$

□

Lemma 57. For all $i \in C_\times$, $\Pr[E_{i_L}^{\text{KS}} | E_{i_L}^{\text{auth}}] \geq 1 - (d_i - d_{i_L} - 1) \cdot N^{-\varepsilon}$ and $\Pr[E_{i_R}^{\text{KS}} | E_{i_R}^{\text{auth}}] \geq 1 - (d_i - d_{i_R}) \cdot N^{-\varepsilon}$ (where i_L and i_R are the indices of the parents of multiplication g_i , and d_i, d_{i_L}, d_{i_R} are the pebbling depths of g_i, g_{i_L}, g_{i_R}).

Proof of Lemma 57. By definition, $\text{sh}_{i_L,1} - \text{sh}_{i_L,0} = P_{i_L}(\vec{y}_{\text{s-priv}})$ and $\text{sh}_{i_R,1} - \text{sh}_{i_R,0} = P_{i_R}(\vec{y}_{\text{s-priv}})$, so, by Lemma 21:

$$\begin{aligned} \Pr \left[\text{sh}_{i_L,1}^{\text{auth},d_i-1} - \text{sh}_{i_L,0}^{\text{auth},d_i-1} = k_{d_i-1} \cdot P_{i_L}(\vec{y}_{\text{s-priv}}) | E_{i_L}^{\text{auth}} \right] &\geq 1 - (d_i - 1 - d_{i_L}) \cdot N^{-\varepsilon} \\ \Pr \left[\text{sh}_{i_R,1}^{\text{auth},d_i} - \text{sh}_{i_R,0}^{\text{auth},d_i} = k_{d_i} \cdot P_{i_R}(\vec{y}_{\text{s-priv}}) | E_{i_R}^{\text{auth}} \right] &\geq 1 - (d_i - d_{i_R}) \cdot N^{-\varepsilon} . \end{aligned}$$

□

Lemma 58. For all $i \in C_\times$, $\Pr[E_i | E_{i_L}^{\text{KS}}, E_{i_R}^{\text{KS}}, E_{\text{corr}}] \geq 1 - N^{-\varepsilon}$.

Proof of Lemma 58. Let $i \in [C']$ such that g_i is a multiplication gate whose two operands are g_{i_L} and g_{i_R} (with $i_L, i_R < i$). Denote P_{i_L} and P_{i_R} the polynomials computed by g_{i_L} and g_{i_R} respectively. By core lemma 1, conditioned on $E_{i_L}^{\text{KS}}, E_{i_R}^{\text{KS}}$, and E_{corr} :

$$\begin{aligned} &\left[-\text{DDLog} \left((c_{d_i-1})^{P_{i_L}(\vec{y}_{\text{s-priv}}) \cdot P_{i_R}(\vec{y}_{\text{s-priv}})} + (h_i)^{P_{i_L}(\vec{y}_{\text{s-priv}}) \cdot \text{sh}_{i_R,1}^{\text{auth},d_i}} \right. \right. \\ &\quad \left. \left. + (h_{i+1})^{P_{i_R}(\vec{y}_{\text{s-priv}}) \cdot \text{sh}_{i_L,1}^{\text{auth},d_i-1}} + g^{-\text{sh}_{i_L,1}^{\text{auth},d_i-1} \cdot \text{sh}_{i_R,1}^{\text{auth},d_i}} \right) \right] \\ &\quad - \left[-\text{DDLog} \left(g^{-\text{sh}_{i_L,0}^{\text{auth},d_i-1} \cdot \text{sh}_{i_R,0}^{\text{auth},d_i}} \right) \right] \equiv k_{d_i} \cdot P_i(\vec{y}_{\text{s-priv}}) \pmod{N^\zeta} \end{aligned}$$

We conclude, by Lemma 9 (because $k_{d_i} \cdot P_i(\vec{y}_{\text{s-priv}}) \leq N \cdot B \leq N^{\zeta-\varepsilon}$), that $\Pr[E_i | E_{i_L}^{\text{KS}}, E_{i_R}^{\text{KS}}, E_{\text{corr}}] \geq 1 - N^{-\varepsilon}$. □

Lemma 59. Let $i \in [C_{\text{rm}}]$. If g'_i (the i^{th} gate of C_{rm}) is the constant gate 1, then $\Pr[E'_i] = 1$ and $\Pr[E'_{i,\text{auth}} | E_{[C]}] = 1$.

Lemma 59 follows immediately from the definitions of $\text{sh}_{|C|+i,1}$, $\text{sh}_{|C|+i,0}$, $\text{sh}_{|C|+i,1}^{\text{auth}}$, and $\text{sh}_{|C|+i,0}^{\text{auth}}$.

Lemma 60. *Let $i \in [|C_{\text{rm}}|]$. If g'_i (the i^{th} gate of C_{rm}) is the addition of gates g'_{i_L} and g'_{i_R} , then $\Pr[E'_i|E'_{i_L}, E'_{i_R}] = 1$ and $\Pr[E'_{i,\text{auth}}|E'_{i_L,\text{auth}}, E'_{i_R,\text{auth}}] = 1$.*

Proof of Lemma 60. Conditioned on E'_{i_L} and E'_{i_R} ,

$$\begin{aligned} \text{sh}_{|C|+i,1} - \text{sh}_{|C|+i,0} &= (\text{sh}_{|C|+i_L,1} + \text{sh}_{|C|+i_R,1}) - (\text{sh}_{|C|+i_L,0} + \text{sh}_{|C|+i_R,0}) \\ &= (\text{sh}_{|C|+i_L,1} - \text{sh}_{|C|+i_L,0}) + (\text{sh}_{|C|+i_R,1} - \text{sh}_{|C|+i_R,0}) \\ &= C(\vec{y}_{\text{s-priv}}) \cdot P_{i_L}(\vec{x}_{\text{priv}}) + C(\vec{y}_{\text{s-priv}}) \cdot P_{i_R}(\vec{x}_{\text{priv}}) \\ &= C(\vec{y}_{\text{s-priv}}) \cdot (P_{i_L} + P_{i_R})(\vec{x}_{\text{priv}}) \\ &= C(\vec{y}_{\text{s-priv}}) \cdot P_i(\vec{x}_{\text{priv}}) \end{aligned}$$

and therefore $\Pr[E'_i|E'_{i_L}, E'_{i_R}] = 1$.

Similarly, conditioned on $E'_{i_L,\text{auth}}$ and $E'_{i_R,\text{auth}}$,

$$\begin{aligned} \text{sh}_{|C|+i,1}^{\text{auth}} - \text{sh}_{|C|+i,0}^{\text{auth}} &= (\text{sh}_{|C|+i_L,1}^{\text{auth}} + \text{sh}_{|C|+i_R,1}^{\text{auth}}) - (\text{sh}_{|C|+i_L,0}^{\text{auth}} + \text{sh}_{|C|+i_R,0}^{\text{auth}}) \\ &= (\text{sh}_{|C|+i_L,1}^{\text{auth}} - \text{sh}_{|C|+i_L,0}^{\text{auth}}) + (\text{sh}_{|C|+i_R,1}^{\text{auth}} - \text{sh}_{|C|+i_R,0}^{\text{auth}}) \\ &= k_D \cdot C(\vec{y}_{\text{s-priv}}) \cdot P_{i_L}(\vec{x}_{\text{priv}}) + k_D \cdot C(\vec{y}_{\text{s-priv}}) \cdot P_{i_R}(\vec{x}_{\text{priv}}) \\ &= k_D \cdot C(\vec{y}_{\text{s-priv}}) \cdot (P_{i_L} + P_{i_R})(\vec{x}_{\text{priv}}) \\ &= k_D \cdot C(\vec{y}_{\text{s-priv}}) \cdot P_i(\vec{x}_{\text{priv}}) \end{aligned}$$

and therefore $\Pr[E'_{i,\text{auth}}|E'_{i_L,\text{auth}}, E'_{i_R,\text{auth}}] = 1$. □

Lemma 61. *Let $i \in [|C_{\text{rm}}|]$. If g'_i (the i^{th} gate of C_{rm}) is a scalar multiplication gate with parent g'_j , then $\Pr[E'_i|E'_j] = 1$ and $\Pr[E'_{i,\text{auth}}|E'_j] = 1$.*

Proof of Lemma 61. Conditioned on E'_j ,

$$\begin{aligned} \text{sh}_{|C|+i,1} - \text{sh}_{|C|+i,0} &= (\alpha \cdot \text{sh}_{|C|+j,1}) - (\alpha \cdot \text{sh}_{|C|+j,0}) \\ &= \alpha \cdot (\text{sh}_{|C|+j,1} - \text{sh}_{|C|+j,0}) \\ &= \alpha \cdot C(\vec{y}_{\text{s-priv}}) \cdot P_j(\vec{x}_{\text{priv}}) \\ &= C(\vec{y}_{\text{s-priv}}) \cdot (\alpha \cdot P_j)(\vec{x}_{\text{priv}}) \\ &= C(\vec{y}_{\text{s-priv}}) \cdot P_i(\vec{x}_{\text{priv}}) \end{aligned}$$

and therefore $\Pr[E'_i|E'_j] = 1$.

Similarly, conditioned on $E'_{j,\text{auth}}$,

$$\begin{aligned} \text{sh}_{|C|+i,1}^{\text{auth}} - \text{sh}_{|C|+i,0}^{\text{auth}} &= (\alpha \cdot \text{sh}_{|C|+j,1}^{\text{auth}}) - (\alpha \cdot \text{sh}_{|C|+j,0}^{\text{auth}}) \\ &= \alpha \cdot (\text{sh}_{|C|+j,1}^{\text{auth}} - \text{sh}_{|C|+j,0}^{\text{auth}}) \\ &= \alpha \cdot k_D \cdot C(\vec{y}_{\text{s-priv}}) \cdot P_j(\vec{x}_{\text{priv}}) \\ &= k_D \cdot C(\vec{y}_{\text{s-priv}}) \cdot (\alpha \cdot P_j)(\vec{x}_{\text{priv}}) \\ &= k_D \cdot C(\vec{y}_{\text{s-priv}}) \cdot P_i(\vec{x}_{\text{priv}}) \end{aligned}$$

and therefore $\Pr[E'_{i,\text{auth}}|E'_{j,\text{auth}}] = 1$. □

Lemma 62. *Let $i \in [|C_{\text{rm}}|]$. If g'_i (the i^{th} gate of C_{rm}) is the j^{th} input gate, then $\Pr[E'_i|E_{|C|}] \geq 1 - N^{-\epsilon}$ and $\Pr[E'_{i,\text{auth}}|E_{|C|}] \geq 1 - N^{-\epsilon}$.*

Proof of Lemma 62. By Lemma 8, conditioned on $E_{|C|}$,

$$\text{DDLog}((c_{j,0}^{\text{sh}_{|C|,1}^{\text{auth}}}) \cdot (c_{j,1}^{\text{sh}_{|C|,1}^{\text{auth}}})) - \text{DDLog}((c_{j,0}^{\text{sh}_{|C|,0}^{\text{auth}}}) \cdot (c_{j,1}^{\text{sh}_{|C|,0}^{\text{auth}}})) \equiv C(\vec{y}_{\text{s-priv}}) \cdot x_{\text{priv},j} \bmod [N^\zeta]$$

and

$$\begin{aligned} & \text{DDLog}(((c_{j,0}^{\text{auth}})^{\text{sh}_{|C|,1}^{\text{auth}}}) \cdot ((c_{j,1}^{\text{auth}})^{\text{sh}_{|C|,1}^{\text{auth}}})) \\ & - \text{DDLog}(((c_{j,0}^{\text{auth}})^{\text{sh}_{|C|,0}^{\text{auth}}}) \cdot ((c_{j,1}^{\text{auth}})^{\text{sh}_{|C|,0}^{\text{auth}}})) \equiv k_D \cdot C(\vec{y}_{\text{s-priv}}) \cdot x_{\text{priv},j} \bmod [N^\zeta] \end{aligned}$$

Because $C(\vec{y}_{\text{s-priv}}) \cdot x_{\text{priv},j}$, $k_D \cdot C(\vec{y}_{\text{s-priv}}) \cdot x_{\text{priv},j} \leq N \cdot B^2 \leq N^{\zeta-\varepsilon}$, the desired results therefore follow from Lemma 9. $\square$

Lemma 63. Let $i \in [|C_{\text{rm}}|]$. If g'_i (the i^{th} gate of C_{rm}) is a multiplication of the j^{th} input gate and gate g'_{i_R} , then $\Pr[E'_i | E'_j, E_{|C|+i_R}] \geq 1 - N^{-\varepsilon}$ and $\Pr[E'_{i,\text{auth}} | E'_j, E_{|C|+i_R}] \geq 1 - N^{-\varepsilon}$.

Proof of Lemma 63. By Lemma 8, conditioned on $E_{|C|+i_R}$,

$$\text{DDLog}((c_{j,0}^{\text{sh}_{|C|+i_R,1}^{\text{auth}}}) \cdot (c_{j,1}^{\text{sh}_{|C|+i_R,1}^{\text{auth}}})) - \text{DDLog}((c_{j,0}^{\text{sh}_{|C|+i_R,0}^{\text{auth}}}) \cdot (c_{j,1}^{\text{sh}_{|C|+i_R,0}^{\text{auth}}})) \equiv C(\vec{y}_{\text{s-priv}}) \cdot P_{i_R}(\vec{x}_{\text{priv}}) \bmod [N^\zeta]$$

and

$$\begin{aligned} & \text{DDLog}(((c_{j,0}^{\text{auth}})^{\text{sh}_{|C|+i_R,1}^{\text{auth}}}) \cdot ((c_{j,1}^{\text{auth}})^{\text{sh}_{|C|+i_R,1}^{\text{auth}}})) \\ & - \text{DDLog}(((c_{j,0}^{\text{auth}})^{\text{sh}_{|C|+i_R,0}^{\text{auth}}}) \cdot ((c_{j,1}^{\text{auth}})^{\text{sh}_{|C|+i_R,0}^{\text{auth}}})) \equiv k_D \cdot C(\vec{y}_{\text{s-priv}}) \cdot P_{i_R}(\vec{x}_{\text{priv}}) \bmod [N^\zeta] \end{aligned}$$

Because $C(\vec{y}_{\text{s-priv}}) \cdot P_{i_R}(\vec{x}_{\text{priv}})$, $k_D \cdot C(\vec{y}_{\text{s-priv}}) \cdot P_{i_R}(\vec{x}_{\text{priv}}) \leq N \cdot B^2 \leq N^{\zeta-\varepsilon}$, the desired results therefore follow from Lemma 9. $\square$

Lemma 64. Correctness with an idealised PRF (i.e. the event “ $y = C(\vec{y}_{\text{s-priv}}) \cdot C_{\text{rm}}(\vec{x}_{\text{priv}})$ ”) holds with all but negligible probability.

Proof of Lemma 64. The proof is completely analogous to that of Lemma 48. We first rename/order the events $E_{\text{cst}}, E_{\text{s-priv},1}^{\text{auth}}, \dots, E_{\text{s-priv},n_{\text{s-priv}}}^{\text{auth}}, E_1^{\text{auth}}, \dots, E_{|C|}^{\text{auth}}, E'_1, E'_{1,\text{auth}}, \dots, E'_{|C_{\text{rm}}|}, E'_{|C_{\text{rm}}|,\text{auth}}$ as $A_1, \dots, A_{n_{\text{s-priv}}+|C|+2|C_{\text{rm}}|+1}$.

$$\Pr[E'_{|C_{\text{rm}}|}] \geq 1 - \sum_{i=1}^{n_{\text{s-priv}}+|C|+2|C_{\text{rm}}|+1} \Pr[\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j]$$

Note that:

- If $i = 1$, $\Pr[\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j]$ is negligible by Lemma 50.
- If $2 \leq i \leq n_{\text{s-priv}} + 1$, $\Pr[\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j]$ is negligible by Lemma 51.
- If $2 + n_{\text{s-priv}} + 1 \leq n \leq 2 + n_{\text{s-priv}} + |C|$, $\Pr[\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j]$ is negligible by Lemmata 52 to 58 (the precise lemmata to be invoked depend on the nature of the i^{th} gate of C).
- If $2 + n_{\text{s-priv}} + |C| + 1 \leq n \leq 2 + n_{\text{s-priv}} + |C| + 2|C_{\text{rm}}|$, $\Pr[\overline{A_i} \cap \bigcap_{1 \leq j < i} A_j]$ is negligible by Lemmata 59 to 63 (the precise lemmata to be invoked depend on the nature of the i^{th} gate of C_{rm}).

Therefore there is a negligible function negl such that $\Pr[E'_{|C_{\text{rm}}|,\text{auth}}] \geq 1 - \text{negl}$. $\square$

Real PRF. We now no longer assume that PRF is instantiated as a truly random function. Consider the adversary which, tasked with distinguishing outputs of PRF and a random oracle, runs the “HSS correctness experiment” and outputs 1 if and only if correctness holds. By security of PRF, the advantage of this adversary can be at most ε_{PRF} , and since we have already established that correctness (respectively authenticated correctness) in the ideal world holds with probability at least $p \cdot (1 - N^{-\varepsilon})$ (respectively p), where $p = 1 - \text{negl}$, it follows that correctness (respectively authenticated correctness) in the real world holds with all but negligible probability.

A.2.2 Key-switching security.

Lemma 20. *Let $N \xleftarrow{\$} \text{RSA.Gen}(1^\lambda)$, $g \xleftarrow{\$} (\mathbb{Z}/N^{\zeta+1}\mathbb{Z})^\times$, $x, y \xleftarrow{\$} [0, N)$ and $r \xleftarrow{\$} [0, N^\zeta)$. Then, the distributions*

$$(N, g, g^x, g^y, g^{xy} \cdot \exp(y)) \quad \text{and} \quad (N, g, g^x, g^y, g^{xy} \cdot \exp(r))$$

are computationally indistinguishable under the DCR assumption.

Proof. We give a hybrid proof, starting from the real distribution $(N, g, g^x, g^y, g^{xy} \cdot \exp(y))$.

- $\mathcal{H}_1$. Let $g_N = g^{N^\zeta}$. Use DCR to replace g with g_N in the distribution. ⁵
- $\mathcal{H}_2$. Sample $x \xleftarrow{\$} [0, 2^\lambda N^{\zeta+1})$ instead of $x \xleftarrow{\$} [0, N)$. This is indistinguishable because x is only used in the exponent of g_N , which has order dividing φ , and both distributions have $x \bmod \varphi$ statistically indistinguishable from uniform. $x \xleftarrow{\$} [0, N)$ can be divided into $x \in [0, \varphi)$, where $x \bmod \varphi$ is uniform, and $x \in [\varphi, N)$, which has negligible probability $\frac{N-\varphi}{N} < \frac{p+q}{N} = O(N^{-\frac{1}{2}})$. Here, p and q are the prime factors of N , which are roughly of order $N^{\frac{1}{2}}$. Similarly, $x \xleftarrow{\$} [0, 2^\lambda N^{\zeta+1})$ can be divided into $x \in [0, 2^\lambda N^{\zeta+1} - (2^\lambda N^{\zeta+1} \bmod \varphi))$, where $x \bmod \varphi$ is uniform, and $x \in [2^\lambda N^{\zeta+1} - (2^\lambda N^{\zeta+1} \bmod \varphi), 2^\lambda N^{\zeta+1})$, which has negligible probability less than $\frac{\varphi}{2^\lambda N^{\zeta+1}} < 2^{-\lambda}$.
- $\mathcal{H}_3$. Set $g_N = g^{N^\zeta} \exp(1)$. By DCR, both distributions for g_N are indistinguishable from uniform.
- $\mathcal{H}_4$. Let $d = \varphi(\varphi^{-1} \bmod N^\zeta)$, and replace x with $x - d$. This has statistical distance $\frac{d}{2^\lambda N^{\zeta+1}} < 2^{-\lambda}$. We can then simplify using

$$g_N^{x-d} = g_N^x g_N^{-d} = g_N^x (g^{N^\zeta} \exp(1))^{-d} = g_N^x \exp(-d) = g_N^x \exp(-1),$$

since g^{N^ζ} has order dividing $\varphi \mid d$, and $d \equiv 1 \pmod{N^\zeta}$. Therefore, $g_N^{(x-d)y} \cdot \exp(y) = g_N^{xy} \cdot \exp(-y) \cdot \exp(y) = g_N^{xy}$. The distribution is now $(N, g_N, g_N^x \exp(-1), g_N^y, g_N^{xy})$.

- $\mathcal{H}_5$. Use DCR to set g_N back to g^{N^ζ} .
- $\mathcal{H}_6$. Sample $y \xleftarrow{\$} [0, 2^\lambda N^{\zeta+1})$ instead of $y \xleftarrow{\$} [0, N)$. This is indistinguishable by the same argument as $\mathcal{H}_2$.
- $\mathcal{H}_7$. Use DCR to again set g_N to $g^{N^\zeta} \exp(1)$.
- $\mathcal{H}_8$. Replace x by $x + d$, and simplify using $g_N^{x+d} = g_N^x \exp(1)$. This is statistically indistinguishable, same as $\mathcal{H}_4$. The distribution is now $(N, g_N, g_N^x, g_N^y, g_N^{xy} \exp(y))$.
- $\mathcal{H}_9$. DCR to again set g_N back to g^{N^ζ} . We're essentially back at $\mathcal{H}_1$, except that x and y are now sampled uniformly from $[0, 2^\lambda N^{\zeta+1})$ instead of $[0, N)$.

⁵ While this is only directly DCR when $\zeta = 1$, the assumption is equivalent to a version where ζ is any $\text{poly}(\lambda)$. [DJ01]

- $\mathcal{H}_{10}$. y is used in two places: in the exponent of g_N^y , and in $\exp(y)$. The former only depends on $y \bmod \varphi$, while the latter only depends on $y \bmod N^\zeta$. Since $y \bmod \varphi N^\zeta$ has statistical distance at most $\frac{\varphi N^\zeta}{2^{\lambda N^\zeta+1}} < 2^{-\lambda}$ from uniform, we can use CRT to replace y with two variables: $y \xleftarrow{\$} [0, \varphi)$ used for g_N^y , and $r \xleftarrow{\$} N^\zeta$ used for $\exp(r)$. The distribution is now $(N, g_N, g_N^x, g_N^y, g_N^{xy} \exp(r))$.
- $\mathcal{H}_{11}$. Replace $x \xleftarrow{\$} [0, 2^\lambda N^\zeta+1)$ and $y \xleftarrow{\$} [0, \varphi)$ by $x, y \xleftarrow{\$} [0, N)$. This is indistinguishable by a similar argument to $\mathcal{H}_1$.
- $\mathcal{H}_{12}$. Use DCR to replace g_N with g . We are now at the random distribution. $\square$

A.2.3 Security against party $\sigma = 0$. We need to simulate the view of party $\sigma = 0$ using just the numbers $n_{\text{priv}}, n_{\text{s-priv}}$. We present a sequence of hybrids, starting from the real world, and ending in world where the view of party 0 will depend only on these values. This last world becomes the simulation.

- $\mathcal{H}_1$. This is the real world. The adversary sees $\text{ek}_0 \leftarrow (N, g, (h_i)_{i=1}^D, (c_i)_{i=0}^{D-1}, \text{sh}_{1,0}^{\text{auth}}, k_{\text{PRF}}), x_i^{(0)}$ (output by $\text{Share}_{\text{priv}}$) for $i \in [n_{\text{priv}}]$, and $y_i^{(0)}$ (output by $\text{Share}_{1,\text{s-priv}}$) for $i \in n_{\text{s-priv}}$. Note that $\text{HSS.Share}_{0,\text{s-priv}}$ is not provided y_i as an input, so the only thing that depends on private information is the ciphertexts $x_i^{(0)} = (c_{x_i}, c_{x_i}^{\text{auth}})$, so this is what we must show can be simulated.
- $\mathcal{H}_2$. Perform a sequence of hybrids $\mathcal{H}_2^0 \dots, \mathcal{H}_2^D$. Let $\mathcal{H}_2^0 = \mathcal{H}_1$ be the real world and let $\mathcal{H}_2 = \mathcal{H}_2^D$. In the change from $\mathcal{H}_2^i$ to $\mathcal{H}_2^{i+1}$, we replace $c_i \leftarrow g^{k_i k_{i+1}} \cdot \exp(k_{i+1})$ with $c_i \leftarrow g^{k_i k_{i+1}} \cdot \exp(r_{i+1})$, where $r_{i+1} \xleftarrow{\$} [0, N^\zeta)$. Finally, in $\mathcal{H}_2^D = \mathcal{H}_2$ we have set every $c_i = g^{k_i k_{i+1}} \cdot \exp(r_{i+1})$.

We show for all i that the change from $\mathcal{H}_2^i$ to $\mathcal{H}_2^{i+1}$ is indistinguishable, by reduction to Lemma 20. Let $A = g^x, B = g^y, C = g^{xy} \exp(y)$ in the challenge from Lemma 20. Set $c_i \leftarrow C, h_i = A$, and $h_{i+1} = B$. Sample all k_j for $j \notin \{i, i+1\}$ as normal, and compute h_j as normal. Also compute c_j for $j \notin [i-1, i+1]$ as normal, using r_{j+1} when $j < i$. Finally, we have to compute c_{i-1} and c_{i+1} , which depend on the challenge keys $x = k_i$ and $y = k_{i+1}$. We can compute these just given A and B : $c_{i-1} = A^{k_{i-1}} \exp(r_i)$ and $c_{i+1} = B^{k_{i+2}} \exp(k_{i+2})$. These are the only places where k_i and k_{i+1} are used in the adversary's view. Therefore, a distinguisher for $\mathcal{H}_2^i$ and $\mathcal{H}_2^{i+1}$ implies a distinguisher for the real and random distributions in Lemma 20.

- $\mathcal{H}_3$. Now, the only use of k_D in the adversary view is to compute the public keys $h_D = g^{k_D}$ and $c_{D-1} = g^{k_{D-1} k_D} \exp(r_D)$, and to compute the private share ciphertexts $c_{x_i}^{\text{auth}} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}_D}(k_D \cdot x_i)$. Note that h_D is a standard Damgård-Jurik-ElGamal public key, while c_{D-1} can be computed from h_D, k_{D-1} , and $\exp(r_D)$. Therefore, we can use KDM security (theorem 6) to replace $c_{x_i}^{\text{auth}} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}_D}(k_D \cdot x)$ with encryptions of uniformly random plaintexts. Simultaneously, we replace $c_{x_i} \xleftarrow{\$} \text{DJE.Enc}_{\text{pk}_D}(x_i)$ with more encryptions of a random plaintexts.

The distribution is now independent of all private inputs x_i , and indeed of everything the HSS security simulator is not given. Therefore, we can view the adversary's view in this hybrid as the simulator.

A.2.4 Semi-private security against party $\sigma = 1$. We need to simulate the view of party $\sigma = 1$ using just the numbers $n_{\text{priv}}, n_{\text{s-priv}}$, and the values of the semi-private shares y_i . We present a sequence of hybrids, starting from the real world, and ending in world where the view of party 1 will depend only on these values. This last world becomes the simulation.

- $\mathcal{H}_1$. This is the real world. The adversary sees $\text{ek}_1 \leftarrow (N, g, (h_i)_{i=1}^D, (c_i)_{i=0}^{D-1}, \text{sh}_{1,1}^{\text{auth}}, k_{\text{PRF}}), x_i^{(1)}$

- (output by $\text{Share}_{\text{priv}}$) for $i \in [n_{\text{priv}}]$, and $y_i^{(1)} = (y_i, \text{sh}_{\text{s-priv},i,1}^{\text{auth}})$ (output by $\text{Share}_{1,\text{s-priv}}$) for $i \in n_{\text{s-priv}}$.
- $\mathcal{H}_2$. Instead of sampling $\text{sh}_{\text{s-priv},i,0}^{\text{auth}} \xleftarrow{\$} \mathbb{Z}/N^\zeta\mathbb{Z}$ in $\text{HSS.Share}_{0,\text{s-priv}}$ and setting $\text{sh}_{\text{s-priv},i,1}^{\text{auth}} \leftarrow (k_0 \cdot y_i + \text{sh}_{\text{s-priv},i,0}^{\text{auth}}) \bmod N^\zeta$ in $\text{HSS.Share}_{1,\text{s-priv}}$, sample $\text{sh}_{\text{s-priv},i,1}^{\text{auth}} \xleftarrow{\$} \mathbb{Z}/N^\zeta\mathbb{Z}$ and set $\text{sh}_{\text{s-priv},i,0}^{\text{auth}} \leftarrow (-k_0 \cdot y_i + \text{sh}_{\text{s-priv},i,1}^{\text{auth}}) \bmod N^\zeta$. This is an identical distribution.
- $\mathcal{H}_3$. Make a similar change to $\text{sh}_{1,0}^{\text{auth}}$ and $\text{sh}_{1,1}^{\text{auth}}$. Instead of sampling $\text{sh}_{1,0}^{\text{auth}} \xleftarrow{\$} \mathbb{Z}/N^\zeta\mathbb{Z}$ and setting $\text{sh}_{1,1}^{\text{auth}} \leftarrow (k_0 + \text{sh}_{1,0}^{\text{auth}}) \bmod N^\zeta$, sample $\text{sh}_{1,1}^{\text{auth}} \xleftarrow{\$} \mathbb{Z}/N^\zeta\mathbb{Z}$ and set $\text{sh}_{1,0}^{\text{auth}} \leftarrow (-k_0 + \text{sh}_{1,1}^{\text{auth}}) \bmod N^\zeta$. Again, this is an identical distribution.
- $\mathcal{H}_4$. Notice that the adversary's view (other than y_j) is essentially the same as it would if party $\sigma = 0$ were corrupted instead. That is, the real distribution for ek_0 is the same as the previous hybrid's distribution for ek_1 , the private input ciphertexts are equal always ($x_i^{(0)} = x_i^{(1)}$), and the real distribution for $\text{sh}_{\text{s-priv},i,0}$ is the same as the previous hybrid's distribution for $\text{sh}_{\text{s-priv},i,1}$. Therefore, we can replace all of these by a simulation that depends on only n_{priv} and $n_{\text{s-priv}}$, by security against $\sigma = 0$ proven in the previous section. Putting these together with the semi-private inputs y_i revealed to the simulator, this gives a simulation of the $\sigma = 1$ adversary's view.

B Deferred Material on Masking

We now present two instantiations of admissible masking schemes for our garbling scheme.

B.1 Boolean Truth Tables

First, we consider arbitrary boolean gates with $\ell = O(\log(s))$ inputs, i.e., truth tables of size $\text{poly}(s)$. We now show that a simple XOR masking scheme is admissible with these wide-gates.

$$\mathcal{K} = \mathcal{X} = \mathcal{Y} = \{0, 1\} \quad \text{Mask}_k(x) = k \oplus x \quad \text{UnMask}_k(y) = k \oplus y \quad (4)$$

Lemma 65. *The XOR masking scheme (eq. (4)) and size $\text{poly}(s)$ boolean truth tables are admissible for $\text{RMS} \otimes \text{VP}$.*

Proof. Need to be able to evaluate a masked version of any arbitrary truth table $T[x_1, \dots, x_\ell]$ on $\ell = O(\log(s))$ bits. That is, we need to write

$$(k_1, \dots, k_\ell, k', y_1, \dots, y_\ell) \mapsto T[k_1 \oplus y_1, \dots, k_\ell \oplus y_\ell] \oplus k'$$

as an $\text{RMS} \otimes \text{VP}$ circuit over $\mathbb{Z}$:

$$\sum_{i=0}^{2^\ell-1} F_i(k_1, \dots, k_\ell, k') G_i(y_1, \dots, y_\ell).$$

We do this by setting

$$F_i = T[k_1 \oplus i_1, \dots, k_\ell \oplus i_\ell] \oplus k' \quad G_i = (1 \oplus i_1 \oplus y_1) \cdots (1 \oplus i_\ell \oplus y_\ell),$$

where $(i_1, \dots, i_\ell)$ are the bits of the ℓ -bit integer i . That is, F_i computes the masked truth table on input $y = i$, while G_i computes 1 if $x = i$ and 0 otherwise. Here, we have used that any truth table on logarithmically many inputs can be evaluated by a polynomial-sized RMS program. $\square$

B.2 Multivariate Polynomials

Next, we consider rate-1 garbling of arithmetic circuits with plaintext values in some bounded subset $[-B, B]$ of $\mathbb{Z}$, à la [MORS24]. Since both the HSS and arithmetic circuit are defined over $\mathbb{Z}$, we use additive masking over $\mathbb{Z}$.

$$\begin{aligned} \mathcal{K} &= [B, B + 2B(2^\lambda - 1)] \\ \mathcal{X} &= [-B, B] & \text{Mask}_k(x) &= x + k \\ \mathcal{Y} &= [0, 2B \cdot 2^\lambda] & \text{UnMask}_k(y) &= y - k \end{aligned} \tag{5}$$

Because $\mathbb{Z}$ is infinite, we have to make the set of masks $\mathcal{K}$ much wider than the set of plaintext values $\mathcal{X}$, and even then we only get statistical privacy. Note that while the masked values are bigger than the unmasked ones, the rate $\frac{\log_2(B)}{\log_2(B) + \lambda}$ approaches 1 if $\log_2 = \omega(\lambda)$.

Lemma 66. *Additive masking (eq. (5)) is a one-time private masking scheme (definition 24).*

Proof. Let the simulator sample $y \xleftarrow{\$} \mathcal{Y}$. For all $x \in [-B, B]$, the only time when y is more likely to be sampled as $y \xleftarrow{\$} \mathcal{Y}$ than as $y \leftarrow \text{Mask}_k(x)$ is when $y \notin \mathcal{K} + x$. Therefore, the statistical distance from the simulation is

$$\Pr_{y \xleftarrow{\$} \mathcal{Y}} [y \notin \mathcal{K} + x] = \frac{2B2^\lambda + 1 - 2B(2^\lambda - 1) - 1}{2B2^\lambda + 1} = \frac{2B}{2B2^\lambda + 1} < 2^{-\lambda}.$$

□

Next, we must show admissibility together with a class of wide-gates. Following [Cou19, BCM23], we choose our wide-gate class to be $\log(s)$ -degree multivariate polynomials that can be written explicitly with $\text{poly}(s)$ monomials.

Lemma 67. *Additive masking, together with the class of gates that can evaluate any polynomial p with degree $\log(s)$ and $\text{poly}(s)$ monomials, is admissible for $\text{RMS} \otimes \text{VP}$. That is,*

$$(k_1, \dots, k_\ell, k', y_1, \dots, y_\ell) \mapsto T[y_1 - k_1, \dots, y_\ell - k_\ell] + k'$$

can be written as an $\text{RMS} \otimes \text{VP}$ circuit

$$\sum_{i=1}^r F_i(k_1, \dots, k_\ell, k') G_i(y_1, \dots, y_\ell)$$

for some circuits F_i, G_i , and $r = \text{poly}(s)$.

Proof. Because in the end we get to sum over polynomially many terms, we only need to prove the case where p is a single monomial $x_{i_1} x_{i_2} \dots x_{i_{\log(s)}}$.⁶ Setting $x_{i_j} = \text{UnMask}_{k_{i_j}}(y_{i_j})$ and expanding, we get

$$x_{i_1} x_{i_2} \dots x_{i_{\log(s)}} = \sum_{t_1, \dots, t_{\log(s)} \in \{0,1\}} (-k_{i_1})^{1-t_1} \dots (-k_{i_{\log(s)}})^{1-t_{\log(s)}} \cdot y_{i_1}^{t_1} \dots y_{i_{\log(s)}}^{t_{\log(s)}}.$$

This is a sum of $2^{\log(s)} = \text{poly}(s)$ terms, and each is a product of a monomial in k and a monomial in y . Therefore, p can be written as an $\text{RMS} \otimes \text{VP}$ circuit. □

⁶ With the indices allowed to repeat.